

DOSSIER DE PROJET

Cinés Délices

Plateforme communautaire de recettes inspirées du cinéma et des séries

Nathalie Leduc

Titre Professionnel — Concepteur Développeur d'Applications (CDA)

RNCP 37873 - Niveau 6 — Ministère du Travail

École O'Clock — Promotion Francfort

22 mai 2026

Session de certification : juin 2026

Le jour de la présentation du projet un QR code est transmis au Jury afin qu'il puisse accéder à tout le projet

Sommaire

Sommaire	2
1. Liste des compétences mises en œuvre dans le cadre du projet	7
1.1 Vue synoptique des compétences couvertes	7
1.2 Synthèse — couverture des 11 compétences	8
2. Cahier des charges ou expression des besoins du projet	9
2.1 Présentation du projet	9
2.2 Objectifs et limites du projet	9
Objectifs (ce que l'application DOIT faire)	9
Limites (ce que l'application ne fait PAS, volontairement)	9
2.3 Définition de la cible	10
2.4 Fonctionnalités détaillées (priorisation MoSCoW)	10
2.4.1 Périmètre MVP livré	10
2.4.2 Évolutions potentielles (hors MVP)	11
2.5 Contraintes du projet	11
2.6 Navigateurs compatibles	12
2.7 Analyse des risques	12
3. Présentation de l'entreprise et du service	14
3.1 Le commanditaire	14
3.2 Le service — Promotion Francfort, Apothéose CDA	14
3.3 Ma place dans l'équipe Cinés Délices	14
3.4 Environnement humain — qui faisait quoi	14
4. Gestion de projet (planning, suivi, environnement humain, qualité)	16
4.1 Méthodologie agile — Scrum adapté	16
4.2 Planning des sprints — vue Gantt synthétique	16
4.3 Organisation du Kanban — leçons apprises Sprint 1 → Sprint 2	16
Ce que j'ai mis en place pour le Sprint 2	16
Légende des labels Kanban	17
4.4 Workflow Git — trunk-based simplifié	17
Workflow équipe (4 étapes)	17
Mon workflow personnel — un double remote	18
4.5 Objectifs de qualité — la chaîne complète	18
5. Spécifications fonctionnelles	19
5.1 Plan du site (sitemap)	19
5.2 Maquettes — du zoning à la maquette	19
5.2.1 Wireframes	19
5.2.2 Maquettes (UI/UX)	19
5.3 Diagramme des cas d'utilisation (Use Case)	19
Synthèse des UC par acteur	21
5.4 Modélisation de la base de données — Merise	21
5.4.1 MCD — Modèle Conceptuel de Données	21

5.4.2 MLD — Modèle Logique de Données	22
5.4.3 Dictionnaire de données	23
5.4.4 MPD — Diagramme de classes UML	27
5.4.5 Règles On Delete — pourquoi ces choix	28
5.4.6 Règles de gestion (RG)	29
5.5 Script de création et seed de la base de données	29
5.6 Diagrammes de séquence — cas significatifs	29
5.7 Diagramme d'activité — workflow de publication	30
5.8 Diagramme d'états — cycle de vie d'une recette	31
6. Spécifications techniques	33
6.1 Choix d'architecture — multicouche découplée	33
Décisions structurantes et justifications	33
6.2 Schéma de l'architecture technologique	33
6.3 Architecture en couches — diagramme de packages UML	34
6.4 Stack technologique — choix justifiés	34
6.4.1 Frontend (/client)	34
6.4.2 Backend (/api)	35
6.4.3 Infrastructure et outillage	36
6.5 Architecture de déploiement et CI/CD	36
Pipeline CI/CD GitHub Actions — 4 jobs	37
6.6 Choix d'éco-conception	38
6.7 Sécurité — vue d'ensemble	38
7. Réalisations les plus significatives — la création d'une recette	40
7.1 Pourquoi la création de recette ?	40
7.2 Vue d'ensemble — diagramme de séquence	40
7.3 Étapes 1-2 — Saisie côté React (CP2)	42
7.3.1 Étape 1 — La saisie (parsing tolérant des durées)	44
7.3.2 Étape 2 — L'envoi (branchement multipart / JSON + JWT)	44
7.4 Étapes 3 à 6 — Côté serveur (CP3)	46
7.4.1 Étape 3 — authMiddleware (vérification du JWT)	46
7.4.2 Étape 4 — Pipeline image (Multer mémoire + Sharp WebP)	47
7.4.3 Étape 5 — Validation Zod (schéma serveur)	48
7.4.4 Étape 6 — Controller createRecipe (orchestration)	49
7.4.5 Étape 7 — Réponse 201 Created et redirection adaptée	50
7.5 Choix structurants d'accès aux données (CP8)	51
7.5.1 \$transaction pour l'atomicité	51
7.5.2 Sélection explicite — jamais SELECT *	51
7.5.3 Index ciblés sur les requêtes fréquentes	51
7.6 Captures d'écran — du formulaire à la confirmation	51
7.7 Autres réalisations significatives (synthèse)	51
8. Éléments de sécurité de l'application	53
8.1 OWASP Top 10 — où apparaît chaque mesure dans le code	53

8.2 Validation des entrées — Zod côté serveur	53
8.3 Protection injection SQL	54
8.4 Protection XSS	54
8.5 Conformité RGPD	54
8.5.1 Bandeau de consentement cookies	55
8.5.2 Traitement des données personnelles — finalités et durées	55
8.5.3 Cycle automatique d'effacement — 11/12 mois	55
8.5.4 Emails applicatifs RGPD — Nodemailer	57
8.5.5 Pages légales	58
9. Plan de tests	58
9.1 Stratégie de tests — pyramide à 5 niveaux	58
9.2 Plan de tests détaillé — colonnes REAC	59
9.3 Exemples de tests — extraits significatifs	60
9.3.1 Test unitaire Vitest — parseTimeToMinutes	60
9.3.2 Test d'intégration sécurité — JWT falsifié sur Railway	60
10. Jeu d'essai	62
10.1 Scénario testé — workflow complet de publication	62
10.2 Tableau du jeu d'essai	62
11. Veille sécurité	63
11.1 Démarche de veille — intégrée au workflow	63
11.2 Cinq sources, cinq décisions concrètes	63
11.3 Vulnérabilités identifiées et traitées	63
11.4 Analogie — la veille comme un chef qui contrôle ses ingrédients	63
12. Glossaire technique	65
12.1 Architecture & développement	65
12.2 Frontend	65
12.3 Backend	65
12.4 Base de données	66
12.5 Sécurité & RGPD	66
12.6 Tests & Qualité	67
12.7 Déploiement & DevOps	67
12.8 Workflow & Méthodologie	67
12.9 Spécifique projet Cinés Délices	68
Sommaire des annexes	69
Annexe 1 — Cartographie technique étendue	70
Annexe 2 — Correspondances routes Frontend ↔ endpoints API	70
Annexe 3 — Planning Sprint 2, Kanban et workflow Git	70
Annexe 4 — Use Cases détaillés + diagrammes par acteur	71
Annexe 5 — Détail textuel des séquences complexes	71
Annexe 6 — Script SQL généré par Prisma migrate	71
Annexe 7 — Charte graphique complète	71
Annexe 8 — Galerie des wireframes principaux	72

8.1 Galerie complète sur GitHub	72
8.2 Sélection de 4 wireframes représentatifs	72
WF-01 — Accueil.....	72
WF-03 — Détail recette	73
WF-11 — Créer recette	73
WF-14 — Admin validation	73
Annexe 9 — Galerie des maquettes	74
9.1 Galerie complète sur GitHub	74
9.2 Sélection de 4 paires Desktop + Mobile représentatives	74
MF-01 — Accueil	74
MF-03 — Détail recette	75
MF-11 — Créer recette.....	76
MF-14 — Admin validation	77
Annexe 10 — Code source des composants UI (CP2).....	77
10.0 Accès au code complet sur GitHub	78
10.1 AuthContext.jsx — gestion centralisée de l'authentification.....	78
10.2 CreateRecipe.jsx — parseTimeToMinutes (parsing tolérant)	79
Annexe 11 — Code source des composants métier (CP3)	79
11.0 Accès au code complet sur GitHub	80
11.1 inactivityCheck.js — CRON RGPD 11/12 mois.....	80
Annexe 12 — Code source des composants d'accès aux données (CP8)	82
12.0 Accès au code complet sur GitHub	82
12.1 Fusion d'ingrédients atomique — mergeIngredients.....	82
12.2 Sélection explicite, catalogue paginé et fuzzy search.....	83
12.2.1 Sélection explicite — jamais SELECT *	83
12.2.2 Catalogue paginé avec filtres dynamiques.....	83
12.2.3 Recherche floue — Levenshtein côté client.....	84
12.2.4 Normalisation au singulier côté serveur	85
Annexe 13 — OWASP Top 10 2025 + tarteaucitron + emails RGPD	85
13.1 Référentiel OWASP Top 10 — édition 2025	85
13.2 Configuration tarteaucitron.js — conformité CNIL.....	86
13.3 Templates d'emails RGPD — Nodemailer	87
Configuration Nodemailer	87
Template — prévenance 11 mois (extrait)	88
Captures Mailtrap	88
Annexe 14 — Tests unitaires et tests d'intégration	89
14.0 Accès au code des tests sur GitHub	89
14.1 Tests unitaires Vitest — recipeUtils.test.js (extrait).....	89
14.2 Tests d'intégration sécurité — test-api-security.js (extrait)	91
14.3 Synthèse des 7 suites de tests d'intégration	92
Annexe 15 — Captures du jeu d'essai T01 → T10	92
Captures complètes du jeu d'essai sur GitHub	93

Méthodologie d'exécution	93
Cas T01 — Connexion membre	93
Cas T02 — Création recette DRAFT	93
Cas T03 — Soumission DRAFT → PENDING	93
Cas T04 — Accès admin à la liste PENDING	94
Cas T05 — Validation admin → PUBLISHED	94
Cas T06 — Réception notification membre	94
Cas T07 — Suppression non autorisée d'une recette PUBLISHED	94
Cas T08 — Refus admin avec motif	95
Cas T09 — Upload format invalide	95
Cas T10 — Accès route admin sans token	95
Synthèse du jeu d'essai	95
Annexe 16 — Déploiement Railway, Docker, CI/CD	95
16.0 Accès aux fichiers de déploiement sur GitHub	96
16.1 Workflow GitHub Actions complet — ci.yml	96
16.2 Dockerfile API — node:22-slim + Prisma 7	99
16.3 Caddyfile client — cache différencié	99
16.4 docker-compose.yml — environnement de développement	100
16.5 Variables d'environnement Railway — checklist	101
16.6 Checklist déploiement pas-à-pas	101
16.7 Problèmes résolus durant la mise en production	101
Annexe 17 — Captures d'écran : création d'une recette	102
17.1 Correspondance étape ↔ capture (cf. §7 du corps)	102

1. Liste des compétences mises en œuvre dans le cadre du projet

Cette section reprend la grille officielle des 11 compétences professionnelles (CP) du titre CDA, regroupées en 3 Activités Types (AT). Pour chaque compétence, j'indique **précisément ce que j'ai personnellement réalisé** dans le projet Cinés Délices et la section du dossier où le détail figure. L'équipe est de 5 développeurs : je distingue donc clairement **ma contribution individuelle** du travail collectif.

1.1 Vue synoptique des compétences couvertes

CP	Intitulé officiel REAC	Ma contribution dans Cinés Délices	§
AT 1 — Développer une application sécurisée			
CP1	Installer et configurer son environnement de travail en fonction du projet	Docker Compose (Node 24 + PG 18 alpine), .env.example, pnpm workspace, configuration ESLint partagée équipe	§6.1
CP2	Développer des interfaces utilisateur	React 19 + Vite + SCSS /BEM, React Router v7, AuthContext + useCallback, ProtectedRoute, parseTimeToMinutes, debounce 300/400 ms, responsive mobile-first, WCAG 2.1 AA	§7.2
CP3	Développer des composants métier	Controllers Express (recipes, admin, auth), middlewares JWT / Multer / Sharp , workflow DRAFT→PENDING→PUBLISHED, fuzzy search, CRON RGPD , rate-limiter TMDB	§7.3
CP4	Contribuer à la gestion d'un projet informatique	Scrum Master, animation des cérémonies, Kanban GitHub , 4 sprints, planning détaillé Sprint 2, feature branches, PR obligatoires, revue de code	§4
AT 2 — Concevoir et développer une application sécurisée organisée en couches			
CP5	Analyser les besoins et maquetter une application	User Stories MoSCoW , 16 wireframes en responsive (Desktop/Mobile) (j'ai homogénéisé tous les wireframes équipe), 16 maquettes UI, sitemap, use cases UML	§5
CP6	Définir l'architecture logicielle d'une application	Architecture multicouche découplée API REST + SPA, diagramme de packages UML, choix techniques justifiés, séparation 3 couches	§6
CP7	Concevoir et mettre en place une base de données relationnelle	MCD Mocodo , MLD, MPD, dictionnaire de données complet, 9 tables UUID, FK + onDelete (CASCADE / SET NULL / RESTRICT), index composites	§5.4
CP8	Développer des composants d'accès aux données SQL et NoSQL	Schéma Prisma 7 complet + seed v4, migrations versionnées, requêtes typées, \$transaction (catalogue paginé, merge ingrédients, suppression compte)	§7.4

CP	Intitulé officiel REAC	Ma contribution dans Cinés Délices	§
AT 3 — Préparer le déploiement d'une application sécurisée			
CP9	Préparer et exécuter les plans de tests d'une application	Vitest + Testing Library (recipeUtils, ProtectedRoute, CookieConsent), tests d'intégration node:assert + fetch (7 suites sur API Railway), jeu d'essai T01→T10	§9 §10
CP10	Préparer et documenter le déploiement d'une application	Guide Railway pas-à-pas, Swagger /api-docs (JSDoc sur 35+ routes), Dockerfile sur mesure, Caddyfile cache différencié	§6.5
CP11	Contribuer à la mise en production dans une démarche DevOps	Pipeline CI/CD GitHub Actions 4 jobs (lint client + api, test client, test API intégration), webhook Railway, déploiement automatique sur push main	§6.5

Lecture du tableau : chaque ligne « CP » identifie une compétence du REAC ; la colonne « Ma contribution » liste les artefacts précis que j'ai produits seule, indépendamment du travail collectif d'équipe. Les colonnes § renvoient à la section où l'on retrouve le détail technique correspondant.

1.2 Synthèse — couverture des 11 compétences

Les 11 compétences du référentiel sont couvertes par le projet Cinés Délices. Le tableau ci-dessous récapitule mon niveau d'implication personnelle, à distinguer du périmètre couvert par l'équipe.

Activité Type	Compétences	Mon implication personnelle
AT 1 — Application sécurisée	CP1–CP4	Conception, codage, animation Scrum — toutes les CP touchées personnellement
AT 2 — Application en couches	CP5–CP8	Modélisation BDD entièrement portée (MCD/MLD/MPD/Prisma), wireframes homogénéisés, architecture co-définie
AT 3 — Déploiement DevOps	CP9–CP11	Pipeline CI/CD construit, guide Railway rédigé, tests d'intégration écrits

2. Cahier des charges ou expression des besoins du projet

Cinés Délices est un projet **pédagogique** mené dans le cadre de l'Apothéose CDA de l'école O'Clock. Le commanditaire est **fictif** — une entreprise de divertissement souhaitant proposer une expérience à la croisée de la gastronomie et de la culture populaire — mais **les contraintes (délais, équipe, livrables, sécurité, RGPD) sont, elles, bien réelles** et imposées par le cadre certificatif. Cette section précise ce que le projet doit faire, pour qui, dans quelles limites, et avec quelles contraintes.

2.1 Présentation du projet

Cinés Délices est une application web communautaire de partage de recettes de cuisine inspirées du cinéma et des séries télévisées. Chaque recette est associée à un film ou une série : la ratatouille de Remy, le thé du Chapelier Fou, le bœuf bourguignon de Julie & Julia.

Dimension	Détail
Quoi ?	Application web communautaire de recettes inspirées du cinéma et des séries
Qui ?	Entreprise fictive de divertissement (streaming, production cinématographique)
Pour qui ?	Amateurs de cuisine et de cinéma, curieux gourmands, fans de films et séries
Quand ?	Sprint 0 (9 mars 2026) → Sprint 3 + démos (3 avril 2026)
Pourquoi ?	Projet pédagogique pour l'obtention du Titre Professionnel CDA (O'Clock)

2.2 Objectifs et limites du projet

Objectifs (ce que l'application DOIT faire)

Objectif fonctionnel	Réalisation concrète
Catalogue consultable, filtrable, recherche temps réel	Pagination serveur 12/page, debounce 400 ms sur la recherche du catalogue, filtres catégorie + film/série
Workflow de contribution avec modération	Statuts DRAFT → PENDING → PUBLISHED, validation/rejet admin avec motif
Back-office administrateur complet	CRUD recettes/catégories/utilisateurs/ingrédients, notifications in-app
Application mobile-first, accessible, performante	WCAG 2.1 AA, Lighthouse ≥ 90 desktop, images WebP, conformité RGPD

Limites (ce que l'application ne fait PAS, volontairement)

- Pas de système de paiement ni de monétisation — projet pédagogique, hors périmètre.
- Pas de notifications push temps réel : les notifications sont chargées en polling au montage du composant (un push WebSocket reste une évolution possible).

- Pas d'application mobile native : la PWA / app mobile pourra être développée ultérieurement en s'appuyant sur la même API REST découplée.
- Pas de système de commentaires ou de notation : positionné en évolution potentielle (Could).

2.3 Définition de la cible

Quatre profils types ont été identifiés en Sprint 0 et ont guidé toutes les décisions d'ergonomie et d'UX.

Profil	Description	Contraintes UX
Cinéphile gourmand	18–45 ans, passionné de cinéma ET de cuisine	Navigation fluide, belles images, inspiration visuelle
Cuisinier amateur	Cherche des idées originales de recettes	Fiches claires, ingrédients lisibles, étapes ordonnées
Fan de séries	Veut retrouver sa série préférée dans les recettes	Recherche et filtres efficaces par titre de film/série
Contributeur	Souhaite partager ses créations culinaires	Formulaire simple, retour immédiat, workflow clair

2.4 Fonctionnalités détaillées (priorisation MoSCoW)

La priorisation MoSCoW (Must / Should / Could / Won't) a permis de figer le MVP dès le Sprint 0 et de protéger l'équipe contre le scope creep. Légende : ● Must (MVP), ● Should (important), ● Could (bonus).

2.4.1 Périmètre MVP livré

Fonctionnalité	Acteur	Prio
Catalogue recettes public (recherche debounce 400 ms, filtre catégorie, pagination 12/page)	Visiteur	● Must
Page détail recette (photo WebP, ingrédients+qty/unité, instructions, anecdote, film)	Visiteur	● Must
Catalogue films et séries (snapshot TMDB local, affiches WebP en cache)	Visiteur	● Must
Authentification complète (inscription, connexion, JWT, Argon2id)	Visiteur	● Must
Réinitialisation MDP par email (Nodemailer / Mailtrap , token 1 h usage unique)	Visiteur	● Should
Espace membre : CRUD recettes + workflow DRAFT→PENDING→PUBLISHED	Membre	● Must
Upload photo (Multer mémoire + Sharp → WebP –70 % poids)	Membre	● Must

Fonctionnalité	Acteur	Prio
Autocomplétion TMDb (debounce 300 ms + snapshot local + cache node-cache)	Membre	● Must
Fuzzy search ingrédients (contains insensible + normalisation singulier)	Membre	● Must
Back-office admin : validation/rejet + motif + notifications	Admin	● Must
Back-office admin : CRUD catégories, merge ingrédients doublons, gestion users	Admin	● Must
Notifications in-app (admin soumission reçue, membre statut changé)	Admin + Membre	● Should
Job RGPD automatique (node-cron , nettoyage comptes inactifs > 1 an)	Système	● Should
Gestion cookies RGPD (tarteacitronjs , bandeau CNIL , consentement explicite)	Visiteur	● Should
Pages légales RGPD (mentions, confidentialité, cookies, modération)	Visiteur	● Should
CI/CD GitHub Actions avec tests automatisés	DevOps	● Should

2.4.2 Évolutions potentielles (hors MVP)

Fonctionnalité	Prio
Commentaires / likes / notation des recettes	● Could
Système de favoris (sauvegarder des recettes)	● Could
Recommandations personnalisées	● Could
Liste de courses générée depuis plusieurs recettes	● Could
Dashboard de suivi (emails envoyés, suppressions, logs)	● Could
Notifications push temps réel (WebSockets) — actuellement en polling	● Could

2.5 Contraintes du projet

Contrainte	Détail
Technique	Stack imposée commune à toute l'équipe : React + Node.js + Express + PostgreSQL + Prisma. Docker en dev, Railway en prod.
Délai	4 semaines : Sprint 0 (lundi 9 mars 2026) → S1 (13 mars) → S2 (20 mars) → S3 (27 mars) → Démon (3 avril 2026). Suivi hebdomadaire O'Clock chaque mercredi.

Contrainte	Détail
Équipe	5 développeurs en parallèle — conventions Git strictes, PR obligatoires, revue de code.
Sécurité	Authentification JWT, hashage Argon2id, protection OWASP Top 10, conformité RGPD (France).
Accessibilité	WCAG 2.1 niveau AA, RGAA 4.1, Lighthouse ≥ 90 sur tous les axes.
Éco-conception	Images WebP (Sharp -70 % poids), pagination serveur (12/page), score EcolIndex $\geq C$.
RGPD	Consentement cookies (tarteaucitron /CNIL), pages légales, suppression compte, CRON inactivité.
Livable	Application déployée Railway, code source GitHub, dossier de projet PDF, soutenance orale.

2.6 Navigateurs compatibles

Navigateur	Version min.	Notes
Chrome	≥ 110	Support WebP natif, CSS Grid, Flexbox
Firefox	≥ 110	Support complet ES2022+
Safari (macOS / iOS)	≥ 16	Support WebP depuis Safari 14
Edge (Chromium)	≥ 110	Moteur Chromium = comportement Chrome
Chrome Mobile (Android)	≥ 110	Application mobile-first, responsive 390 px
Internet Explorer 11	Non supporté	Architecture ESM + Vite incompatible

2.7 Analyse des risques

Avant le démarrage, j'ai contribué à cartographier les risques principaux. Pour chacun, une mesure préventive a été mise en place dès le Sprint 0.

Risque	Probabilité	Impact	Mesure préventive
Conflits Git en équipe de 5	Haute	Moyen	Feature branches, PR avec 1 review min, rebases réguliers
Scope creep (périmètre glissant)	Haute	Élevé	MVP figé dès Sprint 0, évolutions en colonne dédiée Kanban
Intégration TMDB complexe	Moyenne	Moyen	Spike Sprint 1, cache node-cache, fallback saisie manuelle
Indisponibilité d'un membre	Moyenne	Élevé	Documentation continue, pair programming sur parties critiques
Faibles sécurité (XSS, SQLi)	Moyenne	Élevé	JWT + Helmet + Zod + Argon2 + Rate-limit + Prisma — protection OWASP Top 10

Risque	Probabilité	Impact	Mesure préventive
Performances catalogue large	Faible	Moyen	Pagination serveur, index BDD (status, slug, userId, categoryId)
RGPD non traité	Faible	Élevé	Pages légales, tarteaucitron CNIL, suppression compte, CRON
Déploiement défaillant	Moyenne	Élevé	Guide pas-à-pas Railway, webhook GitHub auto
Délai 4 semaines	Haute	Moyen	Sprints de 1 semaine, démos hebdomadaires, kanban précis Sprint 2

3. Présentation de l'entreprise et du service

3.1 Le commanditaire

Comme précisé au §2, le commanditaire est une entreprise fictive du secteur du divertissement (streaming et production cinématographique), souhaitant enrichir son offre en ligne par une plateforme communautaire de recettes inspirées de son catalogue.

Ce cadre fictif m'a permis de travailler avec de vraies contraintes métier : délais imposés, livrables définis, exigences RGPD, sécurité applicative et mise en production réelle sur Railway.

3.2 Le service — Promotion Francfort, Apothéose CDA

La promotion Francfort (2026) est ma promotion d'origine. L'Apothéose est le module final du cursus CDA : sur 4 semaines, des équipes de 4 à 6 stagiaires conçoivent et développent une application web complète, de la modélisation à la mise en production. Le rendu est évalué par un formateur référent et préparé pour la soutenance devant le jury de certification.

3.3 Ma place dans l'équipe Cinés Délices

Au sein de l'équipe de 5 développeurs, j'ai occupé **deux rôles complémentaires** :

- **Scrum Master** — j'ai animé les cérémonies Scrum (sprint planning, daily, rétrospective), facilité la communication, structuré le backlog et le Kanban. C'est moi qui ai préparé en amont le planning détaillé du Sprint 2 (toutes les issues codées et numérotées, branches associées) après le retour d'expérience du Sprint 1.
- **Développeuse Fullstack** — j'ai contribué à toutes les couches techniques de l'application : modélisation BDD, schéma Prisma, authentification, pipeline images, CRON RGPD, fuzzy search, parseur de temps, CI/CD, et de nombreux correctifs front-end.

3.4 Environnement humain — qui faisait quoi

Les rôles ci-dessous correspondent aux responsabilités définies au Sprint 0. Tous les membres ont contribué fullstack, mais avec des focus différents.

Membre	Rôle	Responsabilités principales
Nathalie Leduc (moi)	Fullstack + Scrum Master	Animation Scrum, facilitation, supervision backlog. Conception architecture serveur, schéma BDD, développement fullstack, tests, sécurité, CI/CD.
Hameni Abanya	Lead Backend (Sprint 0)	A participé au Sprint 0 uniquement : conception, architecture serveur, wireframes.
Vincent Duverger	Lead Frontend	Architecture frontend, design system (composants réutilisables, tokens SCSS/BEM), responsive, accessibilité WCAG/RGAA.
Orianne Jaunet	Fullstack	Développement transverse, intégration des services externes, tests automatisés (Vitest + Testing Library), audits qualité, sécurité.

Membre	Rôle	Responsabilités principales
Emilie Vatin	Fullstack	Développement de fonctionnalités fullstack, contribution UI / logique métier / API, participation aux revues de code.

4. Gestion de projet (planning, suivi, environnement humain, qualité)

4.1 Méthodologie agile — Scrum adapté

L'équipe a adopté la méthodologie Scrum adaptée au contexte pédagogique. Chaque sprint se termine par un livrable déployé : tests, déploiement et démo font partie intégrante de la Definition of Done. Echanges quotidiens et Démonstrations sur Discord et Slack. Documentations sur un drive Google dédié au projet, ou dans les PR, ou directement dans le code.

Cérémonie	Fréquence	Mon rôle (Scrum Master)
Sprint Planning	Début de sprint (vendredi)	Animation, création des issues avec labels, attribution
Daily Standup	Quotidien (15 min)	Facilitation, identification des blocages
Sprint Review / Démo	Fin de sprint (vendredi)	Présentation du travail accompli, démo live
Rétrospective	Fin de sprint	Identification des améliorations (Keep / Stop / Start)

4.2 Planning des sprints — vue Gantt synthétique

Le projet s'est déroulé sur 4 sprints d'une semaine chacun, précédés d'un Sprint 0 de conception.

Sprint	Période	Focus principal	Ma contribution clé
Sprint 0	9–13 mars 2026	Conception, architecture, maquettes, MCD/MLD/MPD	Dossier de conception, User Stories MoSCoW, MCD Mocodo, wireframes équipe, rôles équipe
Sprint 1	13–19 mars 2026	Back-end + Front-end + tests + déploiement	Schéma Prisma + seed, auth JWT + Argon2id, middlewares, RGPD 3 volets, Sharp, déploiement Render
Sprint 2	20–26 mars 2026	Finalisation fonctionnalités + correctifs + déploiement	Fuzzy search, merge ingrédients, parser temps, notifications, seed v4, migration Railway, planning Sprint 2 complet
Sprint 3	27 mars – 3 avril 2026	Polish, correctifs finaux, mise en production	Refactoring recipeUtils.js, optimisation Lighthouse, CI/CD GitHub Actions, 3 READMEs, guide Railway

4.3 Organisation du Kanban — leçons apprises Sprint 1 → Sprint 2

Le Sprint 1 a révélé trois faiblesses d'organisation : PR trop volumineuses, télescopages entre front et back, et un Kanban insuffisamment précis. Pour le Sprint 2, en tant que Scrum Master, j'ai entièrement restructuré le board.

Ce que j'ai mis en place pour le Sprint 2

- Préparation en amont d'un planning détaillé avec toutes les issues codées (C-01 à C-05 pour les correctifs critiques, B-xx pour le back, F-xx pour le front, D-xx pour le déploiement, SW-xx pour Swagger).
- Labellisation systématique de chaque issue (BACK / FRONT, CRITIQUE / CORRECTION / NEW, DEPLOY / SWAGGER, et jour cible LUNDI → VENDREDI).
- Auto-assignation par les membres : chacun se positionne selon ses disponibilités, sans attribution top-down.
- Règles claires : 100 % back en S1 → FRONT obligatoire en S2 (et inversement). Front + Back même semaine OK, mais pas le même jour.

Légende des labels Kanban

Label	Signification
BACK / FRONT	Couche technique concernée
CRITIQUE	Bloquant — à traiter en priorité absolue
CORRECTION / NEW	Correctif sur l'existant / Nouvelle fonctionnalité
DEPLOY / SWAGGER	Déploiement / Documentation API
LUNDI → VENDREDI	Jour cible dans le sprint

i Planning Sprint 2 complet (issues, branches, deadlines) et extrait Kanban GitHub en Annexe 3.

4.4 Workflow Git — trunk-based simplifié

Branches courtes feat/fix/refactor depuis develop. Develop est mergé dans main pour déclencher le déploiement Railway. Convention de commit en anglais au format conventionnel (feat:, fix:, refactor:, chore:, docs:).

Préfixe	Usage	Exemple
feat:	Nouvelle fonctionnalité	feat: add fuzzy search on ingredients
fix:	Correction de bug	fix: prevent duplicate ingredient creation
refactor:	Refactoring	refactor: centralize recipe utils
chore:	Config, dépendances	chore: add GitHub Actions workflow
docs:	Documentation	docs: update README deployment section

Workflow équipe (4 étapes)

```
# 1. Créer une branche de travail depuis dev
git checkout dev
git pull origin dev
git checkout -b fix/mon-correctif

# 2. Travailler et committer
git add .
git commit -m "fix: description du correctif"
git push -u origin fix/mon-correctif
```

```
# 3. Merger dans dev puis main (déploiement Railway)
git checkout dev && git merge fix/mon-correctif && git push origin dev
git checkout main && git merge dev && git push origin main

# 4. Nettoyer la branche
git branch -d fix/mon-correctif
git push origin --delete fix/mon-correctif
```

Mon workflow personnel — un double remote

Le dépôt École O'Clock étant en **lecture seule** (partage GitHub non accordé pour Railway), j'ai dû travailler avec **deux remotes** : *origin* (École, lecture seule) et *perso* (mon fork GitHub, lecture + écriture — utilisé pour le déploiement Railway).

i *Détail du workflow perso, commandes essentielles et erreurs fréquentes en Annexe 3.*

4.5 Objectifs de qualité — la chaîne complète

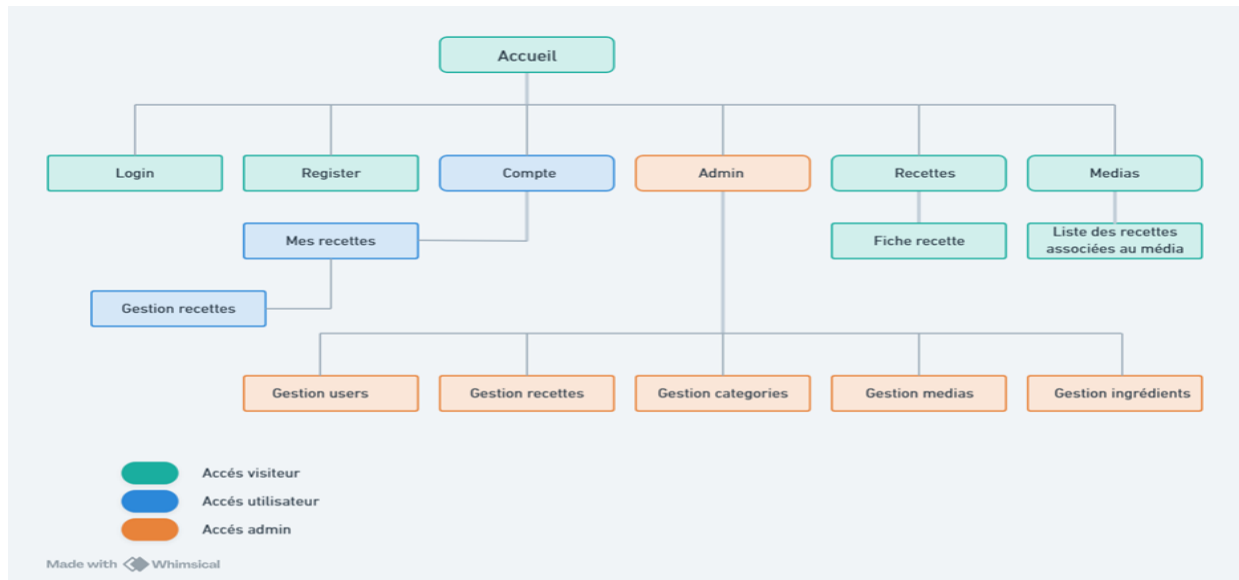
La qualité ne se résume pas à des tests qui passent ; elle est le produit d'une chaîne d'outils et de conventions appliquées tout au long du projet.

Objectif	Outil / Convention	Mise en œuvre
Conventions de code	ESLint 9 (front + back même config)	Bloque le commit si erreurs de lint
Code review	Pull Requests GitHub	1 review minimum avant merge — aucune exception
Tests unitaires	Vitest + Testing Library	7+ tests par fonction utilitaire, composants critiques couverts
Tests d'intégration	node:assert + fetch sur Railway	7 suites (auth, workflow, sécurité, RGPD, fuzzy search)
Pipeline CI	GitHub Actions (4 jobs)	Lint client + Lint API + Test client + Test API intégration
Sécurité	pnpm audit avant chaque release	0 vulnérabilité critique = critère bloquant du DoD
Documentation	Swagger /api-docs, 3 READMEs	35+ routes documentées, à jour à chaque PR
Accessibilité	Lighthouse CI	Score ≥ 90 sur tous les axes
Éco-conception	EcoIndex + analyse poids	Score ≥ C, images WebP, pagination serveur

5. Spécifications fonctionnelles

5.1 Plan du site (sitemap)

Le plan du site présente la navigation accessible à chacun des trois rôles : visiteur (vert), membre connecté (bleu), administrateur (orange).



Sitemap Cinés Délices — accès par rôle

5.2 Maquettes — du zoning à la maquette

La conception visuelle s'est faite en trois étapes : zoning (organisation des blocs), wireframes (structure détaillée), maquettes (rendu graphique final).

5.2.1 Wireframes

Lors du Sprint 0, chaque membre a réalisé 2x2 wireframes (2 Desktop + 2 Mobile), soit 20 wireframes pour 10 pages. Pour homogénéiser le rendu dans le dossier, j'ai repris tous les wireframes et adapté en responsive (Desktop/Mobile) et réalisé ceux qui manquaient : au total 16 wireframes.

i Galerie complète des 16 wireframes (Desktop/Mobile) en Annexe 8.

5.2.2 Maquettes (UI/UX)

Même travail pour les maquettes UI : 16 maquettes en responsive (Desktop + Mobile).

i Galerie complète des 16 maquettes (Desktop/Mobile) en Annexe 9.

5.3 Diagramme des cas d'utilisation (Use Case)

Le diagramme ci-dessous présente les 25 cas d'utilisation regroupés par rôle (visiteur, membre, admin) et leurs relations (include, extend, héritage).

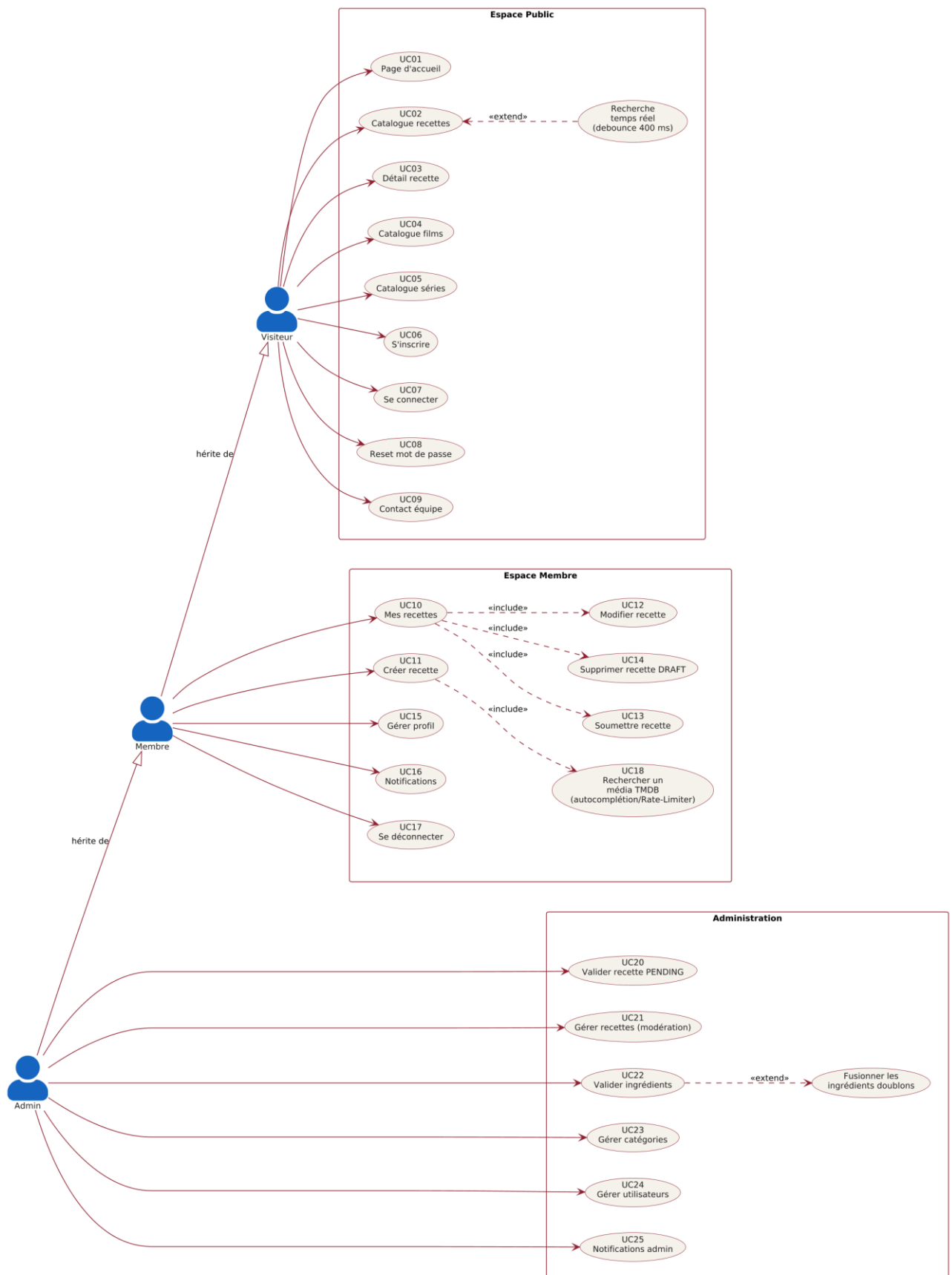


Diagramme des cas d'utilisation — vue complète (25 UC)

Synthèse des UC par acteur

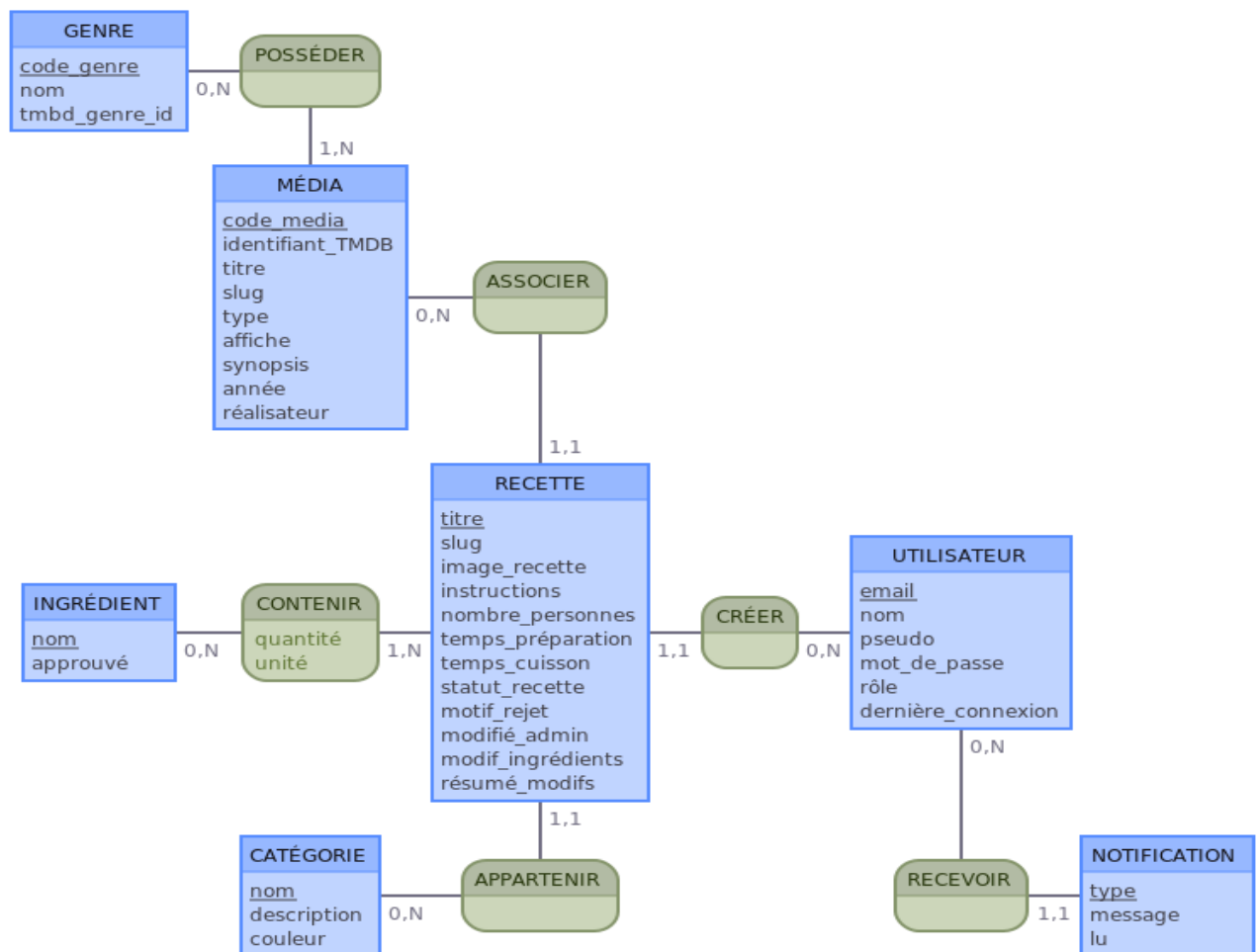
ID	Acteur	Titre	Priorité
UC01–UC05	Visiteur	Accueil, catalogue recettes, détail, catalogue films/séries	● Must
UC06–UC07	Visiteur	Inscription / Connexion	● Must
UC08–UC09	Visiteur	Réinitialisation MDP / Contact	● Should / ● Could
UC10–UC14	Membre	Mes recettes — CRUD complet (lire, créer, modifier, soumettre, supprimer)	● Must
UC15–UC18	Membre	Profil, notifications, déconnexion, recherche TMDB	● Must / ● Should
UC20–UC22	Admin	Validation recettes PENDING, modération, validation ingrédients	● Must
UC23–UC25	Admin	Catégories, utilisateurs, notifications admin	● Must / ● Should

i Use Cases détaillés (UC01 à UC25 — pré-conditions, déroulement nominal, alternatives) en Annexe 4.

5.4 Modélisation de la base de données — [Merise](#)

La conception suit la méthode Merise : Modèle Conceptuel (MCD) → Modèle Logique (MLD) → Modèle Physique (MPD). J'ai porté entièrement cette modélisation, du MCD jusqu'au schéma Prisma.

5.4.1 MCD — Modèle Conceptuel de Données



MCD Cinés Délices — 6 entités principales, 6 associations

Association	Entités liées	Card. A	Card. B	Description
CRÉER	UTILISATEUR ↔ RECETTE	0,N	1,1	Un utilisateur crée 0..N recettes. Une recette est créée par un seul utilisateur.
RECEVOIR	UTILISATEUR ↔ NOTIFICATION	0,N	1,1	Une notification a un destinataire unique.
APPARTENIR	CATÉGORIE ↔ RECETTE	0,N	1,1	Une recette appartient à une seule catégorie.
CONTENIR	INGRÉDIENT ↔ RECETTE	0,N	1,N	Une recette contient 1..N ingrédients (avec quantité et unité).
POSSÉDER	GENRE ↔ MÉDIA	0,N	1,N	Un média possède 1..N genres TMDB.
ASSOCIER	MÉDIA ↔ RECETTE	0,N	1,1	Une recette est associée à un seul média (film ou série).

5.4.2 MLD — Modèle Logique de Données

Le passage du MCD au MLD applique trois règles classiques :

- Une association 1..N (One-To-Many) devient une clé étrangère dans l'entité côté « 1 ».
- Une association N..N (Many-To-Many) devient une table de liaison avec les deux clés étrangères en clé primaire composite. C'est le cas de CONTENIR → table recipe_ingredients (avec attributs quantity, unit) et de POSSÉDER → table media_genre.
- Pas d'association 1..1 dans ce projet.

Tables principales (PK soulignées, # = clé étrangère) :

```

USER (user_id, email, pseudo, username, password_hash, role, last_login_at, ...)
RECIPE (recipe_id, title, slug, image_url, instructions, people_number, prep_time, cook_time,
        recipe_status, rejection_reason, #user_id, #category_id, #media_id,
        adminEditedSinceSubmission, adminEditedIngredientsSinceSubmission, adminEditedFieldsSummary)
INGREDIENTS (ingredient_id, name, approved)
MEDIA (media_id, tmdb_id, title, slug, type, poster_url, synopsis, year, director)
GENRE (genre_id, name, tmdb_genre_id)
CATEGORY (category_id, name, description, color)
NOTIFICATIONS (notification_id, type, message, is_read, #user_id, #recipe_id)

-- Tables de liaison
RECIPE_INGREDIENTS (#recipe_id, #ingredient_id, quantity, unit)
MEDIA_GENRE (#media_id, #genre_id)

```

5.4.3 Dictionnaire de données

Le dictionnaire ci-dessous liste **exhaustivement** les 9 tables de la base et leurs attributs. Pour chaque champ : type SQL, nullabilité, contraintes (clés primaires, étrangères, index, valeurs par défaut), et description métier. Il sert de référence pour le passage au MPD (§5.4.4) et au schéma Prisma (§5.5).

Légende : PK = clé primaire · FK = clé étrangère · NON = NOT NULL · OUI = nullable · CASCADE / SET NULL / RESTRICT = politiques onDelete.

Table users — Comptes utilisateurs (membres et administrateurs)

Champ	Type SQL	Null	Spécificités	Description
id (PK)	UUID	NON	@default(gen_random_uuid())	Identifiant unique auto-généré.
email	VARCHAR(255)	NON	@unique, @@index	Adresse email unique de connexion.
pseudo	VARCHAR(100)	NON	@unique	Pseudo public affiché sur les recettes.
nom	VARCHAR(255)	OUI	—	Nom réel, facultatif.
password_hash	VARCHAR(255)	NON	Hash Argon2id	Mot de passe haché — jamais retourné par l'API.
role	ENUM Role	NON	@default(MEMBER)	Rôle de l'utilisateur (MEMBER ou ADMIN).
created_at	TIMESTAMP	NON	@default(NOW())	Date de création du compte.
last_login_at	TIMESTAMP	OUI	—	Date de dernière connexion (usage RGPD).

Champ	Type SQL	Null	Spécificités	Description
reset_token	VARCHAR(255)	OUI	@unique	Token de réinitialisation de mot de passe.
reset_token_expires	TIMESTAMP	OUI	—	Date d'expiration du token (1 h).

Table recipes — Recettes de cuisine inspirées des films et séries

Champ	Type SQL	Null	Spécificités	Description
id (PK)	UUID	NON	@default(gen_random_uuid())	Identifiant unique de la recette.
titre	VARCHAR(255)	NON	—	Titre de la recette.
slug	VARCHAR(255)	NON	@unique, @@index	Texte simplifié pour l'URL.
image_url	VARCHAR(500)	OUI	Sharp WebP	Chemin vers l'image (-70 % poids).
instructions	TEXT	NON	—	Étapes de préparation en texte libre.
nombre_personnes	INTEGER	OUI	—	Nombre de portions.
temps_preparation	INTEGER	OUI	minutes	Durée préparation (parseTimeToMinutes).
temps_cuisson	INTEGER	OUI	minutes	Durée de cuisson en minutes.
status	ENUM RecipeStatus	NON	@default(DRAFT), @@index	État (DRAFT, PENDING, PUBLISHED).
rejection_reason	TEXT	OUI	—	Motif de refus visible par le membre.
admin_edited_since_submission	BOOLEAN	NON	@default(false)	Indique si l'admin a modifié les textes.
admin_edited_ingredients	BOOLEAN	NON	@default(false)	Indique si l'admin a modifié les ingrédients.
admin_edited_fields_summary	TEXT	OUI	—	Résumé des champs modifiés par l'admin.
user_id (FK)	UUID	OUI	FK users.id, SET NULL	Auteur de la recette (RGPD).
category_id (FK)	UUID	NON	FK categories.id, RESTRICT	Type de plat associé.
media_id (FK)	UUID	NON	FK media.id, RESTRICT	Film ou série d'inspiration.
created_at	TIMESTAMP	NON	@default(NOW())	Date de création.

Champ	Type SQL	Null	Spécificités	Description
updated_at	TIMESTAMP	NON	@updatedAt (Prisma)	Date de mise à jour automatique.

Table media — Films et séries (snapshot TMDB)

Champ	Type SQL	Null	Spécificités	Description
id (PK)	UUID	NON	@default(gen_random_uuid())	Identifiant unique du média.
tmdb_id	INTEGER	NON	@@unique([tmdb_id, type])	Identifiant TMDB (film/série peuvent partager le même).
titre	VARCHAR(255)	NON	—	Titre original du film ou de la série.
slug	VARCHAR(255)	NON	@unique, @@index	Texte simplifié pour l'URL.
type	ENUM MediaType	NON	MOVIE ou SERIES, @@index	Format du média cinématographique.
poster_url	VARCHAR(500)	OUI	Sharp WebP	Chemin vers l'affiche stockée localement.
synopsis	TEXT	OUI	—	Résumé court fourni par TMDB.
annee	INTEGER	OUI	—	Année de sortie du média.
realisateur	VARCHAR(255)	OUI	—	Réalisateur ou créateur du média.
created_at	TIMESTAMP	NON	@default(NOW())	Date d'ajout du média en base.

Table categories — Type de recette (Entrée, Plat, Dessert, Boisson)

Champ	Type SQL	Null	Spécificités	Description
id (PK)	UUID	NON	@default(gen_random_uuid())	Identifiant unique de la catégorie.
nom	VARCHAR(255)	NON	@unique	Nom de la catégorie (ex : Entrée).
description	VARCHAR(100)	OUI	—	Description courte de la catégorie.
color	VARCHAR(255)	OUI	—	Couleur associée à la catégorie (UI).

Table genres — Genre du média (horreur, comédie, etc.)

Champ	Type SQL	Null	Spécificités	Description
id (PK)	UUID	NON	@default(gen_random_uuid())	Identifiant unique du genre.

Champ	Type SQL	Null	Spécificités	Description
nom	VARCHAR(255)	NON	@unique	Nom du genre (ex : Horreur).
tmdb_genre_id	INTEGER	NON	@unique	Identifiant de genre propre à TMDB.

Table ingredients — Ingrédients pour les recettes (sel, farine, etc.)

Champ	Type SQL	Null	Spécificités	Description
id (PK)	UUID	NON	@default(gen_random_uuid())	Identifiant unique de l'ingrédient.
nom	VARCHAR(255)	NON	@unique, @@index	Nom normalisé (singulier, lowercase, sans accent).
approved	BOOLEAN	NON	@default(false)	Indique si l'ingrédient est validé par l'admin.

Table notifications — Notifications in-app

Champ	Type SQL	Null	Spécificités	Description
id (PK)	UUID	NON	@default(gen_random_uuid())	Identifiant unique de la notification.
type	ENUM NotificationType	NON	RECIPE_SUBMITTED	Nature de la notification.
message	TEXT	NON	—	Corps textuel de l'alerte.
is_read	BOOLEAN	NON	@default(false), @@index	État de lecture de la notification.
user_id (FK)	UUID	NON	FK users.id, CASCADE	Destinataire (supprimé si compte supprimé).
recipe_id (FK)	UUID	OUI	FK recipes.id, SET NULL	Recette liée (devient NULL si supprimée).
created_at	TIMESTAMP	NON	@default(NOW()), @@index	Date d'émission de la notification.

Table recipe_ingredients — Liaison N..N avec attributs

Champ	Type SQL	Null	Spécificités	Description
recipe_id (FK)	UUID	NON	FK recipes.id, CASCADE	Recette concernée.
ingredient_id (FK)	UUID	NON	FK ingredients.id, RESTRICT	Ingrédient utilisé.
quantity	VARCHAR(50)	OUI	Attribut de liaison	Quantité (ex : "200", "1/2").

Champ	Type SQL	Null	Spécificités	Description
unit	VARCHAR(50)	OUI	Attribut de liaison	Unité (ex : "g", "cl", "cuillère").
—	—	—	PK composite (recipe_id, ingredient_id)	Clé primaire composite.

Table media_genres — Liaison N..N entre médias et genres

Champ	Type SQL	Null	Spécificités	Description
media_id (FK)	UUID	NON	FK media.id, CASCADE	Média concerné.
genre_id (FK)	UUID	NON	FK genres.id, CASCADE	Genre TMDB associé.
—	—	—	PK composite (media_id, genre_id)	Clé primaire composite.

Synthèse — index et clés composites

Les index composites améliorent les requêtes les plus fréquentes :

- `@@index([status, categoryId])` sur `recipes` : filtre catalogue par catégorie + status PUBLISHED.
- `@@unique([tmdbId, type])` sur `media` : un film et une série peuvent partager le même tmdbId, mais pas deux films.
- `@@index([userId, isRead])` sur `notifications` : remontée rapide des notifications non lues d'un utilisateur.

5.4.4 MPD — Diagramme de classes UML

Le MPD est représenté par le diagramme de classes UML ci-dessous, qui ajoute les types précis, les énumérations (Role, RecipeStatus, MediaType, NotificationType) et les politiques onDelete (CASCADE, SET NULL, RESTRICT) appliquées par Prisma.

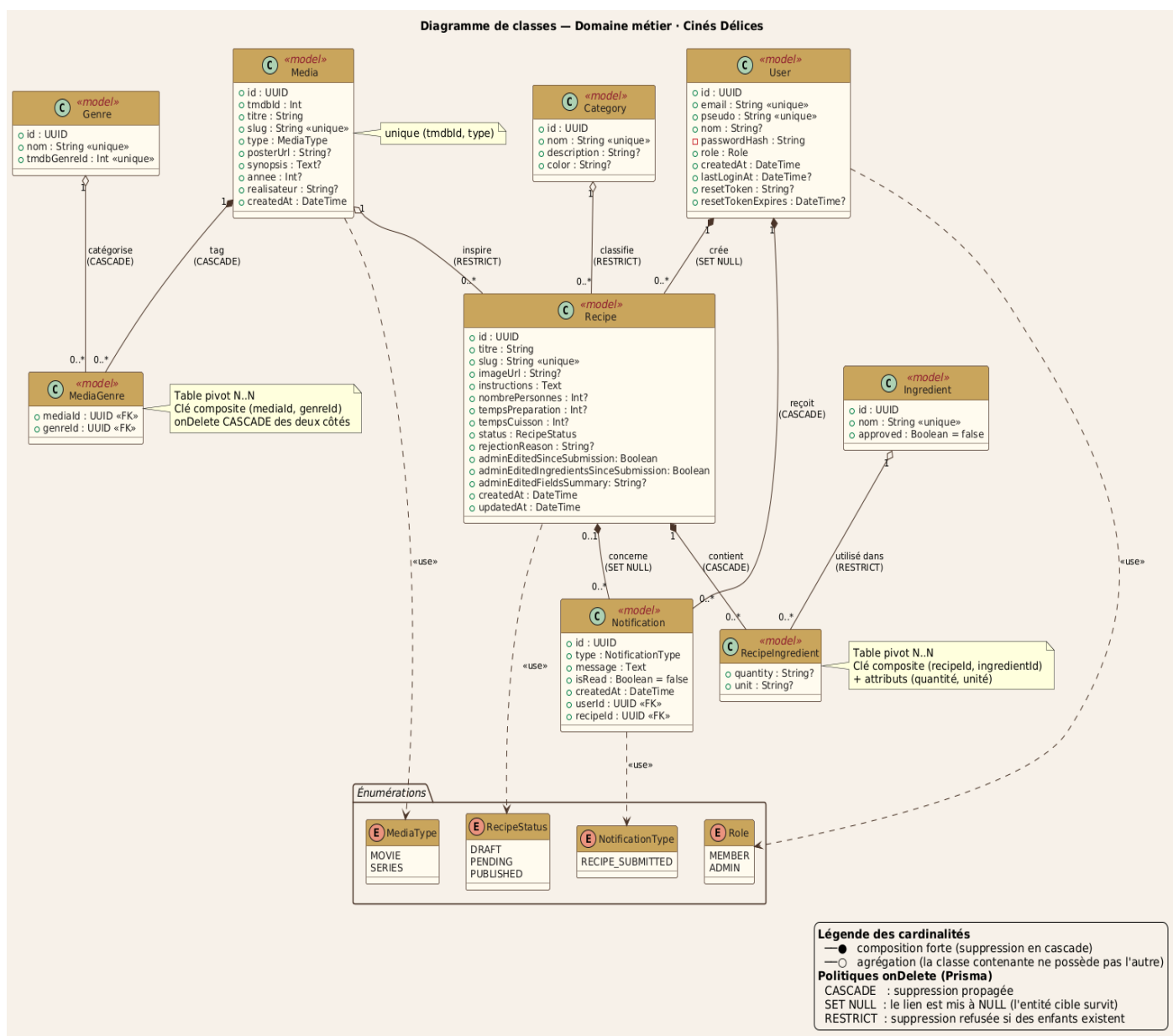


Diagramme de classes — domaine métier Cinés Délices

5.4.5 Règles On Delete — pourquoi ces choix

Chaque relation a une politique de suppression choisie selon le besoin métier et la conformité RGPD.

Relation	Politique	Raison du choix
user → recipe	SET NULL	RGPD : à la suppression d'un compte, les recettes PUBLISHED restent visibles (userId → null), seul l'auteur disparaît. Front : on affiche « ancien membre ».
category → recipe	RESTRICT	Impossible de supprimer une catégorie utilisée par au moins une recette (renvoie 409 Conflict).
media → recipe	RESTRICT	Idem : un film/série lié à une recette ne peut pas être supprimé.
recipe → recipe_ingredients	CASCADE	Si la recette disparaît, les lignes ingrédients liées disparaissent automatiquement.

Relation	Politique	Raison du choix
ingredient → recipe_ingredients	RESTRICT	Un ingrédient utilisé dans une recette ne peut pas être supprimé directement (oblige à passer par la fusion / merge).
user → notification	CASCADE	À la suppression du compte, les notifications personnelles disparaissent (nettoyage RGPD).
recipe → notification	SET NULL	Si la recette est supprimée, la notification reste mais perd sa référence (on peut toujours afficher « Une recette a été supprimée »).

5.4.6 Règles de gestion (RG)

ID	Règle de gestion
RG-01	Un membre ne peut soumettre que ses propres recettes (DRAFT) — vérification ownership avant PATCH /submit.
RG-02	Seul un ADMIN peut publier ou rejeter une recette PENDING (motif obligatoire en cas de rejet).
RG-03	Un membre ne peut modifier ou supprimer que ses recettes DRAFT ou PENDING — jamais une recette PUBLISHED.
RG-04	À la suppression d'un compte : recettes PUBLISHED conservées (userId → NULL), DRAFT/PENDING supprimées.
RG-05	Un même média TMDB (film ou série) ne peut être enregistré deux fois — contrainte @@unique([tmdbld, type]).
RG-06	Inactivité 11 mois → email de prévenance CNIL. Inactivité 12 mois → suppression automatique.

5.5 Script de création et seed de la base de données

Le schéma Prisma est versionné dans le dépôt. La commande `npx prisma migrate dev` génère le script SQL et applique la migration. En production sur Railway, `prisma migrate deploy` applique les migrations sans les régénérer.

```
# Workflow migration
npx prisma migrate dev --name init_schema    # dev – crée et applique
npx prisma migrate deploy                    # prod – applique uniquement
npx prisma db seed                          # exécute prisma/seed.js v4

# Seed v4 – 16 médias TMDB + comptes admin/membre + catégories + ingrédients
```

i Script SQL complet généré par Prisma migrate (CreateTable, CreateIndex, AddForeignKey) en Annexe 6.

5.6 Diagrammes de séquence — cas significatifs

Deux diagrammes de séquence couvrent les flux les plus complexes : la création d'une recette (détaillée en §7 comme fonctionnalité phare) et l'autocomplétion TMDb avec snapshot local.

i Diagramme de séquence Ajout d'une recette (fonctionnalité phare) détaillé en §7.

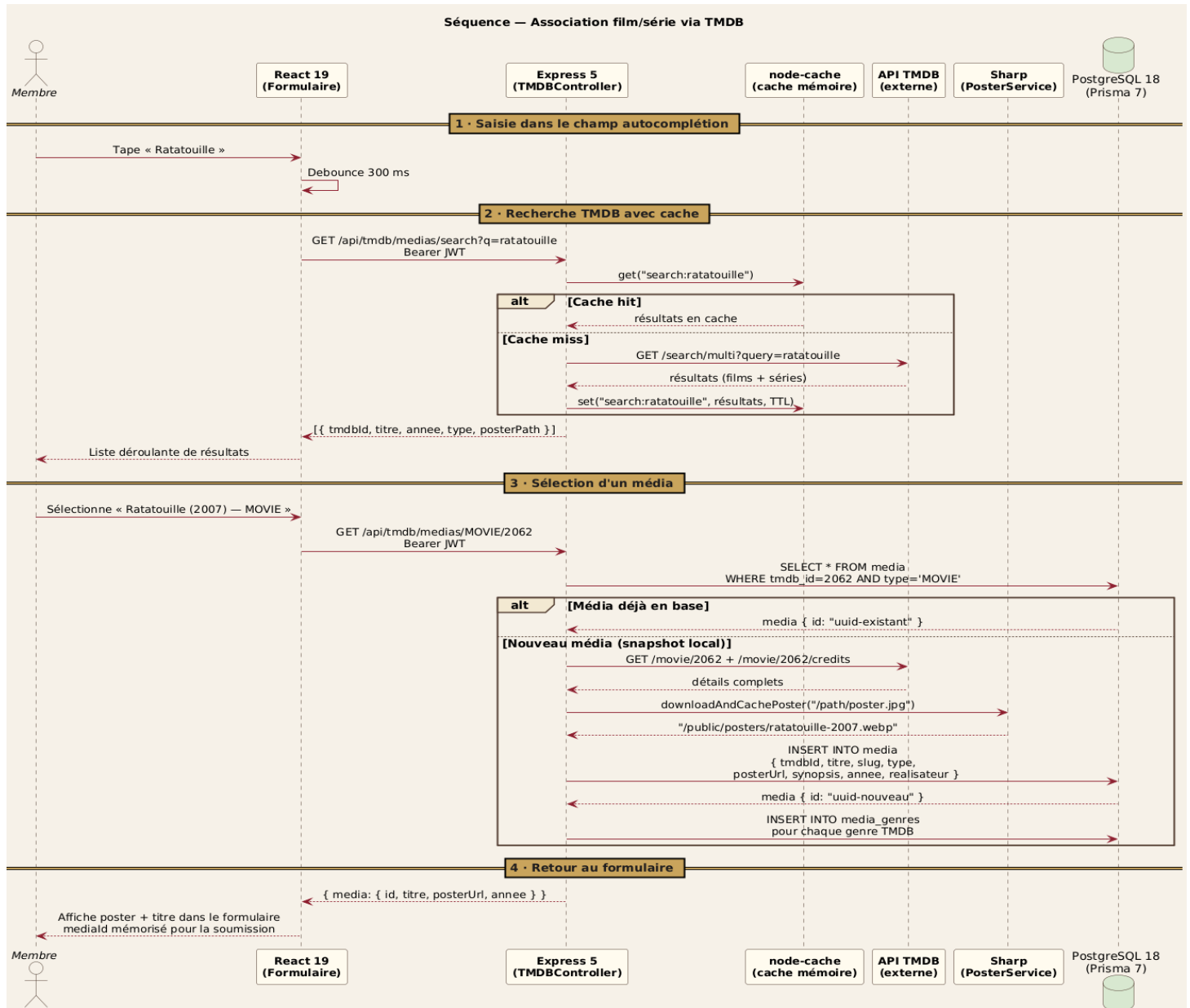
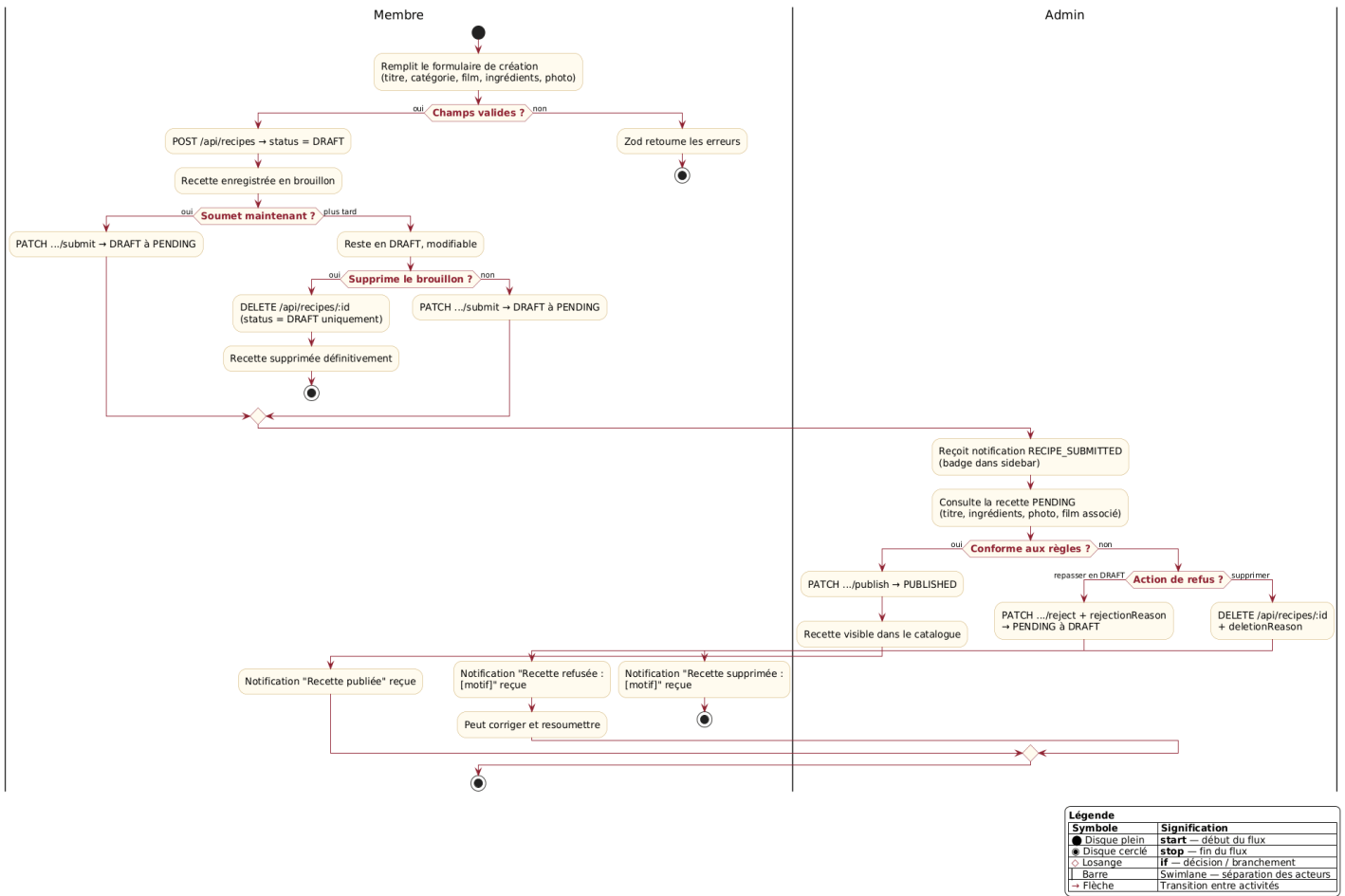


Diagramme de séquence — Autocomplétion + snapshot TMDb

5.7 Diagramme d'activité — workflow de publication

Le diagramme d'activité ci-dessous formalise le workflow complet de publication d'une recette, du formulaire de création à la notification de retour, côté membre ET côté admin.

Activité — Workflow de publication d'une recette



Workflow de publication d'une recette — Membre / Admin

5.8 Diagramme d'états — cycle de vie d'une recette

Une recette traverse trois états (DRAFT → PENDING → PUBLISHED) avec des transitions contrôlées par les règles de gestion RG-01 à RG-03.

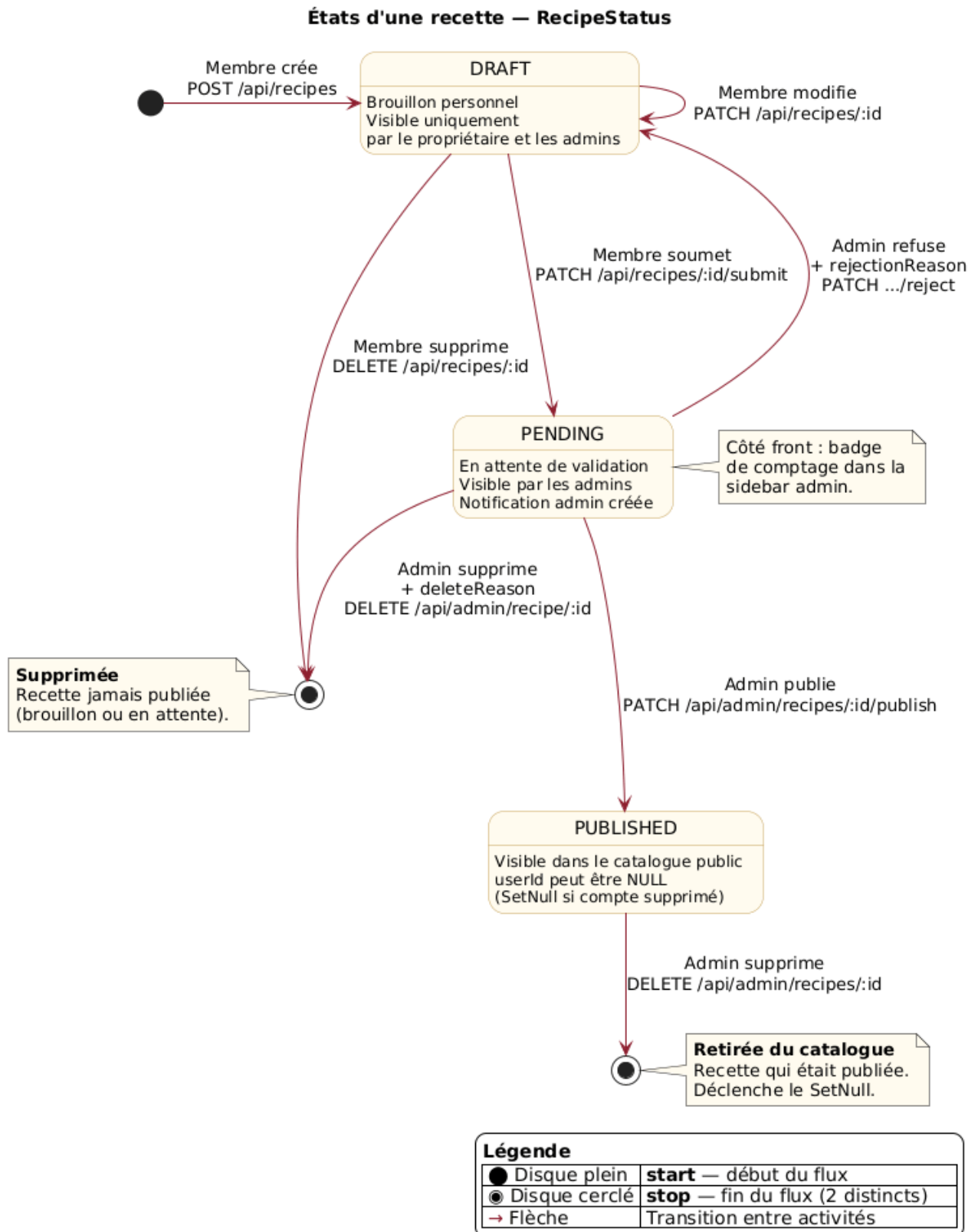


Diagramme d'états RecipeStatus — DRAFT, PENDING, PUBLISHED

6. Spécifications techniques

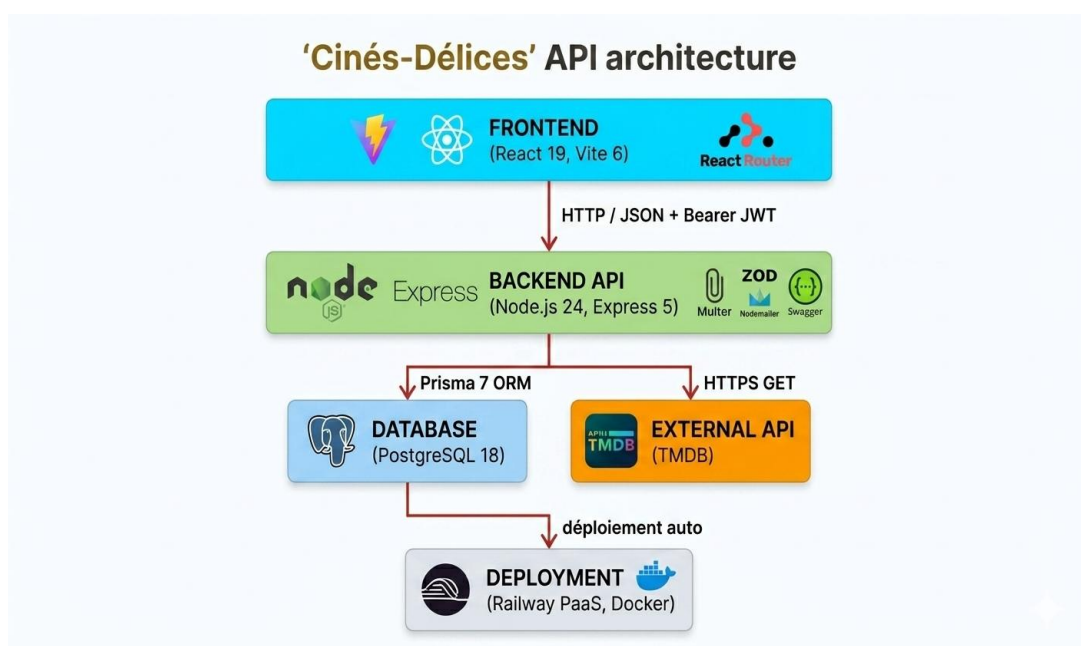
6.1 Choix d'architecture — multicouche découplée

Architecture API REST + SPA — frontend et backend en applications indépendantes communiquant uniquement via JSON HTTP. Cette séparation permet de faire évoluer chacune des couches sans impacter l'autre, et ouvre la porte à un client mobile natif futur consommant la même API.

Décisions structurantes et justifications

Décision	Justification métier / technique
API REST découplée	Front et back évoluent indépendamment. L'API pourra être consommée par une future app mobile.
SPA React (pas de SSR)	Navigation fluide sans rechargement — optimal pour catalogue avec filtres et recherche temps réel.
Prisma ORM (pas de SQL brut)	0 injection SQL par construction. Migrations versionnées. Types générés depuis le schéma.
Docker Compose en dev	Environnement identique sur les 5 postes développeurs. Dev local : Node 24-alpine + PostgreSQL 18. Prod Railway : Dockerfile dédié node:22-slim (incompatibilité Prisma 7 + Nixpacks détectée et résolue — cf. §11).
Railway (remplace Render)	Déploiement auto sur push GitHub. Pas de cold start. Render présentait des instabilités.
Argon2id (pas bcrypt)	Argon2id résistant GPU/ASIC, standard OWASP depuis 2015. Supérieur à bcrypt face aux attaques modernes.

6.2 Schéma de l'architecture technologique



Lecture en cascade : le navigateur (React 19 + Vite) parle au backend Express en HTTP/JSON avec un Bearer JWT. Le backend parle à PostgreSQL via Prisma ORM (pas de SQL brut), et à TMDB en HTTPS pour récupérer les films/séries. Tout est conteneurisé Docker et déployé en continu sur Railway.

6.3 Architecture en couches — diagramme de packages UML

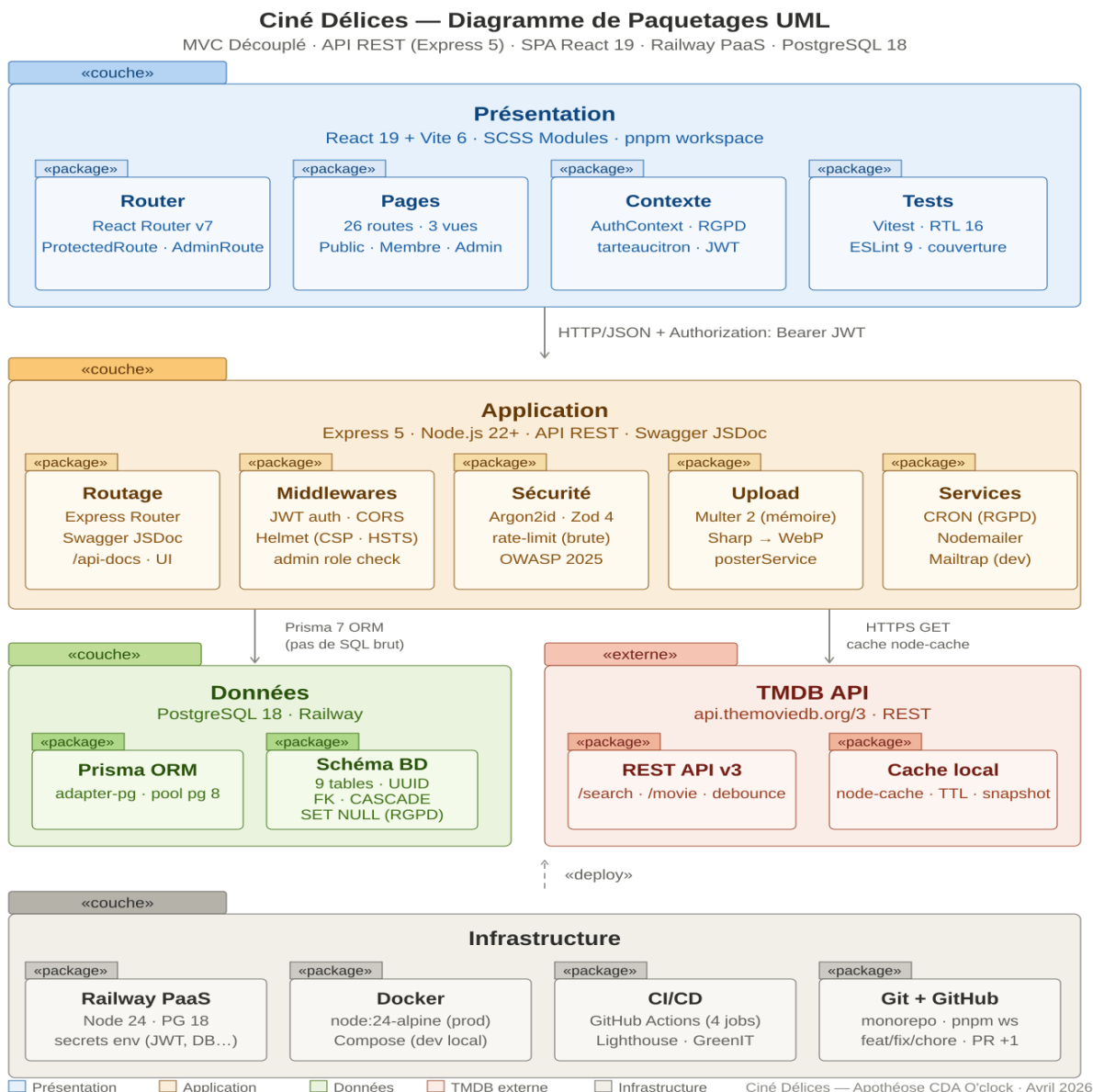


Diagramme de paquetages UML — Présentation, Application, Données, TMDB externe, Infrastructure

6.4 Stack technologique — choix justifiés

6.4.1 Frontend (/client)

Technologie	Version	Justification du choix
React	19.2	Bibliothèque UI dominante, écosystème mature, support long terme Meta.
Vite	6.4	Build ultra-rapide (ESM natif), HMR instantané, standard projets React modernes.
React Router DOM	7.13	Solution officielle de routing React, v7 apporte le type-safety et les loaders.
SASS (SCSS)	1.98	CSS organisé en équipe : variables, mixins, nesting → pas de duplication.
taiteaucitronjs	1.31	Bibliothèque française prête à l'emploi, conforme CNIL/RGPD.
Vitest	3.2	Natif Vite, rapide, compatible Jest, zéro configuration supplémentaire.
Testing Library React	16.3	Teste le comportement utilisateur réel (DOM) plutôt que l'implémentation.
ESLint	9.0	Détection statique des erreurs, conventions uniformes sur 5 développeurs.
react-dom	19.2	Rendu React dans le DOM navigateur, complément indissociable de React.
@vitejs/plugin-react	4.x	Plugin officiel Vite — Fast Refresh et transformation JSX.
@testing-library/jest-dom	6.x	Matchers DOM enrichis pour Vitest (toBeInTheDocument, toHaveClass...).
jsdom	24.x	Émulation DOM en Node.js — permet à Vitest de tester les composants sans navigateur.

6.4.2 Backend (/api)

Technologie	Version	Justification du choix
Node.js	≥ 24	Runtime JS serveur, partage de code et types avec le front, support natif ESM.
Express	5.2	Framework minimaliste mature, Express 5 gère les erreurs async sans try/catch.
Prisma 7 + adapter-pg	7.6 / 7.0	ORM type-safe, migrations versionnées, impossible d'écrire du SQL brut accidentellement.
PostgreSQL	18-alpine	SGBD relationnel ACID, robuste, version 18 partagée dev (Docker) / prod (Railway).
Zod	4.3	Validation schema-first, perfs ×14 vs Zod 3, corrige la CVE ReDoS de Zod 3.

Technologie	Version	Justification du choix
Multer	2.1	Middleware Express upload de référence, mode mémoire évite l'écriture disque inutile.
Sharp	0.34	Conversion WebP –70 % de poids, redimensionnement automatique, natif Node.
Argon2	0.44	Hashage MDP recommandé OWASP, résistant GPU/ASIC.
jsonwebtoken (JWT)	9.0	Authentification stateless signée HS256.
Helmet	8.1	En-têtes HTTP de sécurité, anti-XSS, anti-clickjacking, force HTTPS.
express-rate-limit	8.3	Anti-bruteforce login + rate-limit TMDB.
Nodemailer	8.0	Emails applicatifs — compatible Mailtrap (dev) et Brevo (prod).
node-cron	4.2	Job CRON natif, suffit pour le nettoyage RGPD hebdomadaire.
Swagger JSDoc + UI	6.2 / 5.0	Documentation API vivante synchronisée avec le code.
<code>cors</code>	2.8	CORS restreint à CLIENT_URL — bloque les origines non autorisées.
<code>node-cache</code>	5.1	Cache LRU TMDB en mémoire — économise quota TMDB et latence.
<code>dotenv</code>	16.x	Chargement automatique des variables .env en dev (Railway les injecte en prod).

6.4.3 Infrastructure et outillage

Outil	Type	Rôle concret
Docker + Compose	Conteneurisation dev	Environnement identique sur tous les postes en dev local (Node 24 + PG 18). Une commande lance api + client + db.
Railway	PaaS cloud prod	Héberge l'API + base PostgreSQL managée. Webhook GitHub = déploiement auto sur push main.
Git + GitHub	Versioning + revue	Historique complet, feature branches, PR obligatoires avec 1 review minimum.
GitHub Actions	CI / CD	Pipeline 4 jobs : lint client + lint api + test client + test api intégration.
pnpm	Gestionnaire de paquets	Plus rapide et économe en disque que npm. Gère le monorepo api + client.

6.5 Architecture de déploiement et CI/CD

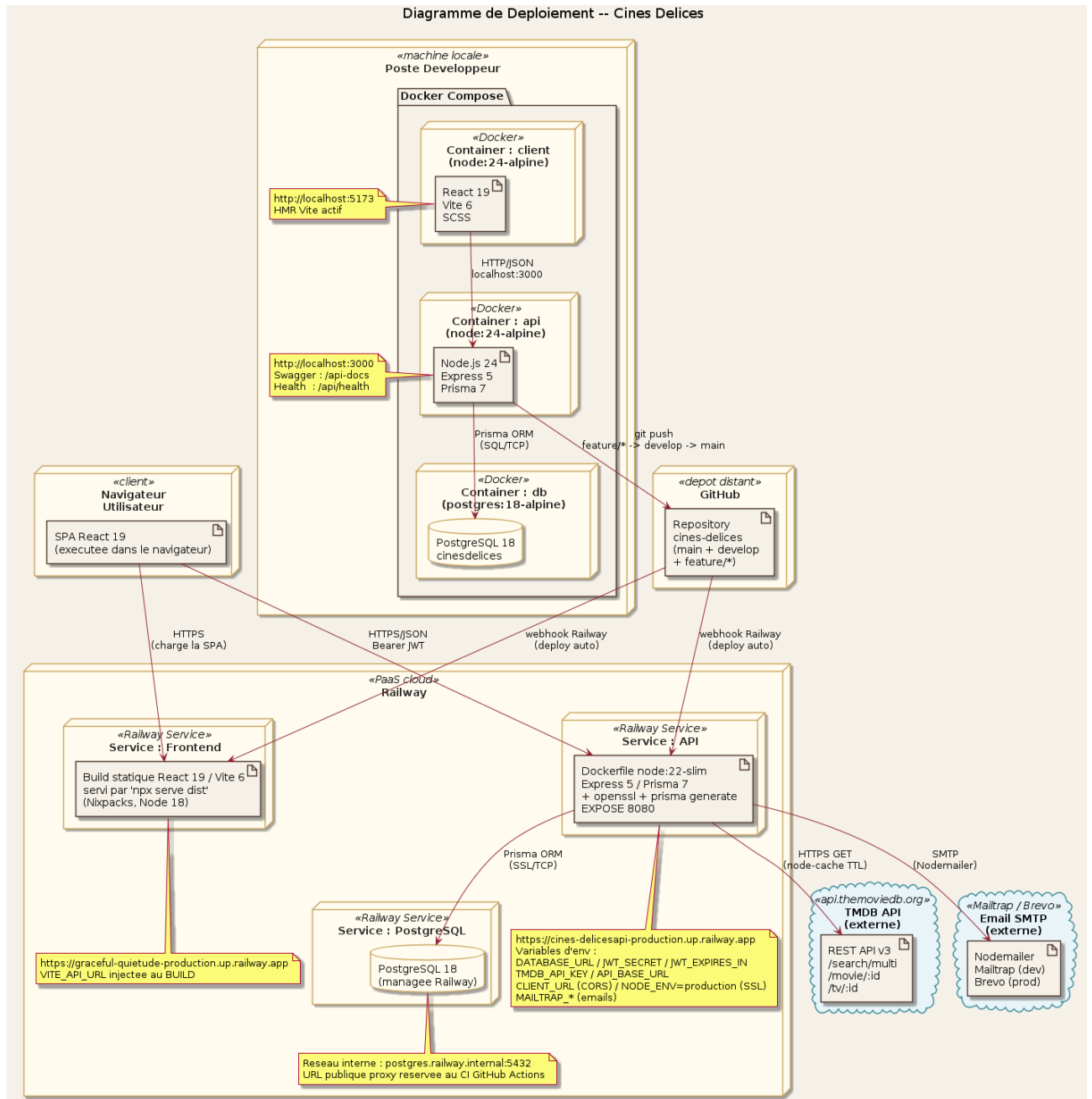


Diagramme de déploiement — local Docker, GitHub, Railway, TMDB externe

Pipeline CI/CD GitHub Actions — 4 jobs

Job	Outil	Déclencheur	Contenu
lint-client	ESLint	push / PR → main, develop	Qualité du code React / Vite
test-client	Vitest	push / PR → main, develop	Tests unitaires + composants (recipeUtils, ProtectedRoute, Navbar...)

Job	Outil	Déclencheur	Contenu
lint-api	ESLint	push / PR → main, develop	Qualité du code Node.js / Express
test-api-integration	node:assert + fetch	push → main uniquement	7 suites d'intégration contre l'API Railway (auth, workflow, sécurité, RGPD, fuzzy)

i Workflow GitHub Actions complet (*ci.yml*), *Dockerfile*, *Caddyfile*, *docker-compose.yml* et guide pas-à-pas Railway en Annexe 16.

6.6 Choix d'éco-conception

L'éco-conception a été pensée dès le Sprint 0 et appliquée tout au long du projet, avec des mesures concrètes et chiffrées.

Choix d'éco-conception	Mise en œuvre	Gain mesuré / cible
Compression d'images	Pipeline Multer + Sharp → WebP qualité 80, max 1200×1200	−70 % de poids vs JPEG d'origine
Pagination serveur	12 recettes/page, max 50 (hardcap), pas de SELECT * géant	Payload limité quelle que soit la taille du catalogue
Code splitting Vite	Lazy loading des routes admin / membre	Bundle initial réduit, mobile Lighthouse 68 → 86+
Cache HTTP différencié (Caddyfile)	1 an sur assets hashés, 30 j sur images, no-cache sur HTML	Moins de requêtes réseau, moins d'énergie consommée
Cache TMDB en mémoire (node-cache)	TTL sur recherches répétées + snapshot local	Moins d'appels externes inutiles
Sélection explicite Prisma (select)	Jamais de SELECT * — uniquement les colonnes utiles	Payload réduit + sécurité (jamais le password_hash)
Index BDD ciblés	Index sur status, slug, userId, categoryId, mediald	Requêtes catalogue O(log n) plutôt que scan complet
Suppression CRON RGPD	Nettoyage automatique des comptes inactifs > 12 mois	Pas de stockage inutile de données personnelles

Cible **EcoIndex** ≥ **C** pour la home et le catalogue, vérifiée régulièrement via l'extension Chrome *GreenIT-Analysis*. Audit Lighthouse intégré au CI sur les pages clés.

6.7 Sécurité — vue d'ensemble

La sécurité fait l'objet d'une section dédiée (§8) où chaque mesure est mise en regard du code qui la matérialise et du risque OWASP qu'elle adresse. Vue synthétique ici :

- Authentification stateless JWT signé HS256, hashage Argon2id, refresh côté client localStorage.
- Validation de chaque entrée serveur avec Zod 4 (corrige la CVE ReDoS de Zod 3).
- Prisma ORM = pas de SQL brut → injection SQL impossible par construction.
- Helmet pour les en-têtes HTTP de sécurité, CORS restreint à CLIENT_URL.

- Rate-limit ciblé sur les routes login (5/15 min) et TMDB (30/min).
- Conformité RGPD : tarteaucitron, pages légales, suppression compte, CRON inactivité 11/12 mois.

7. Réalisations les plus significatives — la création d'une recette

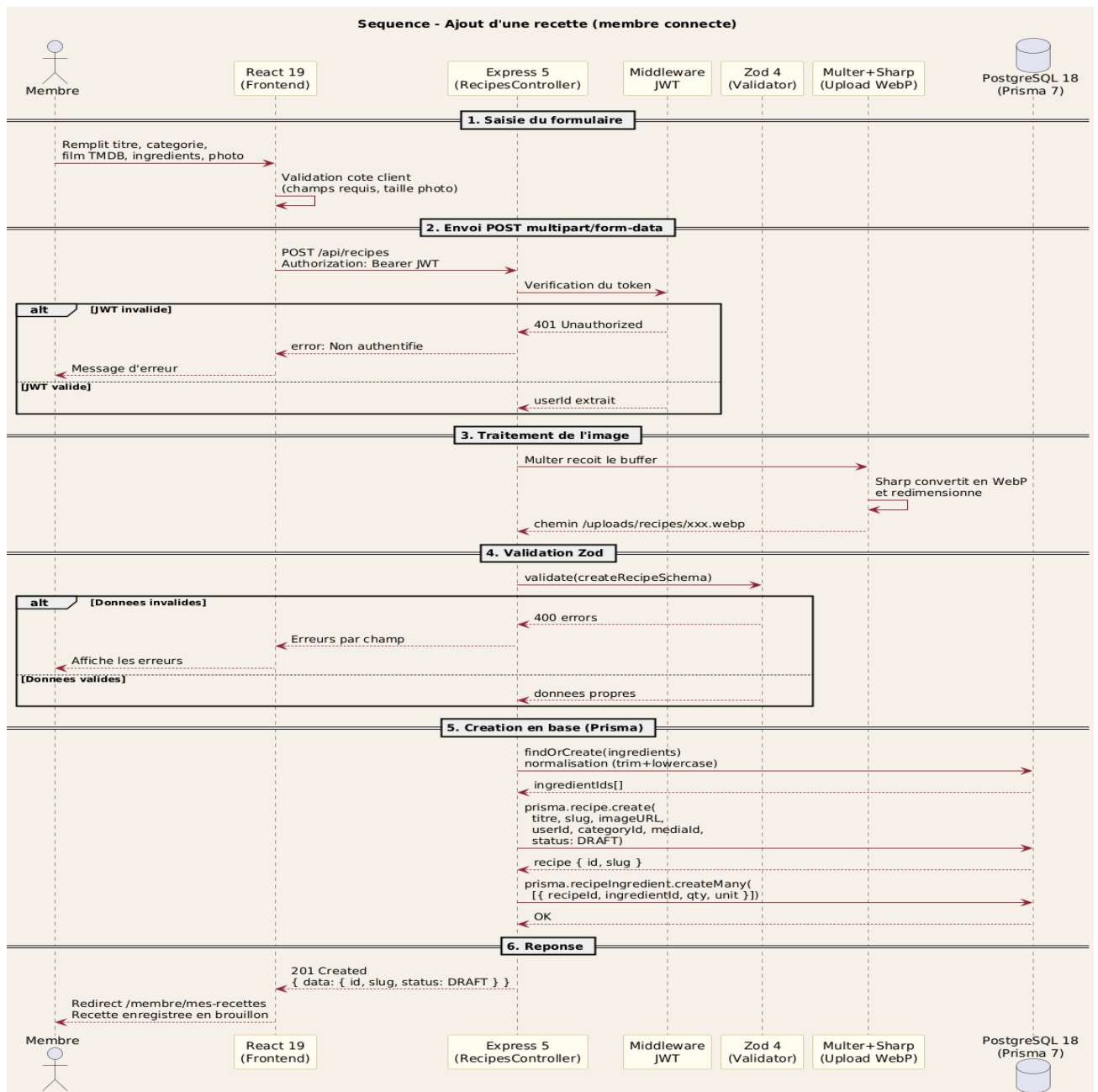
7.1 Pourquoi la création de recette ?

Sur l'ensemble du projet, je retiens la **création d'une recette par un membre connecté** comme fonctionnalité phare, pour quatre raisons.

- **Elle traverse les 3 couches** de l'architecture : interface React → middlewares + controller Express → ORM Prisma → PostgreSQL.
- **Elle illustre la sécurité** en application : JWT vérifié à chaque requête, fileFilter MIME, validation Zod, hashage des chemins, ownership user.
- **Elle mobilise un pipeline non trivial** — Multer mémoire → Sharp WebP qualité 80 → fichier disque avec nom horodaté unique.
- **Elle implique une notification asynchrone** — création d'une notification admin via `prisma.notification.create` — donc un effet de bord propre.

7.2 Vue d'ensemble — diagramme de séquence

Le diagramme de séquence ci-dessous présente, dans l'ordre exact d'exécution, les six étapes du flux : saisie du formulaire, envoi POST multipart, traitement de l'image, validation Zod, création en base, réponse 201.



Création d'une recette — 7 étapes du clic « Soumettre » à la réponse 201 Created

Pour la lisibilité, je double ce diagramme d'un déroulé pas-à-pas en **7 étapes** : chaque étape pointe sur le fichier source, la couche d'architecture concernée et la compétence CDA mobilisée. Les sections §7.3 à §7.4.5 reprennent ces 7 étapes dans l'ordre, avec le code et l'analyse à chaque étape.

#	Étape	Fichier source	Couche / CP
1	Saisie du formulaire React	client/src/pages/CreateRecipe/CreateRecipe.jsx	Présentation / CP2
2	Construction FormData + envoi POST avec JWT	CreateRecipe.jsx – handleSubmit()	Présentation / CP2

#	Étape	Fichier source	Couche / CP
3	authMiddleware — vérification JWT	api/src/middlewares/authMiddleware.js	Métier / CP3
4	Pipeline image (Multer mémoire + Sharp WebP)	api/src/middlewares/recipeImageUploadMiddleware.js	Métier / CP3
5	Validation Zod du payload	api/src/validators/recipesValidator.js	Métier / CP3
6	Controller createRecipe avec \$transaction	api/src/controllers/recipesController.js	Métier + Données / CP3+CP8
7	Réponse 201 Created + redirection adaptée	recipesController.js → React Router	Présentation / CP2

7.3 Étapes 1-2 — Saisie côté React (CP2)

Fichier : `client/src/pages/CreateRecipe/CreateRecipe.jsx` (~340 lignes au total, extrait significatif ci-dessous). Pour la saisie des durées, ce composant délègue à un utilitaire factorisé : `client/src/utils/recipeUtils.js` (extrait significatif ci-dessous).

Titre de la recette *

Tarte Tatin

Choisir un film ou une série *

La Bataille De Gaulle : L'âge de fer

* Recherche connectée à l'API TMDB

Type

Film

Ajouter une image

Glisser un .png/.jpg/.webp ici ou choisir un fichier



.1.0&iid=M3wxMjA3fDB8MHxzZWfyY2h8Nnx8dGFydGUIMjBhdXglMjBwb21tZXN8ZW58MHx8fDA%3D

Catégorie *

Dessert

Temps de préparation

45mn

Temps de cuisson

30mn

Nombre de personnes

6

Ingrédients *

creme fraiche

20

cl

+ Ajouter l'ingredient

6 pièce(s) pomme

Étapes de préparation *

1

eplucher les pommes

2

préparer la pâte

3

mettre au four

Étape 4...

Enregistrer en brouillon

Soumettre pour validation

Côté React, le composant *CreateRecipe* assume **les deux premières étapes du flux** : la saisie du formulaire (étape 1) puis sa soumission (étape 2). Chacune mobilise un mécanisme clé que je détaille dans les deux sous-paragraphes ci-dessous.

7.3.1 Étape 1 — La saisie (parsing tolérant des durées)

Pour que l'utilisateur puisse saisir « 1h30 », « 1:30 », « 90min », « 90 » ou « 1.5h » indifféremment, j'ai mis en place trois patterns regex testés dans l'ordre, après normalisation préalable (lowercase, suppression des espaces, virgule → point). Cette logique avait été initialement écrite directement dans *CreateRecipe.jsx*, puis je l'ai retrouvée dupliquée dans trois autres composants (*MemberRecipes*, *Admin/Dashboard*, *Admin/Recettes*). En avril 2026, j'ai factorisé cette fonction dans un module utilitaire dédié, *client/src/utls/recipeUtils.js*, illustrant la compétence CP2 (composants réutilisables) : un seul point de vérité, importable par tous les composants qui en ont besoin. *CreateRecipe.jsx* l'importe ligne 27 (`import { parseTimeToMinutes } from '../utls/recipeUtils.js';`) et l'appelle pour convertir les champs `tempsPrep` et `tempsCuisson` du formulaire avant envoi à l'API. Analogie : un assistant qui comprend toutes les façons de dire un temps, isolé dans sa propre "boîte à outils" pour pouvoir être réutilisé partout.

```
// Normalisation préalable : lowercase, suppression des espaces, virgule → point
const str = String(value).trim().toLowerCase()
  .replace(/\s+/g, '').replace(/,/g, '.');

// Pattern 1 : "1h30", "1.5h", "2h" — heures et minutes optionnelles
const hMatch = str.match(/^(\d+(:|\d+)?h(?:\d+)?(:min)?)?$/);
if (hMatch) {
  const total = Math.round(parseFloat(hMatch[1]) * 60 + parseInt(hMatch[2] || '0', 10));
  return total > 0 ? total : undefined;
}

// Pattern 2 : "1:30", "01:30" — format heure:minute
const colonMatch = str.match(/^(\d+):(\d+)(?:\d+)?$/);
if (colonMatch) {
  const total = parseInt(colonMatch[1], 10) * 60 + parseInt(colonMatch[2], 10);
  return total > 0 ? total : undefined;
}

// Pattern 3 : "90", "90min", "90mn", "90m" — minutes seules
const minMatch = str.match(/^(\d+(:|\d+)?)(?:min|mn|m)?$/);
if (minMatch) {
  const parsed = Math.round(parseFloat(minMatch[1]));
  return Number.isNaN(parsed) || parsed <= 0 ? undefined : parsed;
}
return undefined;
```

Choix technique : conversion systématique en minutes entières avant stockage en base. Le serveur reçoit toujours un *INT4* PostgreSQL, ce qui simplifie tous les calculs et tris en aval. Cette fonction est couverte par 7 tests Vitest (cf. §9).

7.3.2 Étape 2 — L'envoi (branchement multipart / JSON + JWT)

La fonction `handleSubmit()` choisit dynamiquement le format d'envoi selon la présence d'un fichier image : **multipart/form-data** si l'utilisateur a uploadé une image (FormData transporte le fichier + les champs texte), sinon **JSON pur** (payload plus léger). Dans les deux cas, le Bearer JWT est récupéré dans `localStorage` et injecté dans l'en-tête `Authorization`. Le champ `submitForReview` transporte l'intention de l'utilisateur (`true` = soumission directe → `PENDING` ; `false` = brouillon → `DRAFT`).

```
// handleSubmit() — choix dynamique multipart/JSON selon présence d'image
```

```

const token = localStorage.getItem('token');
const hasImageFile = form.image instanceof File;
const headers = { ...(token ? { Authorization: `Bearer ${token}` } : {}), };
const requestInit = { method: 'POST', headers, };

if (hasImageFile) {
  // — BRANCHE MULTIPART — fichier image présent —
  const formData = new FormData();
  formData.append('titre', payload.titre);
  formData.append('film', payload.film);
  formData.append('filmId', String(payload.filmId));
  formData.append('type', payload.type);
  formData.append('categorie', payload.categorie);
  formData.append('instructions', payload.instructions);

  // Champs optionnels : ajoutés seulement s'ils ont une valeur définie
  // (sinon multipart envoie la chaîne "undefined" → Zod rejette)
  if (payload.tempsPreparation !== undefined) {
    formData.append('tempsPreparation', String(payload.tempsPreparation));
  }
  if (payload.tempsCuisson !== undefined) {
    formData.append('tempsCuisson', String(payload.tempsCuisson));
  }
  if (payload.nbPersonnes !== undefined) {
    formData.append('nbPersonnes', String(payload.nbPersonnes));
  }

  formData.append('ingredients', JSON.stringify(payload.ingredients));
  formData.append('etapes', JSON.stringify(payload.etapes));
  formData.append('image', form.image);

  // On transmet l'intention de l'utilisateur au back.
  // En multipart toutes les valeurs sont des strings : on envoie "true" ou "false", // le
  // Validator Zod côté serveur coerce cela en booléen.
  formData.append('submitForReview', String(submitAction === 'submit'));

  requestInit.body = formData; // boundary multipart ajouté auto par le navigateur
} else {
  // — BRANCHE JSON — pas d'image : payload plus léger —
  requestInit.headers = {
    ...headers,
    'Content-Type': 'application/json',
  };
  // En JSON pur on envoie un vrai booléen.
  // Le back lit le même champ qu'en multipart : `submitForReview`.
  requestInit.body = JSON.stringify({
    ...payload,
    submitForReview: submitAction === 'submit',
  });
}

const response = await fetch(RECIPE_CREATE_API, requestInit);

// ① On parse toujours la réponse, même en cas d'erreur :
// le back renvoie un message JSON utile à afficher à l'utilisateur.
let responsePayload = null;
try {
  responsePayload = await response.json();
} catch {
  responsePayload = null;
}

// ② Erreurs : message contextualisé selon le code HTTP renvoyé par l'API.
if (!response.ok) {

```

```

const apiMessage = getApiMessage(responsePayload);

if (response.status === 400 || response.status === 422) {
  showAlertAtTop('error', apiMessage || 'Champs manquants ou invalides. Vérifie le formulaire.');
```

} else if (response.status === 401 || response.status === 403) {
 showAlertAtTop('error', apiMessage || 'Vous devez être connecté(e) pour créer une recette.');

} else {
 showAlertAtTop('error', apiMessage || 'Erreur serveur lors de la création de la recette.');

}
 setShowSubmitModal(false);
 return;
 }

// 3 Succès : message + redirection adaptés au choix de l'utilisateur.
 // 'submit' → recette envoyée en modération → page "recettes en validation"
 // 'draft' → recette enregistrée en brouillon → page "mes recettes"

```

const successMessage = submitAction === 'submit'
  ? 'La recette a bien été soumise. Elle apparaîtra après validation.'
  : 'La recette a bien été enregistrée en brouillon. Vous pouvez la soumettre depuis "Mes recettes"';

showAlertAtTop('success', successMessage);
setShowSubmitModal(false);
setForm(INITIAL_FORM);

const targetRoute = submitAction === 'submit'
  ? '/membre/mes-recettes/recettes-en-validation'
  : '/membre/mes-recettes';
navigate(targetRoute, { replace: true });
```

i Code complet de `CreateRecipe.jsx` (formulaire, validation côté client, gestion erreurs) en Annexe 10.

7.4 Étapes 3 à 6 — Côté serveur (CP3)

Côté backend, la requête `POST /api/recipes` traverse 4 middlewares dans cet ordre, avant d'atteindre le controller.

7.4.1 Étape 3 — `authMiddleware` (vérification du JWT)

Fichier : `api/src/middlewares/authMiddleware.js`

```

export const authMiddleware = async (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Token manquant ou mal formaté' });
  }
  const token = authHeader.split(' ')[1];
  try {
    // Vérification de la signature et de l'expiration
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    // Le token peut rester valide après un reset DB: on vérifie que le user existe encore.
    const user = await prisma.user.findUnique({
      where: { id: decoded.id },
      select: { id: true, role: true, pseudo: true, email: true },
    });
    if (!user) return res.status(401).json({ error: 'Session invalide. Merci de vous reconnecter.' });
    req.user = user; // injection pour les controllers en aval
    next();
  } catch {
    return res.status(401).json({ error: 'Token invalide' });
  }
}
```

```

    } catch {
      return res.status(401).json({ error: 'Token invalide ou expiré' });
    }
  };

```

7.4.2 Étape 4 — Pipeline image (Multer mémoire + Sharp WebP)

Fichier : `api/src/middlewares/recipeImageUploadMiddleware.js`

Chaque image uploadée passe par : (1) Multer en mode mémoire (pas d'écriture disque intermédiaire), avec `fileFilter` MIME et limite 5 MB ; (2) Sharp qui redimensionne à 1200×1200 max avec `fit:inside`, convertit en WebP qualité 80, et écrit le fichier final sur disque avec un nom horodaté unique. Gain : -70 % de poids. L'extrait ci-dessous reprend les portions clés du code, avec des commentaires explicatifs ajoutés pour faciliter la lecture.

```

// Multer en mode MÉMOIRE – Sharp recevra le buffer RAM directement
const uploadToMemory = multer({
  storage: memoryStorage,
  fileFilter: imageFileFilter, // PNG/JPG/JPEG/WEBP uniquement
  limits: { fileSize: MAX_IMAGE_SIZE_BYTES }, // 5 MB max (constante centralisée)
}).single('image');

// -----
// Conversion Sharp : buffer → fichier .webp
// -----
// 1. Redimensionne si trop grande (max 1200x1200)
// 2. Convertit en WebP (format optimisé)
// 3. Compresse à 80% de qualité
// 4. Écrit le fichier final sur disque
async function convertToWebp(buffer) {
  // Lazy-loading de Sharp avec garde-fou : permet au serveur de démarrer
  // même si Sharp n'est pas installé (cas de certains environnements de test)
  const sharp = await getSharp();
  if (!sharp) {
    throw new Error('Sharp indisponible');
  }

  const unique = `${Date.now()}-${Math.round(Math.random() * 1e9)}`;
  const filename = `recipe-${unique}.webp`;
  const outputPath = path.join(UPLOADS_DIR, filename);

  await sharp(buffer)
    .resize({
      width: SHARP_CONFIG.maxWidth,
      height: SHARP_CONFIG.maxHeight,
      fit: 'inside', // garde les proportions, ne déforme pas
      withoutEnlargement: true, // ne pas agrandir si plus petit que le max
    })
    .webp({
      quality: SHARP_CONFIG.webpQuality,
    })
    .toFile(outputPath);

  return filename;
}

```

Choix technique : le stockage mémoire est plus efficace qu'un upload disque + relecture — Sharp reçoit directement le buffer. Le nom de fichier de sortie est rendu unique par `Date.now()` + `random`, ce qui prévient toute collision même en cas d'uploads simultanés. La sortie est attachée à `req.body.imageUrl` sous la forme d'un chemin relatif (ex: `/uploads/x.webp`) pour exploitation par le `controller` — le préfixage avec l'origine de l'API est ensuite assuré côté client par `buildApiAssetUrl()`.

7.4.3 Étape 5 — Validation Zod (schéma serveur)

Fichier : `api/src/validators/recipesValidator.js` (235 lignes au total, extrait significatif ci-dessous). Le schéma exporté `createRecipeSchema` est consommé par la route `POST /api/recipes` (cf. `api/src/routes/recipesRoutes.js` ligne 28) via le middleware générique `validate()` qui sécurise l'entrée du contrôleur.

Zod valide chaque champ contre un schéma fortement typé. Si une donnée est invalide (titre trop court, format d'UUID incorrect, ingrédient au mauvais format...), le middleware retourne `400 Bad Request` avec la liste des erreurs par champ — sans toucher à la base de données. La vérification d'existence en base (catégorie, média) reste de la responsabilité du contrôleur, qui renvoie alors `404`.

```
// api/src/validators/recipesValidator.js – extrait du schéma createRecipeSchema // Le
middleware `validate(createRecipeSchema)` enveloppe req.body, req.params // et req.query dans
un objet unique avant de le confronter au schéma Zod. // Pour POST /api/recipes, seul le sous-
schéma `body` est défini : on valide // donc uniquement le payload JSON envoyé par le front.
export const createRecipeSchema = z.object({
  body: z.object({
    titre: z
      .string({ required_error: 'Le titre est obligatoire' })
      .min(3, 'Le titre doit contenir au moins 3 caractères')
      .max(100, 'Le titre ne peut pas dépasser 100 caractères')
      .trim(),

    instructions: z
      .string()
      .min(1, 'Les instructions sont obligatoires')
      .max(5000, 'Les instructions ne peuvent pas dépasser 5000 caractères')
      .trim(),

    etapes: z.array(z.string().trim().min(1)).optional(),

    // Catégorie : on accepte soit l'UUID, soit le nom en clair (le controller résout)
    categoryId: z.string().uuid('ID de catégorie invalide').optional(),
    categorie: z.string().min(1, 'La catégorie est obligatoire').optional(),

    // Média : on accepte soit l'UUID local, soit le couple TMDB (filmId + film + type)
    mediaId: z.string().uuid('ID de média invalide').optional(),
    filmId: optionalIntFromInput, // preprocessor : "2062" → 2062
    film: z.string().trim().optional(),
    type: z.enum(['F', 'S', 'movie', 'tv', 'series']).optional(),

    // Champs numériques optionnels – coercion string→number via preprocessor
    nombrePersonnes: optionalIntFromInput,
    nbPersonnes: optionalIntFromInput, // doublon français accepté
    tempsPreparation: optionalIntFromInput,
    tempsCuisson: optionalIntFromInput,

    imageUrl: z.string().url().optional().or(z.literal('')),

    ingredients: z.array(ingredientSchema).optional(),

    // Le membre choisit au moment du clic :
    // true → recette directement envoyée en modération (PENDING)
    // false → recette gardée en brouillon personnel (DRAFT)
    // absent → par défaut DRAFT (sécurité : on n'envoie en modération
    // que si l'utilisateur l'a explicitement demandé)
    submitForReview: booleanFromInput.optional(),
  })
})
// refine() : validation conditionnelle "soit l'un, soit l'autre"
.refine((data) => Boolean(data.categoryId || data.categorie), {
  message: 'La catégorie est obligatoire', path: ['categoryId'],
})
```



```
.refine((data) => Boolean(data.mediaId || data.filmId), {
  message: 'Le média est obligatoire', path: ['mediaId'],
}),
```

7.4.4 Étape 6 — Controller createRecipe (orchestration)

Le controller orchestre la création en base. Le statut initial dépend du choix de l'utilisateur dans le formulaire : si `submitForReview` vaut `true` (bouton « Soumettre pour validation ») la recette part directement en `PENDING` ; sinon (bouton « Enregistrer en brouillon ») elle est créée en `DRAFT` et reste privée. Cinq étapes séquentielles : (1) résolution de la catégorie par `UUID` ou par nom — création réservée aux admins, 404 si non trouvée ; (2) résolution du média via `mediaId` existant OU via `resolveMediaFromTmdb()` qui snapshot depuis `TMDB` si nécessaire ; (3) choix du statut initial selon `submitForReview` ; (4) création de la recette puis des `recipeIngredients` avec normalisation des noms (`normalizeIngredientName` : singulier + minuscule + `NFD` pour fusionner « Citrons » et « citron ») ; (5) notification de tous les admins via `prisma.notification.createMany` uniquement si statut `PENDING` (un brouillon reste privé, aucun admin n'est notifié). Les opérations critiques qui touchent plusieurs tables — `updateRecipe` (suppression puis recréation des `recipeIngredients`) et `submitRecipe` (transition `DRAFT` → `PENDING` + notification admin) — utilisent quant à elles `prisma.$transaction` explicite pour garantir l'atomicité (voir §7.5.1).

```
// api/src/controllers/recipesController.js – extrait createRecipe (lignes 186-345)
export const createRecipe = async (req, res) => {
  try {
    const {
      titre, instructions, etapes,
      categoryId: rawCategoryId, categorie,           // UUID OU nom
      mediaId: rawMediaId, filmId, film, type,       // UUID OU couple TMDB
      imageUrl, nombrePersonnes, nbPersonnes,
      tempsPreparation, tempsCuisson,
      ingredients,
      submitForReview,                               // DRAFT vs PENDING
    } = req.body;
    const userId = req.user?.id;

    // (1) Résolution catégorie : UUID ou nom (lookup insensible à la casse).
    // Si non trouvée → 404. Seul un admin peut créer une catégorie.
    let categoryId = rawCategoryId;
    let category = null;
    if (categoryId) {
      category = await prisma.category.findUnique({ where: { id: categoryId } });
    } else if (categorie) {
      category = await prisma.category.findFirst({
        where: { nom: { equals: String(categorie).trim(), mode: 'insensitive' } },
      });
      categoryId = category?.id;
    }
    if (!category) {
      return res.status(404).json({ message: 'Catégorie introuvable' });
    }

    // (2) Résolution média : par id local, ou snapshot TMDB si nécessaire
    let mediaId = rawMediaId;
    if (mediaId) {
      const existingMedia = await prisma.media.findUnique({ where: { id: mediaId } });
      if (!existingMedia) return res.status(404).json({ message: 'Média introuvable' });
    } else {
      mediaId = await resolveMediaFromTmdb({
        tmdbId: Number(filmId),
        title: String(film).trim(),
      });
    }
  }
}
```

```

        mediaType: ['s', 'tv', 'series'].includes(String(type).toLowerCase())
          ? 'SERIES' : 'MOVIE',
      });
    }

    // (3) Choix du statut initial selon le bouton cliqué côté front
    const initialStatus = submitForReview === true ? 'PENDING' : 'DRAFT';

    // (4) Création recette + recipeIngredients (détail complet en Annexe 11)
    const recipe = await prisma.recipe.create({
      data: {
        titre, slug: await generateUniqueSlug(titre, /* ... */),
        instructions, userId, categoryId, mediaId,
        status: initialStatus,
        // ... autres champs
      },
      include: recipeRelationsInclude,
    });
    for (const ing of ingredients) {
      const ingredientName = normalizeIngredientName(ing.nom); // "Citrons" → "citron"
      let ingredient = await prisma.ingredient.findUnique({ where: { nom: ingredientName } });
      if (!ingredient) ingredient = await prisma.ingredient.create({ data: { nom:
ingredientName } });
      await prisma.recipeIngredient.create({
        data: { recipeId: recipe.id, ingredientId: ingredient.id, quantity, unit },
      });
    }

    // (5) Notification admin UNIQUEMENT si statut PENDING (DRAFT reste privé)
    if (initialStatus === 'PENDING') {
      const admins = await prisma.user.findMany({
        where: { role: 'ADMIN' }, select: { id: true },
      });
      await prisma.notification.createMany({
        data: admins.map((admin) => ({
          type: 'RECIPE_SUBMITTED',
          message: `Nouvelle recette soumise: ${titre}`,
          userId: admin.id, recipeId: recipe.id,
        })),
      });
    }
  }

  return res.status(201).json({
    message: initialStatus === 'PENDING'
      ? 'Recette créée avec succès. Elle sera vérifiée par un administrateur.'
      : 'Recette enregistrée en brouillon. Vous pouvez la soumettre depuis "Mes recettes".',
    recipe,
  });
} catch (error) {
  console.error('[createRecipe]', error);
  return res.status(500).json({ message: 'Erreur serveur lors de la création de la recette.'
});
}
};

```

7.4.5 Étape 7 — Réponse 201 Created et redirection adaptée

Le serveur retourne **201 Created** avec le message adapté au statut (DRAFT « brouillon enregistré » ou PENDING « recette créée, en attente de validation »). Côté React, le composant `CreateRecipe` lit la valeur de `submitAction` et redirige l'utilisateur vers la bonne page : `/membre/mes-recettes/recettes-en-validation` si soumise pour modération, `/membre/mes-recettes` si simple brouillon.

Le flag `replace: true` passé à React Router évite que le retour navigateur ramène l'utilisateur sur le formulaire — comportement attendu après une soumission validée.

7.5 Choix structurants d'accès aux données (CP8)

Sur le plan de l'accès aux données, trois choix structurants se retrouvent dans `createRecipe` et dans tous les autres controllers Cinés Délices.

7.5.1 \$transaction pour l'atomicité

La transaction garantit l'atomicité quand une opération touche plusieurs tables. Par exemple dans `updateRecipe` : on doit (a) supprimer tous les `recipeIngredients` existants, puis (b) recréer ceux du payload, puis (c) éventuellement créer/mettre à jour les notifications admin. Le tout dans `prisma.$transaction(async (tx) => {...})`. Si l'une de ces étapes échoue (par exemple une contrainte unique violée sur un ingrédient), Prisma rollback automatiquement : la recette ne se retrouve pas en base avec ses anciens ingrédients supprimés mais sans les nouveaux. `submitRecipe` applique le même pattern pour la transition DRAFT → PENDING (`recipe.update` + `notification.createMany` dans la même transaction). C'est une garantie ACID, pas un nice-to-have.

7.5.2 Sélection explicite — jamais SELECT *

Pour la lecture, Prisma `select` explicite n'expose jamais les champs sensibles (notamment `passwordHash`). Exemple sur `/api/auth/me` :

```
const user = await prisma.user.findUnique({
  where: { id: req.user.id },
  select: {
    id: true, email: true, nom: true, pseudo: true, role: true, createdAt: true,
    _count: { select: { recipes: true } },
    // passwordHash : VOLONTAIREMENT ABSENT — jamais retourné
  },
});
```

7.5.3 Index ciblés sur les requêtes fréquentes

Le schéma Prisma déclare des index sur les colonnes filtrées : `status` (catalogue PUBLISHED), `slug` (résolution URL), `userId`, `categoryId`, `mediaId`, et un index composite (`status`, `categoryId`) pour le filtre du catalogue.

i Code complet `recipesController.js`, `recipeImageUploadMiddleware.js`, `authController.js`, `authMiddleware.js` en Annexes 10 à 12.

7.6 Captures d'écran — du formulaire à la confirmation

Pour respecter la limite de pages, les captures du flux complet (formulaire de saisie, sélection TMDB autocomplétion, image redimensionnée affichée, deux boutons « Enregistrer en brouillon » / « Soumettre pour validation », message de succès, recette listée selon le choix — DRAFT dans `/mes-recettes` si brouillon, PENDING dans `/mes-recettes/recettes-en-validation` si soumise) sont disponibles sur le dépôt GitHub `cines-delices-docs` (captures prises sur la version Railway de production).

7.7 Autres réalisations significatives (synthèse)

Au-delà de la création de recette, j'ai personnellement porté plusieurs autres réalisations significatives, listées ici en synthèse et détaillées dans les annexes correspondantes.

Réalisation	Couche	Spécificité technique	Annexe
Recherche temps réel avec debounce 400 ms	CP2 UI	setTimeout + cleanup useEffect, isCancelled flag	§7.2 + Annexe 10
AuthContext + useCallback (stabilité refs)	CP2 UI	useMemo + useCallback pour éviter les closures périmées	Annexe 10
ProtectedRoute / AdminRoute	CP2 UI	Garde-fous de route React Router v7	Annexe 10
Job CRON RGPD (inactivityCheck.js)	CP3 métier	node-cron + transaction Prisma + emails 11/12 mois	Annexe 11
Rate-limiter TMDB ciblé	CP3 métier	express-rate-limit 30/min par IP, message cinéma-cohérent	Annexe 11
Catalogue paginé avec filtres dynamiques	CP8 data	\$transaction findMany + count, limit hardcap 50	Annexe 12
Fusion d'ingrédients atomique (merge)	CP8 data	\$transaction + détection des doublons recipe_ingredients	Annexe 12
Fuzzy search ingrédients	CP8 data	contains insensitive + normalisation singulier (NFD)	Annexe 12
Autocomplétion TMDB (déroulé pas-à-pas similaire au §7)	CP3 métier	useEffect + debounce 400 ms + proxy serveur + cache RAM node-cache + snapshot local	Annexe 5

8. Éléments de sécurité de l'application

La sécurité a été traitée en trois axes complémentaires : défense technique en profondeur calquée sur OWASP Top 10, conformité RGPD opérationnelle pilotée par la CNIL, et veille continue sur les vulnérabilités (cette dernière fait l'objet de la §11).

8.1 OWASP Top 10 — où apparaît chaque mesure dans le code

Outil de sécurité	Vulnérabilité OWASP	Où dans le code Cinés Délices
Argon2id (argon2 0.44)	A02 Cryptographic Failures	authController.register : argon2.hash(password) AVANT prisma.user.create
JWT signé HS256	A02 + A07 Auth Failures	authController.signToken() — payload {id, pseudo, role}, jamais le hash
Helmet 8.1	A05 Security Misconfig	app.js : app.use(helmet()) en tête, avant les routes
express-rate-limit 8.3	A07 Auth Failures	tmdbRateLimiter.js (30/min IP) + login limiter (5/15min)
Zod 4.3	A08 Data Integrity	middlewares/validate.js — schéma par route, 400 si invalide
Prisma ORM 7	A03 Injection	Toutes les requêtes — pas une seule ligne de SQL brut dans le projet
authMiddleware + adminMiddleware	A01 Broken Access Control	middlewares/authMiddleware.js — JWT vérifié AVANT le controller
CORS restreint	A05 Security Misconfig	app.js : cors({ origin: CLIENT_URL }) — pas de wildcard
Variables d'environnement	A04 Insecure Design	JWT_SECRET, TMDB_API_KEY, SMTP_* — jamais commités, Railway uniquement
Proxy TMDB serveur	A04 Insecure Design	Clé TMDB jamais exposée au client — appels via /api/tmdb/*
Architecture découplée	A04 Insecure Design	Client en SPA statique, API séparée — surfaces d'attaque distinctes

i *Référentiel OWASP Top 10 2025 complet, avec le détail des mesures Cinés Délices ligne par ligne, en Annexe 13.*

8.2 Validation des entrées — Zod côté serveur

Toute donnée venant du client est revalidée côté serveur, **même si** elle a déjà été validée côté React. Le front peut être contourné (curl, Postman, attaquant) ; le serveur ne fait jamais confiance au client.

```
// middlewares/validate.js – appliqué AVANT chaque controller protégé
// Valide les 3 sources de données HTTP : req.body, req.params, req.query
export const validate = (schema) => (req, res, next) => {
  try {
    // safeParse ne lève pas d'exception : retourne { success, data, error }
```

```

const result = schema.safeParse({
  body: req.body,
  params: req.params,
  query: req.query,
});

if (!result.success) {
  // Formater les erreurs Zod en messages lisibles pour le front
  // ZodError.flatten() regroupe par champ : { fieldErrors, formErrors }
  const formatted = result.error.flatten();
  const messages = [];

  // Erreurs par champ (dans body, params, query)
  for (const [field, errors] of Object.entries(formatted.fieldErrors)) {
    errors.forEach(msg => messages.push(`${field}: ${msg}`));
  }
  // Erreurs globales (ex: refine sur l'objet entier)
  formatted.formErrors.forEach(msg => messages.push(msg));

  return res.status(400).json({
    error: 'Données invalides',
    details: messages, // ex: ["email: Format invalide", ...]
  });
}

// Données valides → on réinjecte les versions nettoyées (trim, lowercase, coerce)
if (result.data.body) req.body = result.data.body;
if (result.data.params) req.params = result.data.params;
if (result.data.query) {
  // ⚠ req.query est en lecture seule dans Express 5 – on copie les propriétés
  // une par une au lieu de réaffecter l'objet.
  Object.assign(req.query, result.data.query);
}

next();
} catch (error) {
  // Ceinture-bretelles : ne devrait jamais arriver (safeParse ne lance pas)
  next(error);
}
};

```

8.3 Protection injection SQL

La protection est **structurelle** : Prisma ne permet pas d'écrire de SQL brut accidentellement. Chaque `prisma.recipe.findMany(...)` génère une requête paramétrée par construction. La seule fonction Prisma qui accepte du SQL brut (`$queryRawUnsafe`) n'est utilisée nulle part dans le projet — un grep le confirme.

8.4 Protection XSS

- **React échappe automatiquement** tout texte interpolé dans le JSX — pas de `dangerouslySetInnerHTML` utilisé dans le projet.
- **Helmet** injecte les headers Content-Security-Policy + X-Content-Type-Options + X-Frame-Options.
- **Zod refuse** les payloads contenant des balises script dans des champs email/pseudo (tests d'intégration sécurité dédiés, cf. §9 et Annexe 14).

8.5 Conformité RGPD

8.5.1 Bandeau de consentement cookies

Bandeau géré par tarteaucitron.js, configuré conformément à la délibération CNIL n°2020-091 : le bouton « Tout refuser » est aussi visible et accessible que « Tout accepter », même importance visuelle, même taille (option DenyAllCta: true). Le cookie JWT d'authentification est déclaré comme **cookie technique strictement nécessaire** au service — il ne nécessite pas de consentement (article 82 loi Informatique & Libertés).

8.5.2 Traitement des données personnelles — finalités et durées

Donnée collectée	Finalité	Durée de conservation
Email	Authentification, notifications, RGPD	Compte + 12 mois inactivité
Pseudo	Affichage public sur les recettes	Durée du compte
Mot de passe (Argon2id)	Authentification	Durée du compte
Date dernière connexion	Détection inactivité RGPD	Durée du compte
Recettes PUBLISHED	Contenu public communautaire	Conservées (userId → NULL si compte supprimé)
Recettes DRAFT/PENDING	Contenu privé en cours	Supprimées avec le compte

8.5.3 Cycle automatique d'effacement — 11/12 mois

La CNIL recommande une durée maximale de conservation de 12 mois d'inactivité (délibération n°2017-012). J'ai implémenté ce cycle via le job CRON node-cron, planifié chaque lundi à 3h du matin ('0 3 * * 1'). Il distingue **deux populations** parmi les comptes MEMBER : à **11 mois** d'inactivité, un email de prévenance est envoyé pour laisser 30 jours de réaction ; à **12 mois**, le compte est supprimé avec un email de confirmation envoyé **avant** la suppression effective (sinon on perdrait l'adresse). Le code inclut aussi les comptes **jamais connectés** (lastLoginAt: null) en se basant sur leur date de création — utile pour les comptes créés avant la mise en place du tracking. La suppression utilise prisma.\$transaction pour garantir l'atomicité : recettes non publiées + user supprimés ensemble, ou rien. Les recettes **PUBLISHED** ne sont pas supprimées : grâce à onDelete: SetNull dans le schéma Prisma, leur userId passe à NULL et elles restent visibles de manière anonyme — conforme RGPD : la donnée personnelle disparaît, l'œuvre publique reste.

```
// api/src/jobs/inactivityCheck.js – extrait significatif
// (templates d'emails buildWarningEmail/buildDeletionEmail en Annexe 11)

const ELEVEN_MONTHS_MS = 11 * 30 * 24 * 60 * 60 * 1000;
const TWELVE_MONTHS_MS = 12 * 30 * 24 * 60 * 60 * 1000;

// — Suppression d'un compte inactif —
async function purgeInactiveAccount(user) {
  // ⚠ Email envoyé AVANT la suppression (sinon on perd l'adresse)
  await sendEmail(buildDeletionEmail(user));

  await prisma.$transaction(async (tx) => {
    // Supprime les recettes non publiées (DRAFT + PENDING)
    await tx.recipe.deleteMany({
      where: { userId: user.id, status: { in: ['DRAFT', 'PENDING'] } },
    });
  });
}
```

```

    // Suppression du user → onDelete: SetNull bascule les PUBLISHED en anonyme
    await tx.user.delete({ where: { id: user.id } });
  });

  console.log(`[CRON] Compte inactif supprimé : ${user.pseudo} (${user.email})`);
}

// — Job principal —
async function checkInactiveAccounts() {
  const now = new Date();
  const warningThreshold = new Date(now.getTime() - ELEVEN_MONTHS_MS);
  const purgeThreshold = new Date(now.getTime() - TWELVE_MONTHS_MS);

  // 1. Comptes à SUPPRIMER : MEMBER inactifs > 12 mois
  const accountsToPurge = await prisma.user.findMany({
    where: {
      role: 'MEMBER',
      OR: [
        { lastLoginAt: { lt: purgeThreshold } },
        { lastLoginAt: null, createdAt: { lt: purgeThreshold } }, // tracking absent
      ],
    },
  });
  for (const user of accountsToPurge) {
    try {
      await purgeInactiveAccount(user);
    } catch (error) {
      // On continue la boucle même si une suppression échoue
      console.error(`[CRON] Erreur suppression ${user.pseudo} :`, error.message);
    }
  }

  // 2. Comptes à PRÉVENIR : MEMBER inactifs entre 11 et 12 mois
  const accountsToWarn = await prisma.user.findMany({
    where: {
      role: 'MEMBER',
      OR: [
        { lastLoginAt: { gte: purgeThreshold, lt: warningThreshold } },
        { lastLoginAt: null, createdAt: { gte: purgeThreshold, lt: warningThreshold } },
      ],
    },
  });
  for (const user of accountsToWarn) {
    try {
      await sendEmail(buildWarningEmail(user));
    } catch (error) {
      console.error(`[CRON] Erreur envoi email ${user.pseudo} :`, error.message);
    }
  }

  console.log(`[CRON] ${accountsToPurge.length} supprimé(s), ${accountsToWarn.length}
prévenu(s)`);
}

// — Démarrage : planification chaque lundi à 3h du matin —
export function startInactivityCron() {
  cron.schedule('0 3 * * 1', () => { // minute=0, heure=3, jour-
    semaine=lundi
    checkInactiveAccounts().catch((error) => {
      console.error('[CRON] Erreur vérification inactivité :', error);
    });
  });
}
next();
} catch (error) {

```



```
// Ceinture-bretelles : ne devrait jamais arriver (safeParse ne lance pas)
next(error);
}
};
```

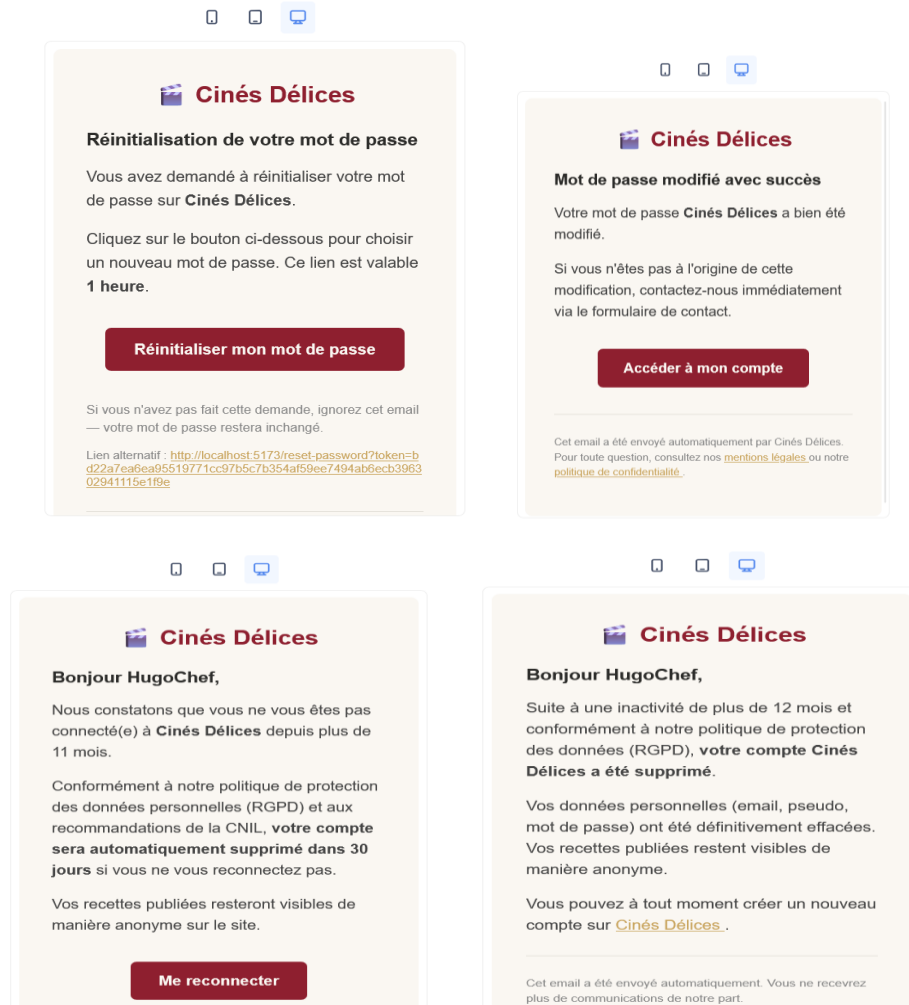
8.5.4 Emails applicatifs RGPD — Nodemailer

Quatre déclencheurs d'email automatiques, en style cinéma-cohérent (bordeaux #8E1F2F sur beige #FAF7F2) :

Email	Déclencheur	Destinataire
Réinitialisation MDP	POST /api/auth/forgot-password	Membre demandeur (token 1 h, usage unique)
Confirmation changement MDP	PATCH /api/users/me/password	Membre connecté
Prévenance inactivité (11 mois)	CRON — lundi 3h	Membres inactifs
Suppression compte (12 mois)	CRON — lundi 3h	Membres supprimés (envoyé AVANT delete)

Configuration SMTP : Mailtrap en développement (emails capturés sans envoi réel), Brevo en production (envoi authentifié SPF/DKIM).

Aperçu visuel des 4 emails (réception Mailtrap mobile) :



Reset MDP — Confirmation MDP — Prévenance 11 mois — Suppression 12 mois (Mailtrap mobile)

8.5.5 Pages légales

Quatre pages légales conformes CNIL, accessibles sans authentification : /mentions-legales, /politique-confidentialite, /politique-cookies, /regles-moderation. Lien pied de page sur toutes les pages.

9. Plan de tests

9.1 Stratégie de tests — pyramide à 5 niveaux

La stratégie suit la pyramide classique : beaucoup de tests unitaires (rapides, isolés, déterministes), un nombre raisonnable de tests d'intégration en environnement réel Railway, et un jeu d'essai fonctionnel manuel pour le workflow critique (§10).

Niveau	Outil	Ce qui est testé	Exécution
Unitaire	Vitest	parseTimeToMinutes, formatMinutes, normalizeCategoryLabel, normalizeIngredientName, mapApiRecipeToCard	CI/CD — job test-client

Niveau	Outil	Ce qui est testé	Exécution
Composants	Vitest + Testing Library + jsdom	ProtectedRoute, AdminRoute, Navbar selon rôle, CookieConsent, ResetPassword, MediaCatalog, RecipeCatalogView	CI/CD — job test-client
Intégration API	node:assert + fetch sur Railway	Auth (register/login), workflow DRAFT→PUBLISHED, admin (publish/reject), sécurité (XSS/SQLi/JWT falsifié), RGPD, fuzzy search — 7 suites	CI/CD — job test-api-integration (push main)
Fonctionnel	Manuel navigateur + API Rest Client	Workflow publication T01–T10, parcours membre/admin, upload, notifications	Manuel — résultats §10
Sécurité	pnpm audit + check headers Helmet	0 vulnérabilité critique (DoD), validation headers HTTP	Avant chaque release

9.2 Plan de tests détaillé — colonnes REAC

Le tableau ci-dessous présente le plan de tests au format imposé par le REAC : Fonctionnalité, Type, Données en entrée, Résultat attendu, Résultat obtenu, Statut. Il porte sur les fonctionnalités les plus critiques.

Fonctionnalité	Type	Données en entrée	Résultat attendu	Résultat obtenu	Statut
parseTimeToMinutes — format 1h30	Unitaire	'1h30'	90	90	☑
parseTimeToMinutes — format 1:30	Unitaire	'1:30'	90	90	☑
parseTimeToMinutes — format 90min	Unitaire	'90min'	90	90	☑
parseTimeToMinutes — invalide	Unitaire	'abc'	undefined	undefined	☑
formatMinutes — heure pile	Unitaire	60	'1h'	'1h'	☑
formatMinutes — heure+minute	Unitaire	90	'1h30min'	'1h30min'	☑
ProtectedRoute — non authentifié	Composant	<ProtectedRoute /> sans token	Redirect /login	Redirect /login	☑
ProtectedRoute — admin connecté	Composant	<ProtectedRoute /> token admin	Redirect /admin	Redirect /admin	☑
POST /api/auth/register valide	Intégration	{email, password, nom, prenom}	201 + token JWT	201 + token JWT	☑

Fonctionnalité	Type	Données en entrée	Résultat attendu	Résultat obtenu	Statut
POST /api/auth/login MDP incorrect	Intégration	{email valide, password faux}	401	401	☑
Headers Helmet présents	Intégration	GET /api/health	X-Frame-Options, X-CTO, CSP	Tous présents	☑
Injection SQL dans login	Sécurité	email: " OR 1=1 --"	Status ≠ 200	400	☑
XSS payload dans register	Sécurité	email: <script>alert</script>	Status ≠ 201	400	☑
JWT falsifié	Sécurité	Bearer eyJ...fake	401	401	☑
Input excessivement long	Sécurité	email: 10 000 chars	Status ≠ 500 (pas de crash)	400 (Zod)	☑

9.3 Exemples de tests — extraits significatifs

9.3.1 Test unitaire Vitest — parseTimeToMinutes

```
// client/src/tests/recipeUtils.test.js (extrait)
describe('parseTimeToMinutes', () => {
  it('comprend 1h30 → 90', () => expect(parseTimeToMinutes('1h30')).toBe(90));
  it('comprend 30min → 30', () => expect(parseTimeToMinutes('30min')).toBe(30));
  it('comprend 1:30 → 90', () => expect(parseTimeToMinutes('1:30')).toBe(90));
  it('comprend 1.5h → 90', () => expect(parseTimeToMinutes('1.5h')).toBe(90));
  it('rejette abc', () => expect(parseTimeToMinutes('abc')).toBeUndefined());
  it('rejette valeur nulle', () => expect(parseTimeToMinutes(null)).toBeUndefined());
  it('rejette 0 minute', () => expect(parseTimeToMinutes('0')).toBeUndefined());
});
```

9.3.2 Test d'intégration sécurité — JWT falsifié sur Railway

```
// api/tests/test-api-security.js (extrait – exécuté contre l'URL Railway)
// Cette suite testJWT() couvre 8 cas limites : token absent, malformé,
// falsifié, mal formaté, élévation de privilège, token valide, mot de
// passe incorrect, compte inexistant. Extrait du cas 4.3 ci-dessous.

async function testJWT() {
  // ... (cas 4.1 token absent, 4.2 token malformé)

  // 4.3 Token falsifié – on encode un vrai header JWT + un payload
  // prétendant être ADMIN, puis on bidouille la signature.
  // Si le middleware n'invalidé PAS ce token, c'est une élévation
  // de privilège grave (vol d'identité admin).
  const fakeToken = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9' // header { alg, typ }
    + '.eyJpZCI6IjEiLCJyb2x1IjoiaURNSU4ifQ' // payload { id, role: ADMIN }
    + '.fake_signature_ici'; // signature bidonnée

  const falsified = await request('/api/auth/me', {
    headers: { Authorization: `Bearer ${fakeToken}` },
  });
  assert.equal(falsified.response.status, 401,
```

```
    'Token avec signature falsifiée doit retourner 401');  
    pass('Token falsifié → 401');  
  
    // ... (4.4 Bearer mal formaté, 4.5 élévation membre→admin via vrai  
    //      token mais sur route admin, 4.6 token valide → profil retourné,  
    //      4.7 mot de passe incorrect, 4.8 compte inexistant)  
  }  
}
```

i Suite complète des tests d'intégration (*test-api.js* 416 lignes + *test-api-security.js* 734 lignes) et fichier *Vitest (recipeUtils.test.js)* en Annexe 14.

10. Jeu d'essai

10.1 Scénario testé — workflow complet de publication

Le jeu d'essai documenté ci-dessous couvre le workflow critique de l'application : la publication d'une recette par un membre, sa modération par l'administrateur, et la notification de retour au membre. Dix cas (T01→T10) valident les chemins nominaux et les rejets attendus. Tous ont été exécutés manuellement en environnement Railway (URLs réelles) le 28 avril 2026, avec les comptes du seed v4.

10.2 Tableau du jeu d'essai

ID	Action testée	Données en entrée	Données attendues	Données obtenues	Statut
T01	Connexion membre	marie@cinesdelices.fr / Member1234!	200 OK + token JWT	200 OK + token JWT	☑
T02	Création recette DRAFT	Titre, catégorie, image, ingrédients, filmId	201 + {id, slug, status: DRAFT} + image WebP	201 + image convertie WebP	☑
T03	Soumission DRAFT → PENDING	PATCH /api/recipes/:id/submit (membre)	200 + status PENDING + notif admin créée	200 + notif admin reçue	☑
T04	Accès admin liste PENDING	Connexion admin + GET /pending	200 + recette T02/T03 visible	Recette visible	☑
T05	Validation admin → PUBLISHED	PATCH /admin/recipes/:id/publish	200 + status PUBLISHED + notif membre	200 + notif Marie reçue	☑
T06	Notification membre	GET /api/notifications (Marie)	200 + 1 notif non lue	1 notif "Recette publiée"	☑
T07	Suppression non autorisée	DELETE membre sur recette PUBLISHED	403 Forbidden (RG-03)	403 Forbidden	☑
T08	Refus admin avec motif	PATCH /reject + rejectionReason	200 + status DRAFT + notif motif	200 + motif visible	☑
T09	Upload format invalide	Fichier .pdf sur POST /recipes	400 Bad Request (fileFilter)	400 Bad Request	☑
T10	Accès admin sans token	GET /admin/pending sans Authorization	401 Unauthorized (authMiddleware)	401 Unauthorized	☑

Résultat global : 10/10 cas PASS. Aucune régression détectée. Le workflow critique est conforme aux spécifications fonctionnelles (User Stories §2.4) et aux règles de gestion (RG-01 à RG-03 §5.4.6).

i Captures d'écran de chaque étape T01→T10 (panneau admin, notifications membre, emails Mailtrap) en Annexe 15.

11. Veille sécurité

11.1 Démarche de veille — intégrée au workflow

La sécurité n'est pas un état figé mais une démarche continue : chaque dépendance peut révéler une CVE, chaque recommandation OWASP évolue, chaque délibération CNIL ajoute une obligation. J'ai intégré cinq sources de veille au workflow projet, chacune avec un impact concret et traçable sur une décision technique ou de conformité.

11.2 Cinq sources, cinq décisions concrètes

Source	Usage dans le projet	Impact concret sur Cinés Délices
OWASP Password Storage Cheat Sheet	Choix de l'algorithme de hashage	Argon2id retenu (premier choix OWASP 2024) plutôt que bcrypt — résistant aux attaques GPU/ASIC.
Base CVE — Zod 3 (ReDoS)	Audit dépendances	Migration Zod 3 → 4 effectuée en Sprint 2 avant toute mise en production. Évite la régression ReDoS.
Prisma Changelog v7	Suivi de version ORM	Correction de la configuration datasource lors de la migration Prisma 6 → 7 — évite un bug bloquant au déploiement Railway (Node 22 + openssl + prisma generate).
CNIL (cnil.fr) — délibérations et fiches	Conformité RGPD	Durée de conservation 12 mois (déliv. 2017-012), format du bandeau de consentement (déliv. 2020-091).
pnpm audit	Chaque sprint et avant chaque release	0 vulnérabilité critique ou haute avant chaque mise en production (critère bloquant du DoD).

11.3 Vulnérabilités identifiées et traitées

- **CVE Zod 3 ReDoS** — une regex de validation d'email pouvait être exploitée pour un déni de service côté serveur. Action : migration immédiate vers Zod 4 (avant la mise en production). Bonus : Zod 4 est 14× plus rapide en bench.
- **Prisma 7 + [Nixpacks](#)** — Nixpacks détectait par défaut une version Node 18 sans openssl, incompatible avec Prisma 7. Action : Dockerfile sur mesure (node:22-slim + apt install openssl + prisma generate dans l'image) — résolu et documenté dans le guide de déploiement Railway.
- **OWASP Top 10 2025 vs 2021** — A04 « Insecure Design » a pris du poids dans la 2025. Action : revue des choix d'architecture (proxy TMDb serveur-side, architecture découplée, variables d'env exclusives), tous tracés en §8.1.

11.4 Analogie — la veille comme un chef qui contrôle ses ingrédients

Comme un chef qui vérifie chaque matin la fraîcheur de ses ingrédients avant l'ouverture, la veille s'exerce AVANT chaque release — pas après. Le critère « 0 vulnérabilité critique » est intégré au Definition of Done, donc bloquant pour la mise en production. Ce n'est pas une étape optionnelle, c'est une condition de livraison.

En plus de la veille sur la documentation essentielle pour ce projet, je me suis inscrite pour recevoir des alertes par email à [daily.dev](#), [Journal du Net](#), [Google Alerts](#). Je suis abonnée à plusieurs comptes Youtube, Je vais régulièrement sur [Mdn](#), [W3Schools](#), etc...

12. Glossaire technique

Ce glossaire regroupe les termes techniques et acronymes utilisés dans le dossier, organisés par domaine, pour faciliter la lecture du jury ou d'un lecteur non-expert sur certains sous-domaines précis.

12.1 Architecture & développement

API REST	Architecture client-serveur basée sur HTTP/JSON, sans état (stateless).
ORM	Object-Relational Mapping — couche d'abstraction qui mappe les tables SQL en objets JavaScript. Ici : Prisma 7.
SPA	Single Page Application — application web chargée une seule fois, les changements d'écran se font côté client (React Router v7).
SSR / CSR	Server-Side Rendering / Client-Side Rendering. Cinés Délices = CSR pur (SPA).
MVC	Model-View-Controller — séparation des responsabilités côté back.
Monorepo	Un seul dépôt Git pour /client et /api (workspace pnpm).
Workspace pnpm	Gestionnaire de paquets qui partage les dépendances entre /client et /api pour économiser le disque et accélérer l'installation.

12.2 Frontend

React 19	Bibliothèque UI déclarative. Composants fonctionnels + Hooks.
Hooks React	useState, useEffect, useMemo, useCallback, useRef, useContext — primitives de gestion d'état et d'effets.
Vite 6	Outil de build moderne — bundler ESM ultra-rapide en dev, optimise pour la prod.
SCSS / Sass	Préprocesseur CSS avec variables, mixins, nesting, fonctions.
BEM	Block Element Modifier — convention de nommage CSS (.bloc__element--modifier).
WCAG 2.1 AA	Web Content Accessibility Guidelines — référentiel d'accessibilité reconnu (RGAA en France).
Lighthouse	Outil Google d'audit qualité web : performance, accessibilité, SEO, best practices.
Debounce	Technique qui retarde l'exécution d'une fonction jusqu'à ce qu'un délai s'écoule sans nouvel appel — évite les requêtes excessives.

12.3 Backend

Node.js	Runtime JavaScript serveur basé sur le moteur V8 de Chrome.
Express 5	Framework web minimaliste pour Node.js. Gère les routes, middlewares, requêtes/réponses HTTP.
JWT	JSON Web Token — jeton signé pour authentification stateless. Format : header.payload.signature.
Bearer Token	Format d'envoi du JWT dans le header HTTP « Authorization: Bearer <token> ».
Argon2id	Algorithme de hashage de mot de passe recommandé OWASP 2024 — résistant aux attaques GPU/ASIC.
HMAC SHA-256	Algorithme de signature symétrique utilisé par JWT pour garantir l'intégrité du jeton.

Zod	Bibliothèque de validation de schémas TypeScript-first. Coerce et valide les données entrantes.
Middleware	Fonction Express intercalée entre la requête et le controller — ex : authMiddleware vérifie le JWT.
CORS	Cross-Origin Resource Sharing — autorise ou refuse les appels HTTP venant d'un autre domaine.
CSP	Content Security Policy — politique de sécurité du contenu, en-tête HTTP injecté par Helmet.
Helmet	Middleware Express qui ajoute automatiquement des en-têtes HTTP de sécurité (CSP, HSTS, etc.).
Rate-limiting	Limitation du nombre de requêtes par IP sur une période — protection contre les abus.

12.4 Base de données

PostgreSQL 18	Système de Gestion de Base de Données relationnelle open-source.
Prisma 7	ORM TypeScript pour Node.js — génère un client type-safe à partir d'un schéma déclaratif.
MCD / MLD / MPD	Modèle Conceptuel / Logique / Physique de Données — étapes successives de la méthode Merise.
MERISE	Méthode française de modélisation de bases de données relationnelles.
UML	Unified Modeling Language — notation standard pour modéliser des systèmes (use cases, classes, séquences, états).
UUID	Universally Unique Identifier — identifiant 128 bits unique, plus sûr et non énumérable qu'un auto-increment.
FK	Foreign Key — clé étrangère liant une table à une autre.
onDelete	Stratégie de suppression cascade en SQL/Prisma : CASCADE (supprime aussi les liés), SET NULL (anonymise), RESTRICT (refuse si liés).
\$transaction	API Prisma pour exécuter plusieurs requêtes de manière atomique — tout réussit ou tout est annulé.
Index composite	Index multi-colonnes pour accélérer les requêtes qui filtrent sur plusieurs champs ensemble.
Migration	Script versionné qui modifie la structure de la base (création/altération de tables).
Seed	Script qui peuple la base avec des données de démonstration ou de référence.

12.5 Sécurité & RGPD

OWASP Top 10	Liste des 10 risques de sécurité web les plus critiques, mise à jour 2025.
ReDoS	Regular Expression Denial of Service — attaque exploitant une regex à complexité catastrophique.
XSS	Cross-Site Scripting — injection de code JavaScript malveillant dans une page.
Injection SQL	Attaque consistant à injecter du SQL dans une requête. Neutralisée par Prisma (requêtes paramétrées).
CVE	Common Vulnerabilities and Exposures — base mondiale des vulnérabilités logicielles connues.
RGPD	Règlement Général sur la Protection des Données (UE 2016/679) — protège les données personnelles.

CNIL	Commission Nationale de l'Informatique et des Libertés — autorité française de contrôle RGPD.
Délibération CNIL	Décision officielle CNIL ayant valeur de recommandation forte (ex. déliv. 2017-012 sur 12 mois d'inactivité).
Élévation de privilège	Vulnérabilité permettant à un utilisateur d'obtenir des droits supérieurs (ex. membre → admin).
Hashage	Transformation à sens unique d'une donnée — utilisé pour stocker les mots de passe (jamais en clair).

12.6 Tests & Qualité

Vitest	Framework de tests unitaires pour Vite — successeur moderne de Jest.
Testing Library	Bibliothèque de tests orientée comportement utilisateur (queries par rôle ARIA, texte visible).
Pyramide de tests	Stratégie : beaucoup de tests unitaires, moins d'intégration, peu d'E2E (end-to-end).
DoD	Definition of Done — critères Scrum qui définissent quand une tâche est « finie ».
Lint / ESLint	Vérification statique du code pour détecter les erreurs et imposer un style cohérent.
Couverture de tests	Pourcentage du code exécuté lors des tests — mesure de l'exhaustivité.

12.7 Déploiement & DevOps

Docker	Plateforme de conteneurisation — isole une application avec ses dépendances dans un conteneur léger.
Docker Compose	Orchestrateur multi-conteneurs local — démarre client + api + db en une commande.
Dockerfile	Recette de construction d'une image Docker (instructions FROM, RUN, COPY, CMD...).
Railway	PaaS (Platform-as-a-Service) — héberge l'application web sans gestion de serveur.
PaaS	Platform-as-a-Service — hébergement abstrait : on déploie du code, le fournisseur gère l'infra.
CI/CD	Continuous Integration / Continuous Deployment — automatisation des tests et du déploiement.
GitHub Actions	Service d'automatisation CI/CD intégré à GitHub — workflows déclenchés par push, PR, etc.
Webhook	URL appelée automatiquement par un événement externe (ex. push GitHub → déploiement Railway).
Caddy	Serveur web moderne — alternative à Nginx avec HTTPS automatique (Let's Encrypt).
Nixpacks	Détecteur automatique de stack technique chez Railway — choisit l'image de base selon le code détecté.

12.8 Workflow & Méthodologie

Scrum	Méthode agile par itérations courtes (sprints) — cérémonies : planning, daily, review, rétro.
Sprint	Itération de développement à durée fixe (1 semaine ici).

Scrum Master	Rôle de facilitation du processus Scrum — pas un chef de projet, mais un coach d'équipe.
Kanban	Tableau visuel de gestion des tâches (colonnes : À faire / En cours / Fini).
MoSCoW	Méthode de priorisation : Must have / Should have / Could have / Won't have.
PR	Pull Request — proposition de fusion de code soumise à revue de l'équipe avant intégration.
Trunk-based	Stratégie Git avec une branche principale unique et des feature branches courtes.
Standup / Daily	Réunion quotidienne courte (15 min max) — chacun dit : hier, aujourd'hui, blocages.
Rétrospective	Réunion de fin de sprint pour identifier les améliorations (méthode Keep / Stop / Start).

12.9 Spécifique projet Cinés Délices

TMDB	The Movie Database — API publique de fiches films/séries (titre, affiche, casting, synopsis).
Sharp	Bibliothèque Node.js de traitement d'images — utilisée pour le redimensionnement et la conversion en WebP.
Multer	Middleware Express pour gérer l'upload de fichiers en multipart/form-data.
node-cron	Bibliothèque de planification de tâches périodiques (syntaxe cron Unix : '0 3 * * 1').
Snapshot TMDB	Copie locale en BDD des données TMDB — évite la dépendance à l'API externe à chaque affichage.
Fuzzy search	Recherche tolérante aux fautes de frappe — utilise la distance de Levenshtein pour matcher les ingrédients.
Levenshtein	Algorithme calculant le nombre minimal d'opérations pour passer d'une chaîne à une autre.
WebP	Format d'image Google — 30 % plus léger que JPEG à qualité équivalente, supporté par tous les navigateurs modernes.
DRAFT / PENDING / PUBLISHED	Les 3 statuts d'une recette : brouillon privé / en attente de modération / publiée publiquement.

Sommaire des annexes

Cette partie regroupe les annexes référencées dans le corps du dossier. Les livrables volumineux (maquettes, diagrammes UML, charte graphique, code complet) sont accessibles sur GitHub via les liens de renvoi indiqués dans chaque annexe.

Annexe 1 — Cartographie technique étendue

- 1.1 Diagramme de composants — vue interne du serveur API
- 1.2 Arborescence du projet (monorepo pnpm)

Annexe 2 — Correspondances routes Frontend ↔ endpoints API

Annexe 3 — Planning Sprint 2, Kanban et workflow Git

- 3.1 Planning Sprint 2 — préparé en tant que Scrum Master
- 3.2 Légende du Kanban
- 3.3 Branches Git associées au Sprint 2
- 3.4 Workflow personnel — un double remote
- 3.5 Erreurs fréquentes rencontrées et solutions
- 3.6 Aide-mémoire — commandes essentielles

Annexe 4 — Use Cases détaillés + diagrammes par acteur

- 4.1 Liste des Use Cases
- 4.2 Use Cases détaillés — sélection des plus représentatifs
- 4.3 Diagrammes de cas d'utilisation par acteur

Annexe 5 — Détail textuel des séquences complexes

→ Contenu intégré au §7.3 du corps du dossier

Annexe 6 — Script SQL généré par Prisma migrate

- 6.1 Création des types ENUM
- 6.2 Tables principales — extrait users + recipes
- 6.3 Index uniques et de performance
- 6.4 Clés étrangères et politiques onDelete

Annexe 7 — Charte graphique complète

- 7.1 Vue d'ensemble de la charte
- 7.2 Palette de couleurs
- 7.3 Typographie
- 7.4 Iconographie
- 7.5 Composants UI
- 7.6 Mise en page — Mobile First

Annexe 8 — Galerie des wireframes principaux

- 8.1 Galerie complète sur GitHub
- 8.2 Sélection de 4 wireframes représentatifs

Annexe 9 — Galerie des maquettes

- 9.1 Galerie complète sur GitHub
- 9.2 Sélection de 4 paires Desktop + Mobile représentatives

Annexe 10 — Code source des composants UI (CP2)

- 10.1 AuthContext.jsx — gestion centralisée de l'authentification
- 10.2 CreateRecipe.jsx — parseTimeToMinutes (parsing tolérant)

Annexe 11 — Code source des composants métier (CP3)

- 11.1 inactivityCheck.js — CRON RGPD 11/12 mois

Annexe 12 — Code source des composants d'accès aux données (CP8)

- 12.1 Fusion d'ingrédients atomique — mergeIngredients
- 12.2 Sélection explicite, catalogue paginé et fuzzy search

Annexe 13 — OWASP Top 10 2025 + tarteaucitron + emails RGPD

- 13.1 Référentiel OWASP Top 10 — édition 2025
- 13.2 Configuration tarteaucitron.js — conformité CNIL
- 13.3 Templates d'emails RGPD — Nodemailer

Annexe 14 — Tests unitaires et tests d'intégration

- 14.1 Tests unitaires Vitest — recipeUtils.test.js (extrait)
- 14.2 Tests d'intégration sécurité — test-api-security.js (extrait)
- 14.3 Synthèse des 7 suites de tests d'intégration

Annexe 15 — Captures du jeu d'essai T01 → T10

Annexe 16 — Déploiement Railway, Docker, CI/CD

- 16.1 Workflow GitHub Actions complet — ci.yml
- 16.2 Dockerfile API — node:22-slim + Prisma 7
- 16.3 Caddyfile client — cache différencié
- 16.4 docker-compose.yml — environnement de développement
- 16.5 Variables d'environnement Railway — checklist
- 16.6 Checklist déploiement pas-à-pas
- 16.7 Problèmes résolus durant la mise en production

Annexe 17 — Captures d'écran : création d'une recette

- 17.1 Correspondance étape ↔ capture (cf. §7 du corps)

Annexe 1 — Cartographie technique étendue

Disponible sur GitHub — cines-delices-docs/Schemas/PNG/

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices-docs/tree/main/Schemas/PNG

Diagramme de composants (vue interne du serveur API), diagramme de déploiement et arborescence complète du monorepo pnpm. Le schéma d'architecture principal reste au §6.2 du corps.

Annexe 2 — Correspondances routes Frontend ↔ endpoints API

Disponible sur GitHub — Documentation Swagger

Cliquez sur le lien :

cines-delicesapi-production.up.railway.app/api/docs

Table complète des correspondances routes Frontend ↔ endpoints API REST, documentée de manière interactive via Swagger (spécification swagger.json à la racine du dépôt cines-delices-docs).

Annexe 3 — Planning Sprint 2, Kanban et workflow Git

Disponible sur GitHub — cines-delices-docs/Gestion-projet/

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices-docs/tree/main/Gestion-projet

Planning détaillé du Sprint 2 (préparé en tant que Scrum Master), extrait du Kanban GitHub, workflow Git personnel et aide-mémoire des commandes. La synthèse figure au §4 du corps.

Annexe 4 — Use Cases détaillés + diagrammes par acteur

Disponible sur GitHub — cines-delices-docs/Schemas/UseCases/

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices-docs/tree/main/Schemas/PNG

github.com/Nathalie-Leduc/cines-delices-docs/tree/main/Documentations

Use Cases détaillés UC01 à UC25 et diagrammes par acteur (visiteur, membre, admin). Le diagramme de synthèse figure au §5.3 ; la création de recette est déroulée bout-en-bout au §7.

Annexe 5 — Détail textuel des séquences complexes

Disponible sur GitHub — cines-delices-docs/Schema/Sequences/

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices-docs/tree/main/Schemas/PNG

Déroulé pas-à-pas de l'autocomplétion TMDb (debounce, proxy, cache, snapshot local). Les diagrammes de séquence des cas significatifs figurent au §5.6 du corps.

Annexe 6 — Script SQL généré par Prisma migrate

Disponible sur GitHub — cines-delices/api/prisma/migrations/

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices/tree/main/api/prisma/migrations

Script SQL complet (CreateTable, CreateIndex, AddForeignKey, types ENUM, politiques onDelete) généré par Prisma migrate. Un extrait (CREATE TABLE recipes) figure au §5.5 du corps.

Annexe 7 — Charte graphique complète

Disponible sur GitHub — cines-delices-docs/Charte-graphique/

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices-docs/tree/main/Charte-graphique

Charte graphique complète : palette de couleurs, typographies (Playfair Display + Inter), iconographie, composants UI (boutons, cartes, hero) et logo.

Annexe 8 — Galerie des wireframes principaux

Lors du Sprint 0, chaque membre de l'équipe a réalisé 2×2 wireframes (2 Desktop + 2 Mobile), soit 20 wireframes pour 10 pages. **J'ai repris l'ensemble pour homogénéiser le rendu** au même modèle et complété les pages manquantes : 16 wireframes au total couvrant tous les écrans principaux de l'application.

8.1 Galerie complète sur GitHub

Pour conserver le dossier papier à un volume raisonnable tout en garantissant l'accès à l'ensemble du travail, les 16 wireframes (Desktop + Mobile homogénéisés) sont disponibles dans le dépôt GitHub dédié à la documentation.

Les 16 wireframes complets

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices-docs/tree/main/Wireframes

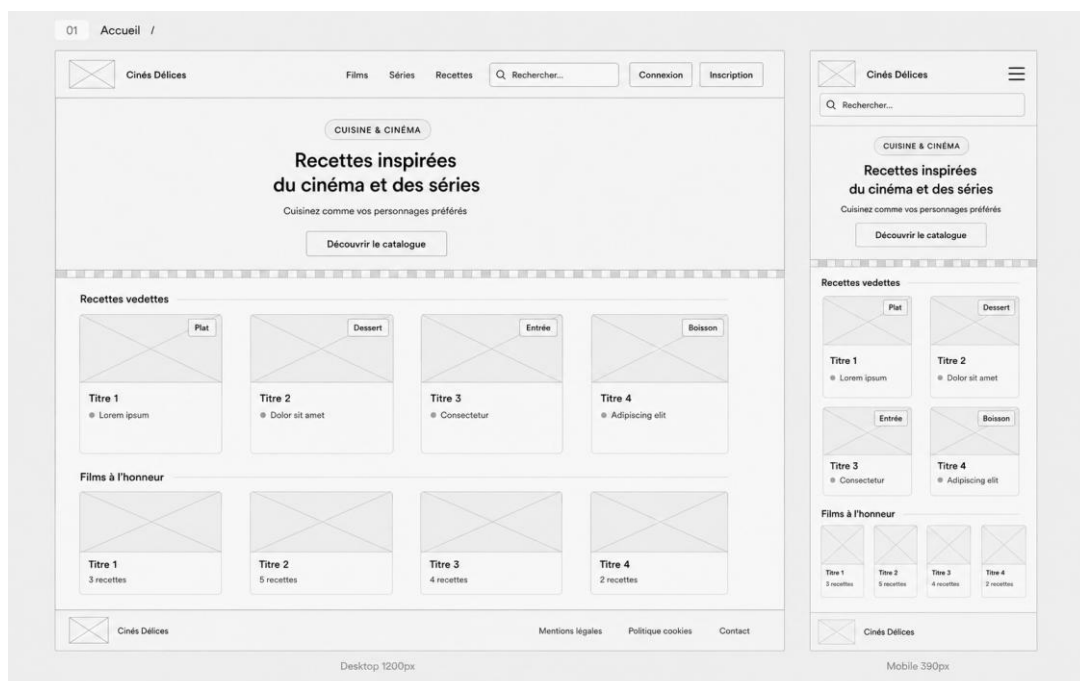
Tous les wireframes (Desktop + Mobile) en PNG haute résolution : accueil, catalogues, détail recette, parcours auth, espace membre, créer recette, back-office admin, mentions légales.

8.2 Sélection de 4 wireframes représentatifs

Les quatre écrans ci-dessous illustrent les principaux types de pages : vitrine publique (accueil), parcours membre (créer une recette), et back-office (validation admin).

WF-01 — Accueil

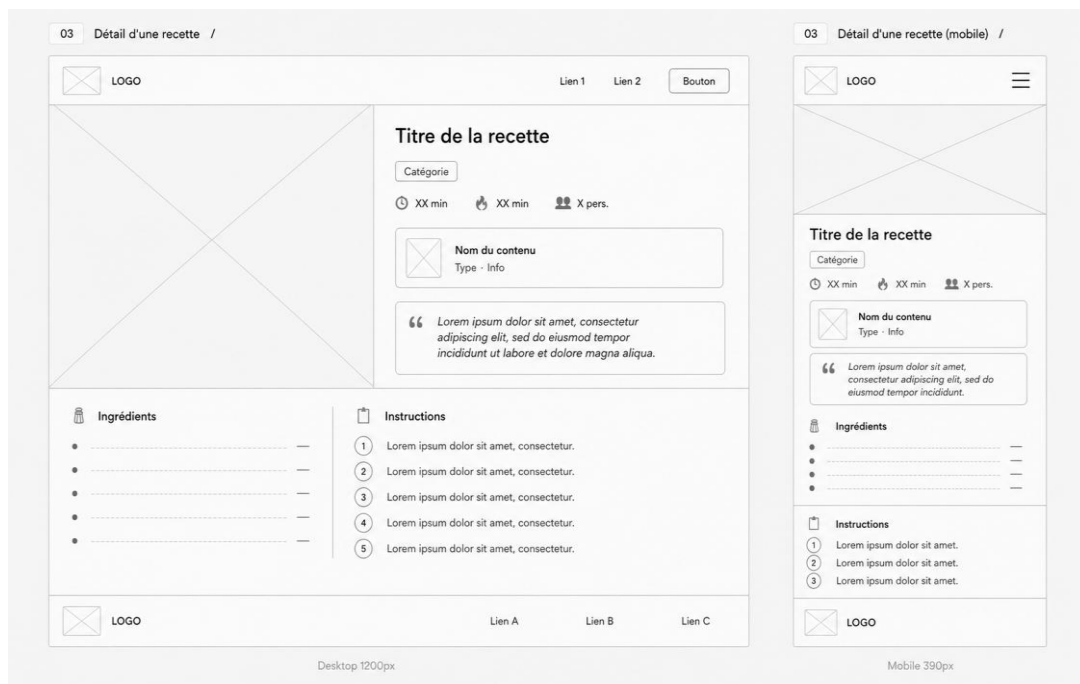
Écran d'entrée du site. Hero immersif avec image culinaire et CTA « Explorer les recettes ». Sections recettes mises en avant (Entrées, Plats, Desserts, Boissons).



WF-01 Accueil — Desktop + Mobile

WF-03 — Détail recette

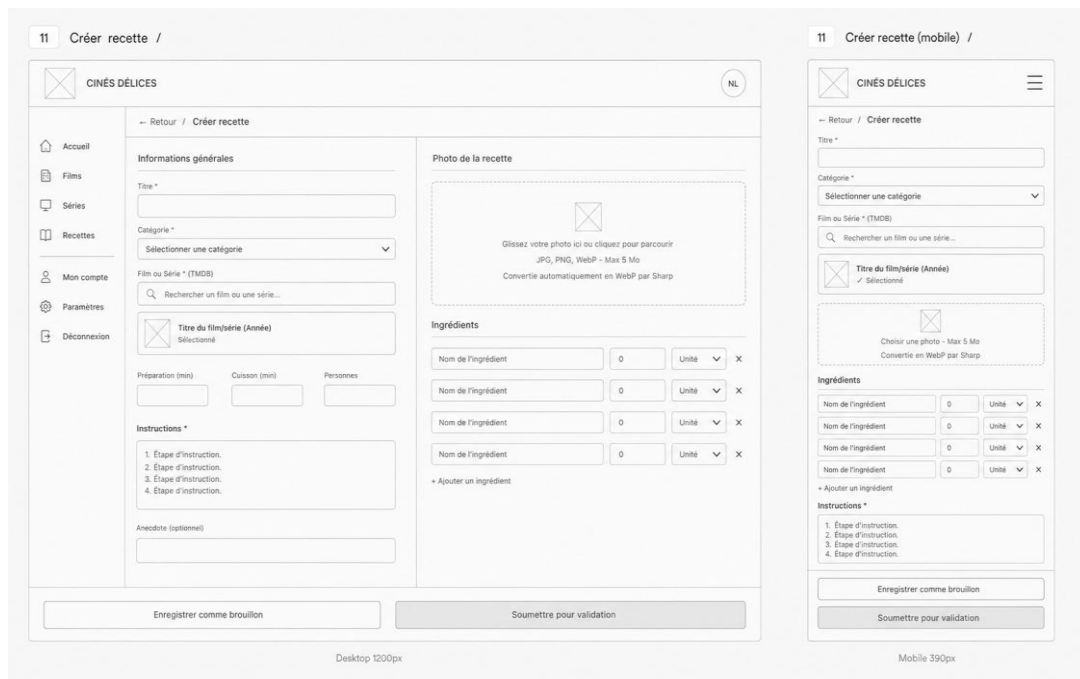
Page de consultation : hero photo, film/série, temps/portions, ingrédients (avec quantités/unités), instructions numérotées. Recettes similaires en bas de page.



WF-03 Détail recette — Desktop + Mobile

WF-11 — Créer recette

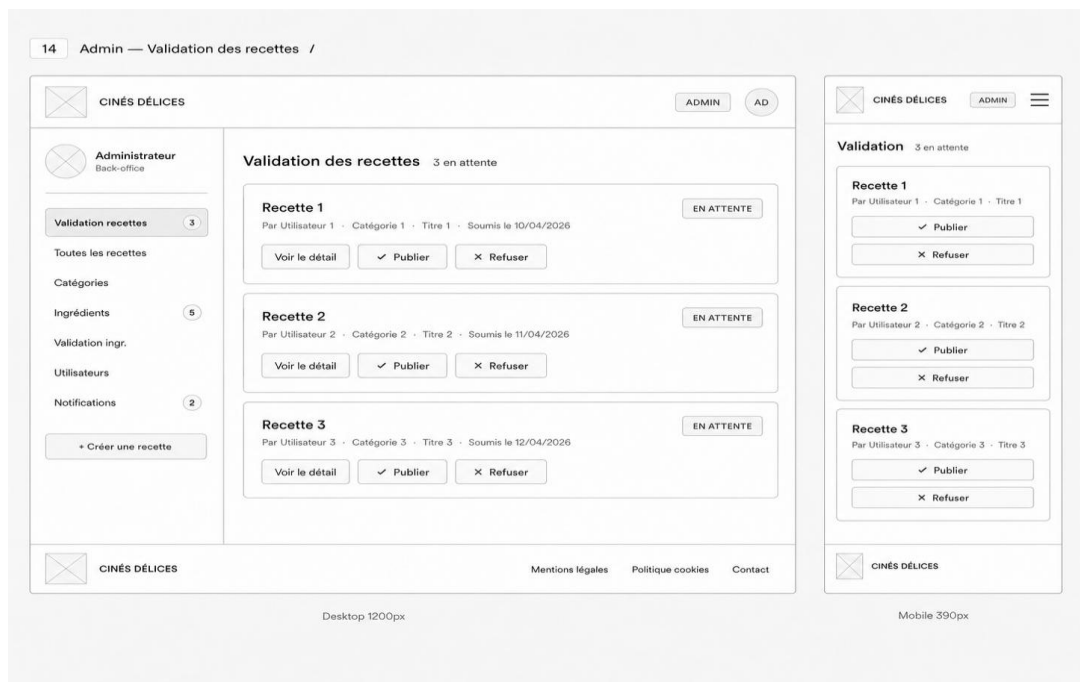
Formulaire complet : titre, catégorie, film TMDB (autocomplétion), ingrédients (fuzzy search), instructions, photo (Multer + Sharp), durées (parseTimeToMinutes).



WF-11 Créer recette — Desktop + Mobile

WF-14 — Admin validation

File des recettes PENDING à modérer. Aperçu, actions Publier / Refuser avec motif obligatoire. Notifications membre déclenchées automatiquement.



WF-14 Admin validation — Desktop + Mobile

Annexe 9 — Galerie des maquettes

Même travail d'homogénéisation que pour les wireframes : 16 maquettes en responsive (Desktop + Mobile), toutes au modèle commun, intégrant la charte graphique (Playfair Display + Inter, palette bordeaux/or/crème).

9.1 Galerie complète sur GitHub

Les 16 maquettes (responsive Desktop/Mobile) sont disponibles dans le dépôt GitHub dédié, en PNG haute résolution.

Les 16 maquettes complètes

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices-docs/tree/main/Maquettes

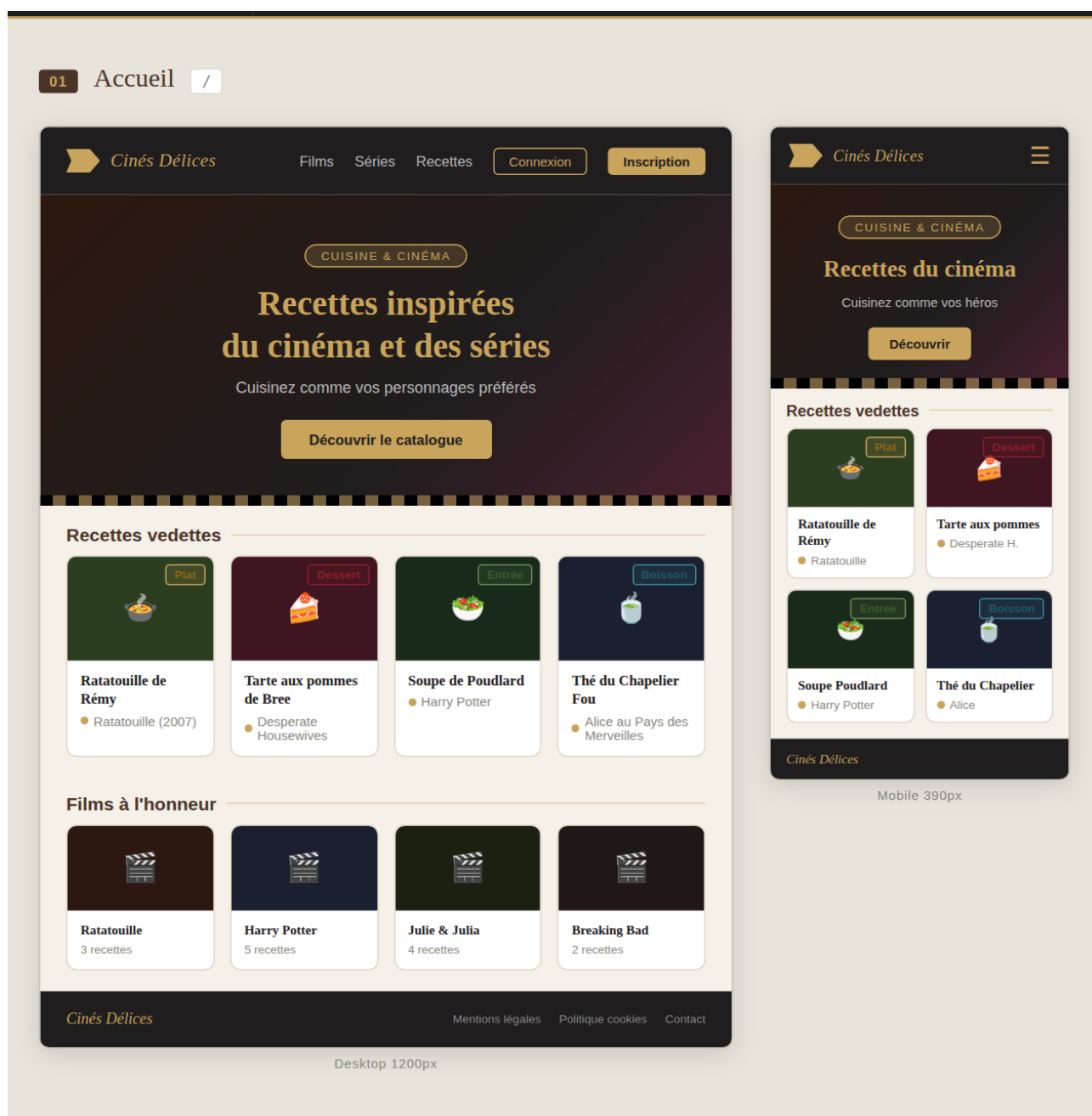
16 maquettes. Tous les écrans du périmètre MVP : vitrine publique, parcours auth, espace membre, créer recette, back-office admin, mentions légales.

9.2 Sélection de 4 paires Desktop + Mobile représentatives

Les quatre écrans ci-dessous présentent côte à côte la version Desktop (en haut) et Mobile (à droite) pour montrer le responsive sur le même écran.

MF-01 — Accueil

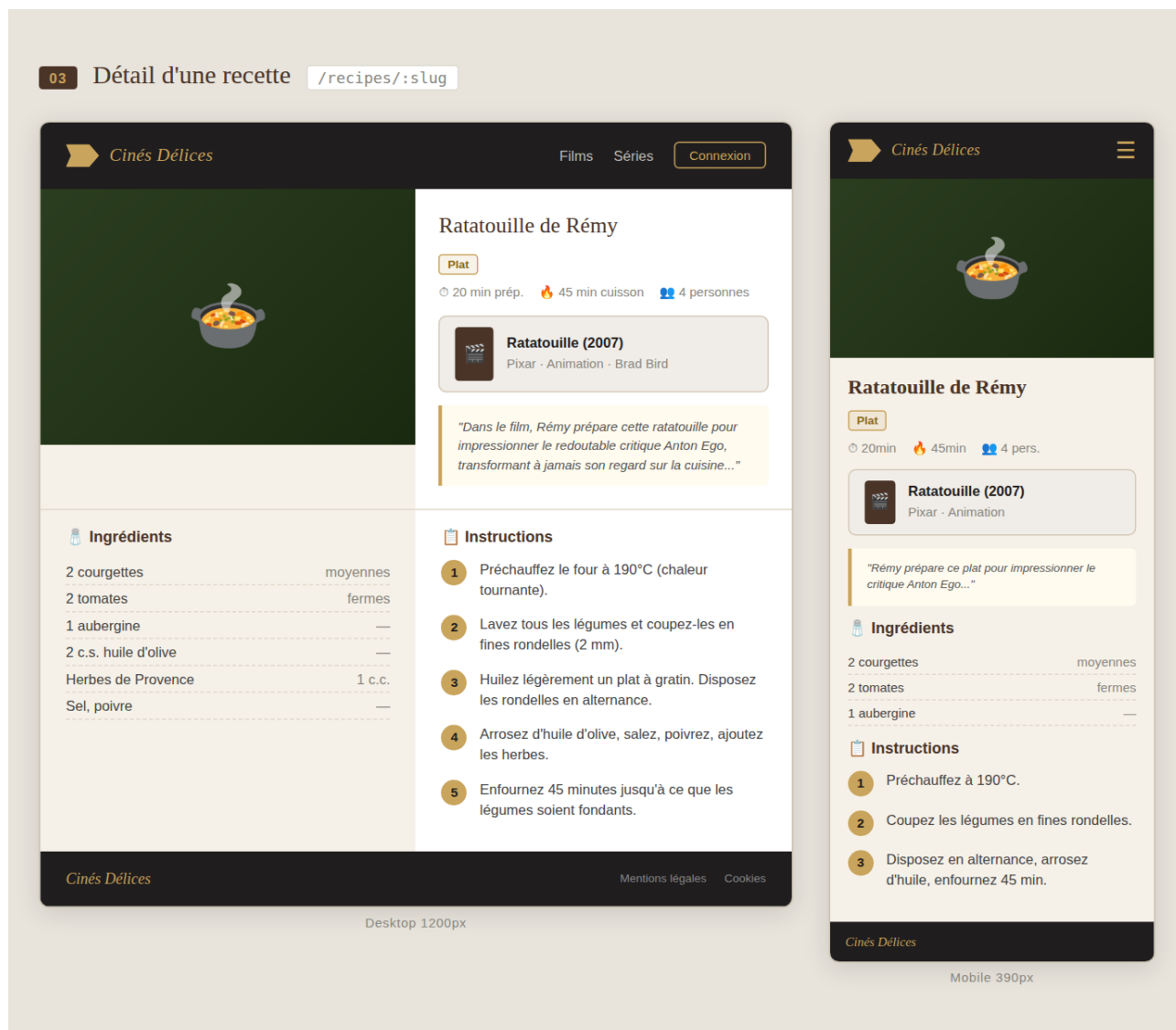
Hero immersif avec image culinaire et slogan « Cuisine le cinéma, savoure les séries ». Sections recettes ordonnées par catégorie.



MF-01 Accueil — Desktop + Mobile

MF-03 — Détail recette

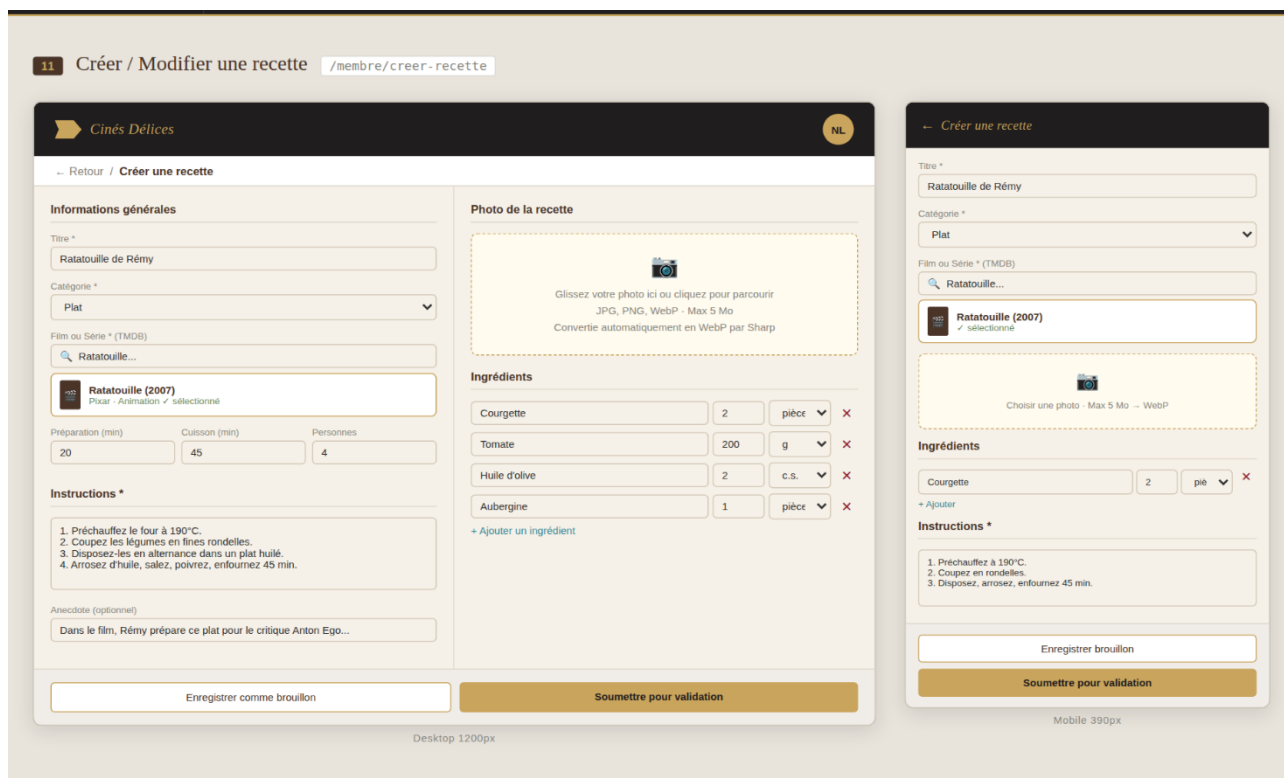
Hero photo, film/série, temps/portions, ingrédients structurés, étapes numérotées avec cercles verts. Recettes similaires en bas.



MF03 Détail d'une recette— Desktop + Mobile

MF-11 — Créer recette

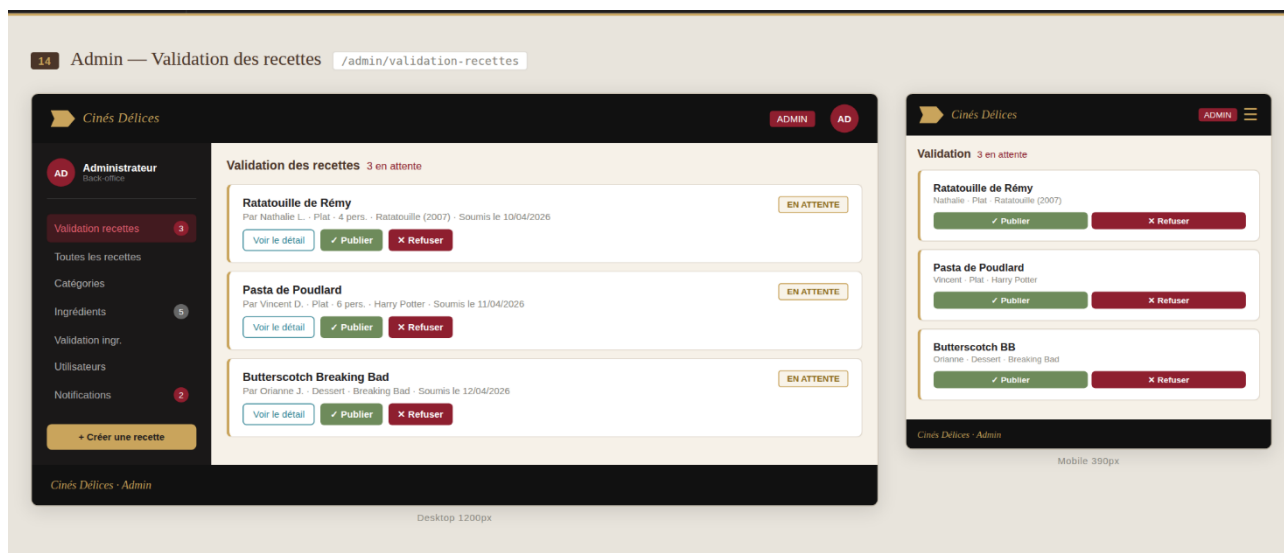
Formulaire avec autocomplétion TMDB, fuzzy search ingrédients, upload image (Multer + Sharp), parseur de temps flexible.



MF11 Créer une recette— Desktop + Mobile

MF-14 — Admin validation

File des PENDING. Aperçu détaillé. Actions Publier / Refuser. Notifications membre automatiques. Badge de comptage dans la sidebar.



MF14 Admin Validation— Desktop + Mobile

Annexe 10 — Code source des composants UI (CP2)

Code complet des composants UI les plus représentatifs, dans l'ordre où ils interviennent dans la fonctionnalité phare « création de recette » (§7 du dossier principal).

10.0 Accès au code complet sur GitHub

Les composants suivants sont disponibles in extenso dans le dépôt code applicatif, sous /client/src : AuthContext, ProtectedRoute, AdminRoute, Navbar (4 useEffect), CreateRecipe (formulaire complet), tarteau citron / CookieConsent, et l'ensemble des hooks et services côté client.

Code source UI complet — /client/src

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices/tree/main/client/src

Tous les composants React : pages, contextes (AuthContext), router (ProtectedRoute, AdminRoute), services API (client.js avec injection JWT), composants réutilisables (Navbar, CookieConsent, RecipeCatalogView), styles SCSS/BEM, tests Vitest.

10.1 AuthContext.jsx — gestion centralisée de l'authentification

Trois mécanismes clés : **(1)** restauration au démarrage avec validation locale du JWT ; **(2)** login/logout enveloppés dans useCallback pour garantir des références stables ; **(3)** hook useAuth() avec garde-fou explicite si utilisé hors AuthProvider.

```
// client/src/contextes/AuthContext.jsx (extrait essentiel)
import { createContext, useCallback, useContext, useMemo, useState } from 'react';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [authState, setAuthState] = useState(getInitialAuthState);

  // login enveloppé dans useCallback : référence STABLE
  const login = useCallback(({ token, user = null }) => {
    const normalizedToken = token?.startsWith('Bearer ') ? token.slice(7) : token;
    if (!normalizedToken || !isJwtValid(normalizedToken)) {
      throw new Error('Token invalide ou expiré');
    }
    localStorage.setItem(AUTH_TOKEN_KEY, normalizedToken);
    if (user) localStorage.setItem(AUTH_USER_KEY, JSON.stringify(user));
    setAuthState({ token: normalizedToken, user, isAuthenticated: true });
  }, []);

  const logout = useCallback(() => {
    localStorage.removeItem(AUTH_TOKEN_KEY);
    localStorage.removeItem(AUTH_USER_KEY);
    setAuthState({ token: null, user: null, isAuthenticated: false });
  }, []);

  const value = useMemo(() => {
    const tokenPayload = decodeJwtPayload(authState.token);
    const role = authState.user?.role ?? tokenPayload?.role ?? null;
    return {
      ...authState, role,
      isAdmin: String(role).toUpperCase() === 'ADMIN',
      login, logout,
    };
  }, [authState, login, logout]);
}
```

```

    return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
  }

// Hook custom avec garde-fou explicite si utilisé hors AuthProvider
export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth doit être utilisé dans AuthProvider');
  return context;
}

```

10.2 CreateRecipe.jsx — parseTimeToMinutes (parsing tolérant)

La fonction accepte trois formats de durée : "1h30", "1:30", "90min". Trois regex testés dans l'ordre après normalisation préalable. Analogie : un assistant qui comprend toutes les façons de dire un temps.

```

// client/src/utils/recipeUtils.js – parseTimeToMinutes
// Analogie : un assistant qui comprend toutes les façons
// de dire un temps et répond toujours en minutes pour la BDD.
export function parseTimeToMinutes(value) {
  if (value === '' || value === null || value === undefined) return undefined;

  // Normalisation : lowercase, suppression des espaces, virgule → point
  const str = String(value).trim().toLowerCase()
    .replace(/\s+/g, '').replace(/,/g, '.');

  // Pattern 1 : "1h30", "1.5h", "2h" – heures et minutes optionnelles
  const hMatch = str.match(/^(\d+(?:\.\d+)?)h(?:\d+)(?:min)?$/);
  if (hMatch) {
    const total = Math.round(parseFloat(hMatch[1]) * 60 + parseInt(hMatch[2] || '0', 10));
    return total > 0 ? total : undefined;
  }

  // Pattern 2 : "1:30", "01:30" – format heure:minute
  const colonMatch = str.match(/^(\d+):(\d+)(?:\d+)?$/);
  if (colonMatch) {
    const total = parseInt(colonMatch[1], 10) * 60 + parseInt(colonMatch[2], 10);
    return total > 0 ? total : undefined;
  }

  // Pattern 3 : "30min", "30mn", "30m", "30" – minutes seules
  const minMatch = str.match(/^(\d+(?:\.\d+)?)?(?:min|mn|m)?$/);
  if (minMatch) {
    const parsed = Math.round(parseFloat(minMatch[1]));
    return Number.isNaN(parsed) || parsed <= 0 ? undefined : parsed;
  }

  return undefined;
}

```

 *ProtectedRoute, AdminRoute, Navbar (debounce 400 ms + 4 useEffect), CookieConsent et tous les autres composants UI sont disponibles dans le dépôt — voir QR code ci-dessus.*

Annexe 11 — Code source des composants métier (CP3)

Middlewares et controllers Express les plus représentatifs, dans l'ordre du flux d'une requête.

11.0 Accès au code complet sur GitHub

Tous les middlewares, controllers, services, jobs CRON, validators Zod et la configuration Express sont disponibles dans le dépôt code applicatif, sous /api/src.

Code source métier complet — /api/src

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices/tree/main/api/src

Middlewares (authMiddleware, adminMiddleware, errorMiddleware, validateMiddleware, recipelImageUploadMiddleware, tmdbRateLimiter), controllers, routes, validators Zod, lib (prisma, mailer, posterService), jobs CRON RGD, configuration Swagger JSDoc.

11.1 inactivityCheck.js — CRON RGD 11/12 mois

Job CRON exécuté chaque lundi à 3h. Deux seuils : 11 mois (email de prévenance laissant 30 jours) et 12 mois (suppression du compte avec email de confirmation). Suppression via prisma.\$transaction : DRAFT/PENDING supprimées puis user supprimé. Les PUBLISHED restent visibles avec userId → NULL grâce à onDelete: SetNull.

```
// api/src/jobs/inactivityCheck.js (extrait)
import cron from 'node-cron';
import { prisma } from '../lib/prisma.js';
import { sendEmail } from '../lib/mailer.js';

// Délais en millisecondes
const ELEVEN_MONTHS_MS = 11 * 30 * 24 * 60 * 60 * 1000;
const TWELVE_MONTHS_MS = 12 * 30 * 24 * 60 * 60 * 1000;

// -----
// Suppression d'un compte inactif
// -----
// Même logique que deleteUser dans adminController :
// - Supprime les recettes DRAFT/PENDING
// - Le User est supprimé → SetNull sur les PUBLISHED
// - Envoie un email de confirmation AVANT la suppression
async function purgeInactiveAccount(user) {
  // Envoyer l'email AVANT la suppression (on a encore l'adresse)
  await sendEmail(buildDeletionEmail(user));

  await prisma.$transaction(async (tx) => {
    await tx.recipe.deleteMany({
      where: {
        userId: user.id,
        status: { in: ['DRAFT', 'PENDING'] },
      },
    });
    await tx.user.delete({ where: { id: user.id } });
  });

  console.log(`[CRON] Compte inactif supprimé : ${user.pseudo} (${user.email})`);
}

// -----
// Job principal – exécuté chaque lundi 3h
```



```
// -----
async function checkInactiveAccounts() {
  const now = new Date();
  const warningThreshold = new Date(now.getTime() - ELEVEN_MONTHS_MS);
  const purgeThreshold = new Date(now.getTime() - TWELVE_MONTHS_MS);

  // 1. Comptes à supprimer (12+ mois)
  // On inclut aussi les comptes JAMAIS connectés dont la création
  // date de plus de 12 mois (comptes créés avant le tracking).
  const accountsToPurge = await prisma.user.findMany({
    where: {
      role: 'MEMBER',
      OR: [
        { lastLoginAt: { lt: purgeThreshold } },
        { lastLoginAt: null, createdAt: { lt: purgeThreshold } },
      ],
    },
  });

  for (const user of accountsToPurge) {
    try {
      await purgeInactiveAccount(user);
    } catch (error) {
      console.error(`[CRON] Erreur suppression ${user.pseudo} :`, error.message);
    }
  }

  // 2. Comptes à prévenir (entre 11 et 12 mois) – logique similaire
  // Cas 1 : lastLoginAt entre 11 et 12 mois
  // Cas 2 : jamais connecté + compte créé entre 11 et 12 mois
  const accountsToWarn = await prisma.user.findMany({
    where: {
      role: 'MEMBER',
      OR: [
        { lastLoginAt: { gte: purgeThreshold, lt: warningThreshold } },
        { lastLoginAt: null,
          createdAt: { gte: purgeThreshold, lt: warningThreshold } },
      ],
    },
  });

  for (const user of accountsToWarn) {
    await sendEmail(buildWarningEmail(user));
  }
}

// -----
// Démarrage du CRON
// -----
export function startInactivityCron() {
  cron.schedule('0 3 * * 1', () => { // chaque lundi 3h
    checkInactiveAccounts().catch(console.error);
  });
}
```

i app.js (Helmet + CORS + trust proxy), recipeImageUploadMiddleware.js (Multer mémoire + Sharp WebP), authController.js (Argon2id + JWT + safeUser), tmdbRateLimiter.js sont disponibles dans le dépôt — voir QR code ci-dessus.

Annexe 12 — Code source des composants d'accès aux données (CP8)

Quatre développements significatifs côté Prisma 7 + PostgreSQL : catalogue paginé, fusion d'ingrédients atomique, recherche floue + normalisation, sélection explicite.

12.0 Accès au code complet sur GitHub

Tous les controllers utilisant Prisma 7 sont dans `/api/src/controllers` du dépôt code applicatif.

Controllers Prisma complets — `/api/src/controllers`

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices/tree/main/api/src/controllers

`recipesController` (catalogue paginé, `$transaction` `createRecipe`, mises à jour atomiques), `adminIngredientsController` (merge atomique), `ingredientsController` (fuzzy search + normalisation singulier), `authController` (select sans `passwordHash`), `mediaController`, `tmdbController`.

12.1 Fusion d'ingrédients atomique — `mergeIngredients`

Quand l'admin fusionne « citrons » → « citron », il faut rediriger toutes les `recipe_ingredients` vers la cible AVANT de supprimer la source. La `$transaction` détecte aussi les doublons (cas où une recette utilise déjà l'ingrédient cible). Si une étape échoue, tout est rollback → pas de demi-fusion qui laisserait la BDD incohérente.

```
// api/src/controllers/adminIngredientsController.js (extrait : mergeIngredients)
// Fusion atomique : redirige les recipe_ingredients de la source vers la cible
// AVANT de supprimer la source. Détecte les doublons (recette utilisant
// déjà l'ingrédient cible). Tout est rollback si une étape échoue.
const updatedTarget = await prisma.$transaction(async (tx) => {
  const sourceLinks = await tx.recipeIngredient.findMany({
    where: { ingredientId: sourceId },
    select: { recipeId: true, quantity: true, unit: true },
  });

  for (const link of sourceLinks) {
    // Cas critique : la recette utilise DÉJÀ l'ingrédient cible
    const existingTargetLink = await tx.recipeIngredient.findFirst({
      where: { recipeId: link.recipeId, ingredientId: targetId },
    });

    if (existingTargetLink) {
      // Doubleton → on supprime simplement la ligne source
      await tx.recipeIngredient.deleteMany({
        where: { recipeId: link.recipeId, ingredientId: sourceId },
      });
    } else {
      // Pas de doubleton → on bascule l'ingredientId vers la cible
      await tx.recipeIngredient.updateMany({
        where: { recipeId: link.recipeId, ingredientId: sourceId },
        data: { ingredientId: targetId },
      });
    }
  }
});

// Marquer comme lue la notification de soumission de l'ingrédient source
```

```

await tx.notification.updateMany({
  where: {
    userId: req.user.id,
    isRead: false,
    message: `Nouvel ingrédient soumis: ${source.nom}`,
  },
  data: { isRead: true },
});

// Maintenant on peut supprimer la source (plus de liens)
await tx.ingredient.delete({ where: { id: sourceId } });
return tx.ingredient.findUnique({ where: { id: targetId } });
});

```

12.2 Sélection explicite, catalogue paginé et fuzzy search

Trois patterns Prisma 7 qui se complètent : sélection explicite pour ne jamais fuiter de champ sensible, catalogue paginé en \$transaction pour garantir la cohérence pagination/comptage, recherche floue côté client pour rattraper les fautes de frappe.

12.2.1 Sélection explicite — jamais SELECT *

Pour la lecture, Prisma select explicite n'expose jamais les champs sensibles (notamment passwordHash). Exemple sur /api/auth/me :

```

// api/src/controllers/authController.js - getMe
// Pour la lecture, Prisma 'select' explicite n'expose jamais les champs
// sensibles (passwordHash, resetToken). 'omit' aurait pu marcher mais
// 'select' rend l'intention évidente à la lecture du code.
const user = await prisma.user.findUnique({
  where: { id: req.user.id },
  select: {
    id: true, email: true, nom: true, pseudo: true, role: true, createdAt: true,
    _count: { select: { recipes: true } },
    // passwordHash : VOLONTAIREMENT ABSENT - jamais retourné
    // resetToken : VOLONTAIREMENT ABSENT - usage interne uniquement
    // lastLoginAt : VOLONTAIREMENT ABSENT - interne RGD
  },
});

```

12.2.2 Catalogue paginé avec filtres dynamiques

La route publique GET /api/recipes accepte trois filtres optionnels (catégorie, recherche full-text, slug média), un page/limit avec hardcap 50, et garantit l'atomicité du couple findMany + count grâce à \$transaction — la pagination affichée correspond toujours exactement aux résultats.

```

// api/src/controllers/recipesController.js - getAllPublishedRecipes
// Catalogue paginé public : hardcap 50 résultats/page, filtres optionnels
// (catégorie, recherche, slug média), atomicité findMany + count via $transaction.
export const getAllPublishedRecipes = asyncHandler(async (req, res) => {
  const page = Number(req.query.page || 1);
  const requestedLimit = Number(req.query.limit || 12);
  const limit = Math.min(Math.max(requestedLimit, 1), 50); // hardcap 50
  const skip = (page - 1) * limit;

  // Filtres dynamiques construits selon les query params reçus
  const andFilters = [{ status: 'PUBLISHED' }];
  if (req.query.category) {

```

```

    andFilters.push({ OR: [
      { categoryId: String(req.query.category) },
      { category: { nom: { equals: req.query.category, mode: 'insensitive' } } },
    ]});
  }
  if (req.query.q) {
    andFilters.push({ OR: [
      { titre: { contains: req.query.q, mode: 'insensitive' } },
      { category: { nom: { contains: req.query.q, mode: 'insensitive' } } },
      { media: { titre: { contains: req.query.q, mode: 'insensitive' } } },
    ]});
  }
  if (req.query.mediaSlug) {
    andFilters.push({ media: { slug: String(req.query.mediaSlug) } });
  }

  // $transaction : findMany + count en une seule unité de cohérence –
  // garantit que la pagination affichée correspond exactement aux résultats.
  const [recipes, totalItems] = await prisma.$transaction([
    prisma.recipe.findMany({
      where: { AND: andFilters },
      orderBy: { createdAt: 'desc' },
      skip, take: limit,
      include: { ...recipeRelationsInclude, user: { select: { id: true, pseudo: true } } },
    }),
    prisma.recipe.count({ where: { AND: andFilters } }),
  ]);

  const totalPages = totalItems === 0 ? 0 : Math.ceil(totalItems / limit);
  res.json({
    recipes,
    pagination: { page, limit, totalItems, totalPages,
      hasNextPage: page < totalPages,
      hasPreviousPage: page > 1 && totalPages > 0 },
  });
});

```

12.2.3 Recherche floue — Levenshtein côté client

Pour la création de recettes, le membre tape parfois un ingrédient avec une faute (« citon » au lieu de « citron »). La recherche normale (contains insensitive) ne le trouverait pas. Le client lance alors une 2^e requête avec les 3 premières lettres en préfixe, puis applique localement la distance de Levenshtein pour classer les candidats.

```

// client/src/pages/CreateRecipe/CreateRecipe.jsx – fuzzy search
// Distance de Levenshtein = nombre minimal d'opérations (ajout, suppression,
// substitution d'un caractère) pour transformer un mot en un autre.
// Analogie : c'est comme un correcteur orthographique de cuisine.
// "citon" → "citron" : distance 1 (ajouter 'r')
// "tomattes" → "tomates" : distance 1 (supprimer un 't')
function levenshtein(a, b) {
  const matrix = Array.from({ length: b.length + 1 }, (_, i) =>
    Array.from({ length: a.length + 1 }, (_, j) => (i === 0 ? j : j === 0 ? i : 0)));
  for (let i = 1; i <= b.length; i++) {
    for (let j = 1; j <= a.length; j++) {
      matrix[i][j] = a[j - 1] === b[i - 1]
        ? matrix[i - 1][j - 1]
        : 1 + Math.min(matrix[i - 1][j - 1], matrix[i][j - 1], matrix[i - 1][j]);
    }
  }
}

```

```

    }
  }
  return matrix[b.length][a.length];
}

// Seuil de distance selon la longueur du mot :
//   ≤ 3 lettres → pas de fuzzy (trop de faux positifs sur les mots courts)
//   4-5 lettres → distance max 1 ("citon" → "citron" ☑)
//   ≥ 6 lettres → distance max 2 ("tomattes" → "tomates" ☑)
function isFuzzyMatch(query, candidate) {
  const q = normalizeForFuzzy(query);
  const c = normalizeForFuzzy(candidate);
  if (q.length <= 3) return false;
  const maxDist = q.length <= 5 ? 1 : 2;
  return levenshtein(q, c) <= maxDist;
}

```

12.2.4 Normalisation au singulier côté serveur

Pour éviter d'avoir « citron » ET « citrons » en doublon dans la BDD, le serveur normalise systématiquement les noms d'ingrédients en singulier avant findOrCreate. Liste d'exceptions pour les mots déjà au singulier qui se terminent par « s » (riz, noix, ananas, brocolis...). Les noms composés (avec espace) sont laissés intacts.

```

// api/src/controllers/ingredientsController.js – normalisation singulier
// Évite d'avoir "citron" ET "citrons" en doublon dans la BDD.
function normalizeIngredientName(name) {
  const str = String(name || '').trim().toLowerCase()
    .normalize('NFD').replace(/[\u0300-\u036f]/g, '');

  // Noms composés (avec espace) : pas de singularisation
  // "fraise des bois" → "fraise des bois" ☑ (et non "fraise des boi" ✕)
  if (str.includes(' ')) return str;

  // Mots déjà au singulier dont la forme se termine par "s"
  const exceptions = new Set([
    'riz', 'noix', 'ananas', 'brocolis', 'radis', 'mais', 'pois',
    'fois', 'buis', 'tapas', 'papas', 'colis',
  ]);
  if (exceptions.has(str)) return str;

  // Cas général : on retire le "s" final si > 3 lettres
  if (str.endsWith('s') && str.length > 3) return str.slice(0, -1);
  return str;
}

```

Annexe 13 — OWASP Top 10 2025 + tarteau citron + emails RGPD

13.1 Référentiel OWASP Top 10 — édition 2025

L'OWASP (Open Web Application Security Project) publie tous les 4 ans la liste des 10 risques web les plus critiques. Cinés Délices couvre les 10 catégories de la mise à jour 2025.

Rang	Vulnérabilité	Contre-mesure Cinés Délices
A01:2025	Broken Access Control	authMiddleware + adminMiddleware + ownership RG-03 (membre limité à ses propres recettes)
A02:2025	Cryptographic Failures	Argon2id (premier choix OWASP) + JWT HS256 + HTTPS Railway + aucune donnée sensible en clair
A03:2025	Injection	Prisma ORM (requêtes paramétrées par construction) + Zod validation entrées
A04:2025	Insecure Design	Proxy TMDB server-side + architecture découplée + variables d'environnement (jamais committées)
A05:2025	Security Misconfiguration	Helmet.js + CORS restreint à CLIENT_URL + NODE_ENV=production
A06:2025	Vulnerable Components	pnpm audit avant chaque release · 0 critique = critère bloquant DoD
A07:2025	Authentication Failures	Rate-limit 5 tentatives/15 min login + 30 req/min TMDB + JWT expirant 7 h
A08:2025	Data Integrity Failures	Zod côté serveur + contraintes FK + onDelete configurés
A09:2025	Logging Failures	Morgan (dev) + logs Railway (prod) + stack traces masquées
A10:2025	Server-Side Request Forgery	Proxy TMDB validé en amont + pas d'URL fournie par l'utilisateur côté serveur

13.2 Configuration tarteaucitron.js — conformité CNIL

Configuration de la bibliothèque tarteaucitron.js qui gère le bandeau de consentement cookies. Conformité CNIL : bouton « Tout refuser » aussi visible et accessible que « Tout accepter » (option DenyAllCta:true), highPrivacy:true (aucun cookie déposé sans consentement explicite). Le cookie JWT d'authentification est strictement technique et n'est pas soumis au consentement (article 82 loi Informatique & Libertés).

```
// client/src/components/CookieConsent/CookieConsent.jsx (extrait : init)
// Délibération CNIL n°2020-091 : "Refuser tout" aussi visible que "Accepter tout"
window.tarteaucitron.init({
  // Lien vers la politique des cookies
  privacyUrl: '/politique-cookies',

  // Position du bandeau
  bodyPosition: 'bottom',
  orientation: 'bottom',

  // Identifiant du cookie de consentement
  hashtag: '#tarteaucitron',
  cookieName: 'tarteaucitron',

  // ⚠ OBLIGATION CNIL : expiration max 6 mois (182 jours)
  cookieExpires: 182,

  // Pas de petit bandeau persistant en bas à droite
  showAlertSmall: false,
  showIcon: true,
```

```

iconPosition: 'BottomLeft',

// ⚠️ RGPD strict : aucun cookie avant consentement explicite
serviceDefaultState: 'wait',

// ⚠️ OBLIGATION CNIL : refuser doit être aussi simple qu'accepter
highPrivacy: true,
DenyAllCta: true,
AcceptAllCta: true,

// Accessibilité – respect du Do-Not-Track navigateur
handleBrowserDNTRequest: false,
removeCredit: false,
reloadThePage: false,
});

// Service 1 : Cookie JWT – technique, pas de consentement nécessaire
// (article 82 loi Informatique & Libertés)
window.tarteacitron.services.cinesdelicesjwt = {
  key: 'cinesdelicesjwt',
  type: 'other',
  name: 'Authentification Cinés Délices',
  needConsent: false, // strictement nécessaire
  cookies: ['token'],
  readmoreLink: '/politique-cookies',
};

// Service 2 : TMDB – service tiers, consentement requis
window.tarteacitron.services.tmdb = {
  key: 'tmdb',
  type: 'api',
  name: 'TMDB (The Movie Database)',
  needConsent: true, // déclenche le bandeau
  cookies: [],
  uri: 'https://www.themoviedb.org/privacy-policy',
};

```

13.3 Templates d'emails RGPD — Nodemailer

Quatre déclencheurs d'email automatiques, en style cinéma-cohérent (bordeaux #8E1F2F sur beige #FAF7F2, emoji 🎬). Envoi via Nodemailer : Mailtrap en développement (capture sans envoi réel), Brevo en production (SMTP authentifié SPF/DKIM).

Configuration Nodemailer

```

// api/src/lib/mailer.js
import nodemailer from 'nodemailer';

// Configuration unique réutilisée par les 4 templates
// Mailtrap en dev (capture sans envoi réel)
// Brevo en prod (SMTP authentifié SPF/DKIM)
const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST, // sandbox.smtp.mailtrap.io | smtp-relay.brevo.com
  port: Number(process.env.SMTP_PORT), // 2525 (Mailtrap) | 587 (Brevo)
  auth: {
    user: process.env.SMTP_USER,
    pass: process.env.SMTP_PASSWORD,
  }
});

```

```

    },
  });

export async function sendEmail({ to, subject, html }) {
  return transporter.sendMail({
    from: process.env.SMTP_FROM || 'Cinés Délices <noreply@cinesdelices.fr>',
    to, subject, html,
  });
}

```

Template — prévenance 11 mois (extrait)

```

// api/src/jobs/inactivityCheck.js - buildWarningEmail (extrait)
function buildWarningEmail(user) {
  return {
    to: user.email,
    subject: 'Cinés Délices – Votre compte sera bientôt supprimé',
    html: `
      <div style="font-family: Arial, sans-serif; max-width: 600px;
        margin: 0 auto; background: #FAF7F2;
        padding: 32px; border-radius: 8px;">
        <div style="text-align: center; margin-bottom: 24px;">
          <h1 style="color: #8E1F2F; font-size: 24px; margin: 0;">
            🍿 Cinés Délices
          </h1>
        </div>
        <h2 style="color: #2C2C2A; font-size: 18px;">
          Bonjour ${user.pseudo},
        </h2>
        <p style="color: #444; line-height: 1.6;">
          Nous constatons que vous ne vous êtes pas connecté(e) à
          <strong>Cinés Délices</strong> depuis plus de 11 mois.
        </p>
        <p style="color: #444; line-height: 1.6;">
          Conformément à notre politique de protection des données personnelles
          (RGPD) et aux recommandations de la CNIL, <strong>votre compte sera
          automatiquement supprimé dans 30 jours</strong> si vous ne vous
          reconnectez pas.
        </p>
        <p style="color: #444; line-height: 1.6;">
          Vos recettes publiées resteront visibles de manière anonyme.
        </p>
        <div style="text-align: center; margin: 32px 0;">
          <a href="${CLIENT_URL}/login"
            style="background: #8E1F2F; color: #FAF7F2;
            padding: 14px 28px; text-decoration: none;
            border-radius: 6px; font-weight: bold;
            display: inline-block;">
            Me reconnecter
          </a>
        </div>
      </div>
    `;
  };
}

```

Captures Mailtrap

i Les captures des 4 emails reçus dans Mailtrap (Reset MDP, Confirmation MDP, Prévenance 11 mois, Suppression 12 mois) sont déjà présentées en §8.5.4 du dossier principal. Cette annexe se concentre sur les configurations et templates côté serveur.

Annexe 14 — Tests unitaires et tests d'intégration

Deux niveaux de tests représentatifs : (1) tests unitaires Vitest qui valident les fonctions utilitaires côté client — exécutés à chaque PR via le job CI/CD test-client ; (2) tests d'intégration sécurité écrits en node:assert + fetch, exécutés contre l'API Railway en environnement réel.

14.0 Accès au code des tests sur GitHub

Les deux fichiers de test complets sont disponibles dans le dépôt code applicatif.

Tests complets — /api/tests + /client/src/tests

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices/tree/main/api/tests

test-api.js (tests d'intégration : auth, workflow recettes, admin, RGPD, fuzzy search, pagination) et test-api-security.js (734 lignes : headers Helmet, injections SQL/XSS, JWT falsifié, DoS basique). Les tests Vitest côté client (recipeUtils.test.js — 392 lignes) sont dans /client/src/tests.

14.1 Tests unitaires Vitest — recipeUtils.test.js (extrait)

Fichier complet : client/src/tests/recipeUtils.test.js (392 lignes, 50+ assertions). Trois groupes de tests : parseTimeToMinutes (parsing tolérant — voir aussi §9.3.1 du dossier principal), formatMinutes (analogie « horloge de cuisine »), normalizeCategoryLabel (analogie « traducteur de menu »).

```
// client/src/tests/recipeUtils.test.js (extraits)
import { describe, expect, it } from 'vitest';
import { parseTimeToMinutes } from '../utils/recipeUtils.js';

// -----
// 1. parseTimeToMinutes – voir aussi §9.3.1 du dossier principal
// Analogie : un assistant qui comprend toutes les façons
// de dire un temps (1h30, 1:30, 90min, 90, 1.5h)
// et répond toujours en minutes pour la BDD.
// -----
describe('parseTimeToMinutes', () => {
  it('comprend "1h30" → 90', () => {
    expect(parseTimeToMinutes('1h30')).toBe(90);
  });
  it('comprend "1.5h" → 90 (heures décimales)', () => {
    expect(parseTimeToMinutes('1.5h')).toBe(90);
  });
  it('comprend "1,5h" → 90 (virgule acceptée)', () => {
    expect(parseTimeToMinutes('1,5h')).toBe(90);
  });
  it('comprend "1:30" → 90', () => {
    expect(parseTimeToMinutes('1:30')).toBe(90);
  });
  it('comprend "30min" → 30', () => {
    expect(parseTimeToMinutes('30min')).toBe(30);
  });
});
```

```

it('comprend "30mn" → 30 (suffixe français)', () => {
  expect(parseTimeToMinutes('30mn')).toBe(30);
});
it('ignore les espaces autour et au milieu', () => {
  expect(parseTimeToMinutes(' 1h 30 ')).toBe(90);
});
it('accepte les majuscules', () => {
  expect(parseTimeToMinutes('1H30')).toBe(90);
});
it('rejette une chaîne vide', () => {
  expect(parseTimeToMinutes('')).toBeUndefined();
});
it('rejette une valeur non numérique', () => {
  expect(parseTimeToMinutes('abc')).toBeUndefined();
});
it('rejette "0" (durée nulle)', () => {
  expect(parseTimeToMinutes('0')).toBeUndefined();
});
});

// -----
// 2. formatMinutes (RecipeCard)
// Analogie : comme une horloge de cuisine –
// 90 minutes → "1h30min", 60 → "1h", 30 → "30 min"
// -----
describe('formatMinutes', () => {
  it('retourne null pour une valeur nulle ou zéro', () => {
    expect(formatMinutes(0)).toBeNull();
    expect(formatMinutes(null)).toBeNull();
    expect(formatMinutes(-5)).toBeNull();
  });
  it('formate les minutes sous 60', () => {
    expect(formatMinutes(30)).toBe('30 min');
    expect(formatMinutes(59)).toBe('59 min');
  });
  it('formate les heures pile', () => {
    expect(formatMinutes(60)).toBe('1h');
    expect(formatMinutes(120)).toBe('2h');
  });
  it('formate les heures et minutes', () => {
    expect(formatMinutes(90)).toBe('1h30min');
    expect(formatMinutes(125)).toBe('2h5min');
  });
});

// -----
// 3. normalizeCategoryLabel – recipeCatalog.shared.js
// Analogie : comme un traducteur de menu –
// "entree", "Entrée", "ENTREE" → toujours "Entrée"
// -----
describe('normalizeCategoryLabel', () => {
  it('normalise les variantes de "Entrée"', () => {
    expect(normalizeCategoryLabel('entree')).toBe('Entrée');
    expect(normalizeCategoryLabel('ENTREE')).toBe('Entrée');
  });
  it('normalise les autres catégories', () => {
    expect(normalizeCategoryLabel('plat')).toBe('Plat');
    expect(normalizeCategoryLabel('dessert')).toBe('Dessert');
    expect(normalizeCategoryLabel('boisson')).toBe('Boisson');
  });
});

```

```
});
```

14.2 Tests d'intégration sécurité — test-api-security.js (extrait)

Fichier complet : api/tests/test-api-security.js (734 lignes). Exécutés contre l'API Railway via fetch + node:assert. Couvre headers Helmet, injections SQL/XSS, DoS basique, JWT falsifié et élévation de privilège.

```
// api/tests/test-api-security.js (extraits)
// Usage : API_BASE_URL=https://... node api/tests/test-api-security.js
// Exécuté contre l'API Railway via le job CI/CD test-api-integration
import assert from 'node:assert/strict';

const API_BASE_URL = process.env.API_BASE_URL || 'http://localhost:3000';

// -----
// Test 1 : Headers HTTP de sécurité (Helmet)
// -----
async function testSecurityHeaders() {
  const { response } = await request('/api/health');
  assert.ok(response.headers.get('x-content-type-options'),
    'X-Content-Type-Options (Helmet)');
  assert.ok(response.headers.get('x-frame-options'),
    'X-Frame-Options (Helmet)');
  assert.ok(
    response.headers.get('x-xss-protection') ||
    response.headers.get('content-security-policy'),
    'Protection XSS (CSP ou X-XSS-Protection)'
  );
}

// -----
// Test 2 : Injection SQL
// On envoie une tentative classique : ' OR 1=1 --
// Prisma utilise des requêtes paramétrées par construction → impossible
// -----
async function testSqlInjection() {
  const { response } = await request('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      email: "' OR 1=1 --", // tentative classique
      password: 'anything',
    }),
  });
  assert.notEqual(response.status, 200,
    "L'injection SQL ne doit pas retourner 200");
  // → 400 (Zod refuse car ce n'est pas un format email)
  // → ou 401 – jamais 200
}

// -----
// Test 3 : JWT falsifié – élévation de privilège grave
// On encode un vrai header JWT + un payload prétendant être ADMIN,
// puis on bidouille la signature. Si le middleware n'invalide PAS
// ce token, c'est une élévation de privilège (vol d'identité admin).
// -----
async function testJwtForgery() {
```

```

const fakeToken = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9' // header
                  + '.eyJpZCI6IjEiLCJyb2x1IjoiaURNSU4ifQ' // payload {role:ADMIN}
                  + '.fake_signature_ici';                 // signature bidonnée
const { response } = await request('/api/auth/me', {
  headers: { Authorization: `Bearer ${fakeToken}` },
});
assert.equal(response.status, 401, 'Token falsifié doit retourner 401');
}

// -----
// Test 4 : DoS basique — input excessivement long
// Si le serveur n'a aucune limite, un input de 10 000 caractères
// pourrait causer un crash ou un timeout.
// -----
async function testLongInput() {
  const longString = 'a'.repeat(10000);
  const { response } = await request('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email: longString, password: longString }),
  });
  assert.notEqual(response.status, 200);
  assert.ok(response.status < 500,
    "Pas de crash 500 sur input excessivement long");
}

```

14.3 Synthèse des 7 suites de tests d'intégration

Le fichier test-api.js regroupe 7 suites de tests d'intégration exécutées contre l'API Railway via le job CI/CD test-api-integration (uniquement sur push main, après validation lint + tests unitaires).

Suite	Fichier	Ce qu'elle teste
Auth	test-api.js	register, login, getMe, deleteMe, reset MDP — toutes les routes /api/auth/*
Workflow recettes	test-api.js	DRAFT → submit → PENDING → publish → PUBLISHED + reject + notifications
Admin	test-api.js	CRUD recettes/catégories/ingrédients/users, fusion ingrédients, validation PENDING
Sécurité OWASP	test-api-security.js	Helmet headers, injections SQL/XSS, JWT falsifié, DoS basique, accès non autorisé
RGPD	test-api.js	Suppression compte (DRAFT/PENDING supprimées, PUBLISHED SetNull), notifications cascade
Fuzzy search	test-api.js	Recherche insensitive contains, normalisation singulier, exceptions (riz, noix...)
Pagination	test-api.js	Filtres dynamiques catégorie/recherche/média, hardcap 50, atomicité findMany+count

Annexe 15 — Captures du jeu d'essai T01 → T10

Cette annexe accompagne le jeu d'essai détaillé en §10 du dossier principal. Pour chaque cas de test, on retrouve : (1) l'action testée, (2) la requête envoyée (URL + headers), (3) la capture d'écran de la réponse, (4) le statut final.

Captures complètes du jeu d'essai sur GitHub

Les captures d'écran prises lors de l'exécution du jeu d'essai (panneau admin, notifications membre, emails Mailtrap, navigateur Chrome) sont disponibles dans le dépôt documentation.

Captures du jeu d'essai T01 → T10

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices-docs/tree/main/Jeu-essai

Captures d'écran de chaque étape T01 à T10 (connexion, création de recette, soumission, validation admin, refus avec motif, notification membre, tentatives non autorisées) et captures Mailtrap des 4 emails RGPD applicatifs.

Méthodologie d'exécution

Tous les tests ont été exécutés **manuellement** le 28 avril 2026 sur l'environnement Railway de production, avec les comptes du seed v4 : *admin@cinesdelices.fr* (role ADMIN), *marie@cinesdelices.fr* (role MEMBER). Outils utilisés : navigateur Chrome (parcours UI) et l'extension API Rest Client pour VS Code (vérifications API directes).

Cas T01 — Connexion membre

Élément	Valeur
Requête	POST /api/auth/login
Body	{ "email": "marie@cinesdelices.fr", "password": "Member1234!" }
Attendu	200 OK + token JWT valide en réponse
Obtenu	200 OK + token JWT (vérifié dans jwt.io : payload valide, signature OK)
Statut	☒ PASS

Cas T02 — Création recette DRAFT

Élément	Valeur
Requête	POST /api/recipes (multipart/form-data)
Headers	Authorization: Bearer <token T01>
Body	Form : titre="Soupe de l'Eldorado", categoryId, mediaId (Indiana Jones), image .jpg 2 MB, ingrédients
Attendu	201 Created + { id, slug, status: "DRAFT" } + image convertie en .webp -70%
Obtenu	201 + image-recipe-1714.webp générée (684 KB → 198 KB), status DRAFT
Statut	☒ PASS

Cas T03 — Soumission DRAFT → PENDING

Élément	Valeur
Requête	PATCH /api/recipes/:id/submit
Headers	Authorization: Bearer <token T01>
Attendu	200 OK + status PENDING + notification admin créée
Obtenu	200 + status PENDING + notification visible dans /admin/notifications
Statut	☒ PASS

Cas T04 — Accès admin à la liste PENDING

Élément	Valeur
Requête	GET /api/admin/recipes/pending
Headers	Authorization: Bearer <token admin>
Attendu	200 OK + array contenant la recette T02/T03
Obtenu	200 + 1 recette PENDING "Soupe de l'Eldorado" présente
Statut	☒ PASS

Cas T05 — Validation admin → PUBLISHED

Élément	Valeur
Requête	PATCH /api/admin/recipes/:id/publish
Headers	Authorization: Bearer <token admin>
Attendu	200 OK + status PUBLISHED + notification créée côté membre
Obtenu	200 + status PUBLISHED + notification "Recette publiée" pour Marie
Statut	☒ PASS

Cas T06 — Réception notification membre

Élément	Valeur
Requête	GET /api/users/me/notifications
Headers	Authorization: Bearer <token Marie>
Attendu	200 OK + au moins 1 notification non lue "Recette publiée"
Obtenu	200 + 1 notification isRead:false, message "Votre recette a été publiée"
Statut	☒ PASS

Cas T07 — Suppression non autorisée d'une recette PUBLISHED

Élément	Valeur
Requête	DELETE /api/recipes/:id (token Marie, recette PUBLISHED)
Attendu	403 Forbidden (RG-03 : membre ne supprime pas PUBLISHED)
Obtenu	403 + message "Cette recette ne peut plus être supprimée"
Statut	☒ PASS

Cas T08 — Refus admin avec motif

Élément	Valeur
Requête	PATCH /api/admin/recipes/:id/reject
Body	{ "rejectionReason": "Photo de mauvaise qualité, veuillez la refaire." }
Attendu	200 OK + status DRAFT + notification membre avec motif visible
Obtenu	200 + status DRAFT + notification Marie contient le motif
Statut	☒ PASS

Cas T09 — Upload format invalide

Élément	Valeur
Requête	POST /api/recipes avec fichier .pdf au lieu d'image
Attendu	400 Bad Request (fileFilter Multer rejette les MIME non-image)
Obtenu	400 + message "Seuls les fichiers PNG, JPG, JPEG ou WEBP sont acceptes"
Statut	☒ PASS

Cas T10 — Accès route admin sans token

Élément	Valeur
Requête	GET /api/admin/recipes/pending (header Authorization absent)
Attendu	401 Unauthorized (authMiddleware rejette AVANT adminMiddleware)
Obtenu	401 + message "Token manquant ou mal formaté"
Statut	☒ PASS

Synthèse du jeu d'essai

Résultat : 10/10 cas PASS. Aucune régression détectée entre la version local Docker et la version Railway. Le workflow critique est conforme aux spécifications fonctionnelles (User Stories §2.4) et aux règles de gestion (RG-01 à RG-06).

Annexe 16 — Déploiement Railway, Docker, CI/CD

16.0 Accès aux fichiers de déploiement sur GitHub

Le pipeline CI/CD complet, le Dockerfile sur mesure pour l'API, le Caddyfile pour le client, le docker-compose.yml de dev local et tous les scripts de déploiement sont disponibles dans le dépôt code applicatif.

Fichiers de déploiement complets

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices/tree/main/.github/workflows

Workflow GitHub Actions ci.yml (4 jobs : lint-client + lint-api + test-client + test-api-integration). À la racine du dépôt : Dockerfile (node:22-slim + Prisma 7 + openssl), Caddyfile (cache différencié 1 an / 30 jours / no-cache), docker-compose.yml (PostgreSQL 18 + api + client en dev).

16.1 Workflow GitHub Actions complet — ci.yml

Le fichier .github/workflows/ci.yml définit le pipeline CI/CD à 4 jobs : lint-client, test-client, lint-api, test-api-integration. Le déploiement Railway est déclenché par webhook GitHub automatiquement à chaque push sur main du repo fork — pas besoin d'action GitHub dédiée pour le deploy.

```
# =====
# CI/CD - Cinés Délices
# =====
name: CI/CD - Cinés Délices

on:
  push:
    branches:
      - main
      - develop
  pull_request:
    branches:
      - main
      - develop

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

env:
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true

# -----
# JOB 1 - Lint Frontend
# -----
jobs:
  lint-client:
    name: 🐞 Lint - Client
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: client
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with:
          version: latest
      - uses: actions/setup-node@v4
```



```
    with:
      node-version: 24
      cache: pnpm
      cache-dependency-path: client/pnpm-lock.yaml
  - run: pnpm install --frozen-lockfile
  - run: pnpm lint
```

```
# -----
# JOB 2 – Tests unitaires Frontend (Vitest)
# -----
```

```
test-client:
  name: 🛠 Tests unitaires – Client (Vitest)
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: client
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v4
      with:
        version: latest
    - uses: actions/setup-node@v4
      with:
        node-version: 24
        cache: pnpm
        cache-dependency-path: client/pnpm-lock.yaml
    - run: pnpm install --frozen-lockfile
    - run: pnpm test:run
```

```
# -----
# JOB 3 – Lint Backend
# -----
```

```
lint-api:
  name: 🔍 Lint – API
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: api
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v4
      with:
        version: latest
    - uses: actions/setup-node@v4
      with:
        node-version: 24
        cache: pnpm
        cache-dependency-path: api/pnpm-lock.yaml
    - run: pnpm install --frozen-lockfile
    - run: pnpm lint
```

```
# -----
# JOB 4 – Tests d'intégration API (Railway)
# Lancé UNIQUEMENT sur push (pas sur les PRs)
# -----
```

```
test-api-integration:
  name: 🚂 Tests d'intégration – API (Railway)
  runs-on: ubuntu-latest
  needs: [lint-client, test-client, lint-api]
```

```

if: github.event_name == 'push'
defaults:
  run:
    working-directory: api
steps:
  - uses: actions/checkout@v4

  - uses: pnpm/action-setup@v4
    with:
      version: latest

  - uses: actions/setup-node@v4
    with:
      node-version: 24
      cache: pnpm
      cache-dependency-path: api/pnpm-lock.yaml

  - name: Installer les dépendances
    run: pnpm install --frozen-lockfile

  - name: Attendre stabilisation Railway (30s)
    run: sleep 30

  - name: Générer le client Prisma
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }
    run: |
      touch .env
      npx prisma generate
    continue-on-error: true

  - name: Debug DATABASE_URL
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }
    run: |
      if [ -z "$DATABASE_URL" ]; then
        echo "❌ DATABASE_URL est VIDE"
      else
        echo "✅ DATABASE_URL défini (${#DATABASE_URL} chars), début:
        ${DATABASE_URL:0:20}..."
      fi

  # Si besoin de relancer le seed
  - name: Rejouer le seed (reset données de test)
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }
      API_BASE_URL: ${ secrets.API_BASE_URL }
    run: |
      touch .env
      node prisma/seed-v4.js
    continue-on-error: true

  - name: Lancer les tests d'intégration API
    env:
      API_BASE_URL: ${ secrets.API_BASE_URL || 'https://cines-delicesapi-
production.up.railway.app' }
    run: node tests/test-api.js

  - name: Lancer les tests de sécurité API

```

```
env:
  API_BASE_URL: ${ secrets.API_BASE_URL || 'https://cines-delicesapi-
production.up.railway.app' }}
run: node tests/test-api-security.js
```

16.2 Dockerfile API — node:22-slim + Prisma 7

Nixpacks détectait une version Node 18 sans openssl, incompatible avec Prisma 7 (Node 22 requis pour les binaires openssl Prisma 7). Solution : Dockerfile sur mesure node:22-slim + apt install openssl + prisma generate dans l'image (sinon le Client Prisma n'est pas dispo au boot).

```
FROM node:22-slim
RUN apt-get update && apt-get install -y openssl && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --ignore-workspace --ignore-scripts
COPY . .
RUN node_modules/.bin/prisma generate
EXPOSE 8080
CMD ["node", "src/app.js"]
```

16.3 Caddyfile client — cache différencié

Caddy sert le bundle Vite avec trois stratégies de cache distinctes : 1 an pour les assets hashés (immuables car le hash change à chaque build), 30 jours pour les images statiques, no-cache pour le HTML (pour que le déploiement soit immédiatement visible).

```
:{ $PORT } {
  root * /app/dist
  encode gzip

  # Assets avec hash dans le nom (JS, CSS générés par Vite) → cache 1 an
  # Le hash change automatiquement à chaque build → jamais de stale cache
  @hashedAssets {
    path /assets/*
  }
  header @hashedAssets Cache-Control "public, max-age=31536000, immutable"

  # Images et icônes statiques → cache 30 jours
  @staticImages {
    path /img/* /icon/* /favicon.svg
  }
  header @staticImages Cache-Control "public, max-age=2592000"

  # HTML → jamais en cache (toujours la dernière version)
  @html {
    path *.html /
  }
  header @html Cache-Control "no-cache, no-store, must-revalidate"

  # SPA — renvoie index.html pour toutes les routes React Router
  try_files {path} /index.html

  file_server
}
```

16.4 docker-compose.yml — environnement de développement

Trois services pour le dev local : PostgreSQL 18-alpine (avec healthcheck), api Node 24-alpine (lance prisma generate + db push au démarrage), client Vite (avec proxy vers l'api). Les volumes node_modules sont nommés pour éviter le conflit hôte/conteneur.

```
services:
  api:
    image: node:24-alpine
    working_dir: /app
    volumes:
      - ./api:/app
      - api_node_modules:/app/node_modules
    env_file:
      - ./api/.env
    environment:
      DATABASE_URL: postgresql://cinesdelices:cinesdelices@db:5432/cinesdelices
    ports:
      - "3000:3000"
    depends_on:
      db:
        condition: service_healthy
    command: sh -c "npm install && npx prisma generate && npx prisma db push && npm run dev"

  client:
    image: node:24-alpine
    working_dir: /app
    volumes:
      - ./client:/app
      - client_node_modules:/app/node_modules
    environment:
      VITE_API_PROXY_TARGET: http://api:3000
    ports:
      - "5173:5173"
    command: sh -c "npm install && npm run dev -- --host"

  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_USER: cinesdelices
      POSTGRES_PASSWORD: cinesdelices
      POSTGRES_DB: cinesdelices
    ports:
      - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U cinesdelices"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  api_node_modules:
  client_node_modules:
  db_data:
```

16.5 Variables d'environnement Railway — checklist

Aucun secret n'est commis : toutes les configurations sensibles passent par les variables Railway, injectées au build (Vite) ou au runtime (Node.js).

Variable	Service	Rôle et point d'attention
DATABASE_URL	api	URL interne PostgreSQL — générée automatiquement par Railway, jamais exposée publiquement.
JWT_SECRET	api	64 caractères aléatoires générés en local, jamais dans le code source.
TMDB_API_KEY	api	Clé v3 TMDB — jamais exposée au client (utilisée uniquement via proxy serveur).
API_BASE_URL	api	URL publique de l'API — utilisée par les tests d'intégration CI/CD et la documentation Swagger.
CLIENT_URL	api	Origine autorisée par CORS — empêche les appels d'origines non listées.
SMTP_HOST / PORT / USER / PASSWORD	api	Configuration SMTP Brevo en production (jamais en dur).
VITE_API_URL	client	⚠ doit être définie AVANT le build — Vite l'injecte à la compilation, pas au runtime.

16.6 Checklist déploiement pas-à-pas

Ordre opérationnel exact des opérations effectuées le 28 avril 2026 pour le déploiement de production.

- Provisionner la base PostgreSQL Railway et récupérer son DATABASE_URL interne.
- Configurer les 6 variables d'environnement côté api (DATABASE_URL, JWT_SECRET 64 chars, TMDB_API_KEY, API_BASE_URL, CLIENT_URL, SMTP_*).
- Configurer VITE_API_URL côté client AVANT le premier build (sinon Vite ne l'injecte pas dans le bundle).
- Push sur main du fork Nathalie-Leduc/cines-delices — Railway déclenche automatiquement le build via webhook.
- Exécuter prisma migrate deploy depuis la console Railway api.
- Exécuter le seed v4 (compte admin + 16 médias TMDB + catégories + ingrédients).
- Vérifier /api/health (doit retourner {"status":"ok"}) et la connexion d'un compte de test.
- Lancer la suite test-api-integration en local contre l'URL Railway pour valider.

16.7 Problèmes résolus durant la mise en production

Problème	Cause identifiée	Solution appliquée
Prisma 7 incompatible avec Node Nixpacks	Nixpacks détectait Node 18 sans openssl, incompatible avec Prisma 7	Dockerfile sur mesure (node:22-slim + apt install openssl + prisma generate dans l'image)

Problème	Cause identifiée	Solution appliquée
Migrations Prisma P3005	BDD Railway désynchronisée — colonnes manquantes	Reset DB de prod + replay intégral de prisma migrate deploy
VITE_API_URL non injectée en prod	Variable Railway ajoutée APRÈS le premier build	Rebuild complet via commit vide (git commit --allow-empty) + vérification du bundle
Push poussé sur la mauvaise branche	Railway écoutait dev au lieu de main → VITE_API_URL pas relue	Correction de la branche source dans la configuration du service Railway
Images non affichées en production	URLs absolues stockées en BDD, cassées dès qu'on change de domaine API ou que API_BASE_URL est absente	Refactor vers des chemins relatifs en BDD + préfixage côté client via buildApiAssetUrl (services/api.js)
CI : pnpm action ordering	setup-node lancé avant pnpm/action-setup, cache pnpm introuvable	Inversion de l'ordre : pnpm/action-setup AVANT actions/setup-node@v4 + cache: pnpm

Annexe 17 — Captures d'écran : création d'une recette

17.1 Correspondance étape ↔ capture (cf. §7 du corps)

Cette annexe est le complément visuel du §7 du corps — la fonctionnalité phare détaillée en sept étapes. Les captures du flux complet (formulaire de saisie, autocomplétion TMDB, image WebP, boutons brouillon / soumission, message de succès, recette listée en DRAFT ou PENDING) sont disponibles sur le dépôt de documentation.

Disponible sur GitHub — [cines-delices-docs/Captures-creation-recette/](https://github.com/Nathalie-Leduc/cines-delices-docs/Captures-creation-recette/)

Cliquez sur le lien :

github.com/Nathalie-Leduc/cines-delices-docs/Capture-ecran

Captures prises sur la version Railway de production, scénario « Ratatouille de Rémy » (film 2007, 6 ingrédients, 8 étapes), modes brouillon et soumission pour comparaison. Correspondance : étape du diagramme ↔ capture ↔ extrait de code (cf. §7).